

```

*   pane did neither, so on every switch the canvas "corrected" the view.
*   This pane now DISPLAYS the stored camera through the same rules, for its
*   own size, and writes the result back - so the canvas has nothing left to
*   correct when it takes over.
* 3. Different drawing areas. A header strip and a footer strip took height
*   from this pane that the live canvas, in the same slot, did not lose. With
*   the camera being the world point at the CENTRE of the pane, that alone
*   shifted the page on every switch. The chrome is now floating, so both
*   draw into the full slot.
*
* And the page you are reading here is the notebook's OWN active page, not a
* private counter - otherwise tapping to write on page 5 put you on page 1.
*/

import React from "react";
import { ArrowLeftRight, ArrowUpDown, ChevronLeft, ChevronRight, Columns2, PenLine, Rows2, X } from "lucide-react";
import { useWhiteboard, useActiveBoard } from "@store/whiteboard";
import {
  clampNotebookCamera,
  getActivePageIndex,
  getPages,
  isNotebook,
  minNotebookZoom,
  PAGE_GAP,
  pageSizeOf,
  paperForPage,
  reanchorDirection,
  visibleSheets,
} from "@lib/whiteboard/notebook";
import { getUnionBounds } from "@lib/whiteboard/geometry";
import {
  BOARD_BACKGROUND,
  BOARD_PATTERN_MAX_SIZE,
  BOARD_PATTERN_MIN_SIZE,
  type Board,
  type Camera,
} from "@lib/whiteboard/types";
import { GRID_SIZE } from "@lib/whiteboard/grid";
import { handOff, handOffPress } from "@lib/whiteboard/pointerHandoff";
import { useTheme } from "next-themes";
import { ElementRenderer, StrokeFilterDefs } from "../ElementRenderer";
import { PdfPageLayer } from "../PdfPageLayer";
import { PaperTiles, paperColorFor } from "../PaperPattern";
import { cn } from "@lib/utls";
import { dividerRatio } from "@lib/whiteboard/split";

const DEFAULT_CAMERA: Camera = { x: 0, y: 0, zoom: 1 };

/**
 * The camera a notebook is SHOWN with in a pane of this size: the stored one,
 * put through exactly the rules the live canvas enforces when it settles.
 * Pure, so the same function answers for both panes.
 */
export function displayCameraFor(
  board: Board | null,
  stored: Camera,
  size: { width: number; height: number }
): Camera {
  if (!board || !isNotebook(board) || !size.width || !size.height) return stored;
  const dim = pageSizeOf(board);
  const floor = minNotebookZoom(size, dim);
  const zoom = floor > 0 && stored.zoom < floor - 1e-3 ? floor : stored.zoom;
  const { x, y } = clampNotebookCamera({ ...stored, zoom }, size, dim, {
    pageCount: getPages(board).length,
    activeIndex: getActivePageIndex(board),
  });
  return { x, y, zoom };
}

// -----
// The chip each pane wears
// -----

/**
 * Which document this is, which page, and - above all - whether it is the one
 * your tools act on. Floating, so it costs the drawing area nothing.
 */

```

```

export function SplitPaneChip({
  board,
  live,
  onPrevPage,
  onNextPage,
  children,
}): {
  board: Board;
  live: boolean;
  onPrevPage?: () => void;
  onNextPage?: () => void;
  children?: React.ReactNode;
}) {
  const notebook = isNotebook(board);
  const pages = getPages(board);
  const index = getActivePageIndex(board);
  return (
    <div
      data-split-chip={live ? "live" : "other"}
      className={cn(
        "pointer-events-auto flex h-8 max-w-[calc(100%-1.5rem)] items-center gap-1 rounded-full border pl-2.5
        pr-1 text-xs shadow-md backdrop-blur-md transition-colors",
        // Blue, not the theme's "primary": in this theme primary is near
        // black, and "live" has to read at a glance from across the table.
        live
          ? "border-blue-500 bg-blue-600 text-white"
          : "border-border bg-white/90 text-foreground dark:bg-zinc-900/90"
      )}
    >
      <span
        className={cn(
          "h-2 w-2 shrink-0 rounded-full",
          live ? "bg-white animate-pulse" : "bg-muted-foreground/50"
        )}
        aria-hidden
      />
      <span className="min-w-0 truncate font-semibold" title={board.name}>
        {board.name}
      </span>
      <span className={cn("shrink-0 text-[10px] font-medium", live ? "opacity-80" : "text-muted-foreground")}>
        {live ? "Editing" : "Tap to edit"}
      </span>

      {notebook && pages.length > 1 && (
        <span className="ml-0.5 flex shrink-0 items-center">
          {onPrevPage && (
            <button
              type="button"
              onClick={onPrevPage}
              disabled={index <= 0}
              aria-label={`"${board.name}": previous page`}
              className={cn(
                "flex h-6 w-6 items-center justify-center rounded-full disabled:pointer-events-none disabled:opacity-35",
                live ? "hover:bg-white/20" : "hover:bg-muted"
              )}
            >
              <ChevronLeft size={14} />
            </button>
          )}
          <span className="min-w-[3.25rem] text-center text-[11px] tabular-nums">
            {index + 1} / {pages.length}
          </span>
          {onNextPage && (
            <button
              type="button"
              onClick={onNextPage}
              disabled={index >= pages.length - 1}
              aria-label={`"${board.name}": next page`}
              className={cn(
                "flex h-6 w-6 items-center justify-center rounded-full disabled:pointer-events-none disabled:opacity-35",
                live ? "hover:bg-white/20" : "hover:bg-muted"
              )}
            >
              <ChevronRight size={14} />
            </button>
          )}
        </span>
      )}
    </div>
  );
}

```

```

        </span>
      )}
    {children}
  </div>
);
}

/**
 * The live pane's chip and frame, mounted over the real canvas. The frame is
 * what tells you, at a glance, which half your pen, colour, paper and settings
 * are about to change.
 */
export function SplitLiveOverlay() {
  const splitBoardId = useWhiteboard((s) => s.splitBoardId);
  const board = useActiveBoard();
  if (!splitBoardId || !board) return null;
  return (
    <>
      <div
        aria-hidden
        data-split-live-frame
        className="pointer-events-none absolute inset-0 z-[35]"
        // Inline on purpose. The frame IS the answer to "which side am I
        // editing", and a Tailwind arbitrary ring (ring-[3px]) was silently
        // dropped from the build - verified at box-shadow: none. A plain style
        // cannot be.
        style={{ boxShadow: "inset 0 0 0 3px rgb(59 130 246)" }}
      />
      <div className="pointer-events-none absolute left-3 top-3 z-40 flex w-1/2">
        <SplitPaneChip board={board} live />
      </div>
    </>
  );
}

// -----
// The other pane
// -----

export function SplitPane() {
  const splitBoardId = useWhiteboard((s) => s.splitBoardId);
  const setSplitBoard = useWhiteboard((s) => s.setSplitBoard);
  const swapSplit = useWhiteboard((s) => s.swapSplit);
  const boards = useWhiteboard((s) => s.boards);
  const splitSide = useWhiteboard((s) => s.splitSide);
  const stacked = useWhiteboard((s) => s.splitOrientation === "stacked");
  const setSplitOrientation = useWhiteboard((s) => s.setSplitOrientation);
  const focusSplitSide = useWhiteboard((s) => s.focusSplitSide);
  const setBoardCamera = useWhiteboard((s) => s.setBoardCamera);
  const reanchorBoardPage = useWhiteboard((s) => s.reanchorBoardPage);
  const goToBoardPage = useWhiteboard((s) => s.goToBoardPage);
  const { resolvedTheme } = useTheme();

  // This pane shows the document that is NOT live, so it sits on the side the
  // live one is not.
  const mySide = splitSide === "left" ? "right" : "left";
  const board = boards.find((b) => b.id === splitBoardId) ?? null;

  const containerRef = React.useRef<HTMLDivElement>(null);
  const [size, setSize] = React.useState({ width: 0, height: 0 });
  React.useEffect(() => {
    const node = containerRef.current;
    if (!node) return;
    const ro = new ResizeObserver(() => setSize({ width: node.clientWidth, height: node.clientHeight }));
    ro.observe(node);
    setSize({ width: node.clientWidth, height: node.clientHeight });
    return () => ro.disconnect();
  }, []);

  const notebook = isNotebook(board);
  const pages = getPages(board);
  const activeIndex = getActivePageIndex(board);
  const dim = pageSizeOf(board);
  const stride = dim.height + PAGE_GAP;

  const stored = board?.camera ?? DEFAULT_CAMERA;
  const camera = displayCameraFor(board, stored, size);

```

```

// Write the settled camera back, so the store holds exactly what is on
// screen and the canvas finds nothing to correct when this side goes live.
// Half a world unit of drift is invisible; skipping it avoids a render loop.
React.useEffect(() => {
  if (!board || !notebook || !size.width) return;
  if (
    Math.abs(camera.x - stored.x) > 0.5 ||
    Math.abs(camera.y - stored.y) > 0.5 ||
    Math.abs(camera.zoom - stored.zoom) > 1e-3
  ) {
    setBoardCamera(board.id, camera);
  }
}, [board, notebook, size.width, camera.x, camera.y, camera.zoom, stored.x, stored.y, stored.zoom, setBoardCamera]);

// Continuous scrolling, the canvas's own rule: past the middle of the gap,
// the next page takes the anchor and the camera slides one stride to match.
React.useEffect(() => {
  if (!board || !notebook) return;
  const dir = reanchorDirection(camera.y, dim.height, activeIndex, pages.length);
  if (dir !== 0) reanchorBoardPage(board.id, dir, stride);
}, [board, notebook, camera.y, dim.height, activeIndex, pages.length, stride, reanchorBoardPage]);

/**
 * Frame a canvas that has never been looked at. A canvas's drawing sits
 * wherever it was drawn, so an untouched camera can show an empty pane.
 * Notebooks never need this: the clamp above already keeps the paper in
 * view. Runs only when NONE of the content is visible - refitting whenever
 * it opened is what used to throw away the position you had left it at.
 */
const fittedFor = React.useRef<string | null>(null);
React.useEffect(() => {
  if (!board || !notebook || !size.width || !size.height) return;
  if (fittedFor.current === board.id) return;
  fittedFor.current = board.id;
  const bounds = getUnionBounds(board.elements ?? []);
  if (!bounds || bounds.width <= 0 || bounds.height <= 0) return;
  const halfW = size.width / 2 / Math.max(stored.zoom, 0.01);
  const halfH = size.height / 2 / Math.max(stored.zoom, 0.01);
  const visible =
    bounds.x < stored.x + halfW &&
    bounds.x + bounds.width > stored.x - halfW &&
    bounds.y < stored.y + halfH &&
    bounds.y + bounds.height > stored.y - halfH;
  if (visible) return;
  const pad = 40;
  const zoom = Math.max(
    0.05,
    Math.min(2, Math.min((size.width - pad) / bounds.width, (size.height - pad) / bounds.height))
  );
  setBoardCamera(board.id, { x: bounds.x + bounds.width / 2, y: bounds.y + bounds.height / 2, zoom });
}, [board, notebook, size.width, size.height, stored.x, stored.y, stored.zoom, setBoardCamera]);

// Scrolling this side makes it the live one - the note you last acted on is
// the note your settings apply to. The wheel is handed to the canvas that
// takes over, so the scroll obeys the canvas's own rules rather than a copy.
React.useEffect(() => {
  const node = containerRef.current;
  if (!node) return;
  const onWheel = (e: WheelEvent) => {
    e.preventDefault();
    const { clientX, clientY, deltaX, deltaY, deltaZ, deltaMode, ctrlKey, metaKey, shiftKey, altKey } = e;
    focusSplitSide(mySide);
    handOff(
      () =>
        new WheelEvent("wheel", {
          bubbles: true,
          cancelable: true,
          clientX,
          clientY,
          deltaX,
          deltaY,
          deltaZ,
          deltaMode,
          ctrlKey,
          metaKey,
          shiftKey,
          altKey,

```

```

    }},
    clientX,
    clientY
  );
};
node.addEventListener("wheel", onWheel, { passive: false });
return () => node.removeEventListener("wheel", onWheel);
}, [focusSplitSide, mySide]);

if (!board) return null;

const goPage = (delta: number) => {
  const target = pages[activeIndex + delta];
  if (!target) return;
  // Turning a page here is acting on this note, so it becomes the live one -
  // and it opens on the page you just turned to.
  goToBoardPage(board.id, target.id, size.height);
  focusSplitSide(mySide);
};

const canvasPaper = board.background;
const canvasPattern = canvasPaper?.pattern ?? "dots";
const canvasPaperSize = Math.min(
  BOARD_PATTERN_MAX_SIZE,
  Math.max(BOARD_PATTERN_MIN_SIZE, canvasPaper?.size ?? GRID_SIZE)
);
const canvasColor = (() => {
  const preset = BOARD_BACKGROUNDS.find((b) => b.id === canvasPaper?.color);
  if (preset) return resolvedTheme === "dark" ? preset.dark : preset.light;
  return canvasPaper?.color || "";
})();

const halfWorldH = size.height / 2 / Math.max(camera.zoom, 0.01);
const sheets = notebook ? visibleSheets(pages.length, activeIndex, dim.height, camera.y, halfWorldH) : [];
const onlySheetEmpty =
  notebook && pages.length === 1 && (pages[0]?.elements?.length ?? 0) === 0 && !pages[0]?.pdf;

return (
  <div className="relative h-full min-w-0 overflow-hidden bg-muted/30">
    <div
      ref={containerRef}
      data-split-other
      className="absolute inset-0 cursor-pointer overflow-hidden"
      style={{ touchAction: "none" }}
      // Touching this side hands it the edit focus AND delivers the touch to
      // the canvas that takes over, so a pen stroke begins under the nib and
      // a finger drag pans - whatever that input does on the live side.
      onPointerDown={(e) => {
        const { clientX, clientY, pointerId, pointerType, button, buttons, pressure, isPrimary } = e;
        focusSplitSide(mySide);
        handOffPress(
          { clientX, clientY, pointerId, pointerType, button, buttons, pressure, isPrimary },
          clientX,
          clientY
        );
      }}
    >
    <svg width="100%" height="100%" className="block">
      <StrokeFilterDefs />

      {/* A canvas has no sheet: its paper fills the pane in screen space,
        offset by the camera so the ruling moves with the drawing. */}
      {!notebook && (
        <>
          <defs>
            <pattern
              id={`split-canvas-paper-${board.id}`}
              x={size.width / 2 - camera.x * camera.zoom}
              y={size.height / 2 - camera.y * camera.zoom}
              width={canvasPaperSize * camera.zoom}
              height={canvasPaperSize * camera.zoom}
              patternUnits="userSpaceOnUse"
            >
              <g transform={`scale(${camera.zoom})`}>
                <PaperTiles pattern={canvasPattern} size={canvasPaperSize} />
              </g>
            </pattern>
          </defs>

```

```

    {canvasColor && <rect width="100%" height="100%" fill={canvasColor} />}
    {canvasPattern !== "plain" && (
      <rect width="100%" height="100%" fill={`url(#split-canvas-paper-${board.id})`} />
    )}
  </>
)}

<g
  transform={`translate(${size.width / 2}, ${size.height / 2}) scale(${camera.zoom}) translate(${-camera.x}, ${-camera.y})`}
>
  {/* Every visible sheet, each in its OWN paper - the same strip the
    live canvas draws, so switching sides changes nothing. */}
  {sheets.map(({ index, offsetY }) => {
    const pg = pages[index];
    if (!pg) return null;
    const paper = paperForPage(board, pg);
    const color = paperColorFor(pg.background ?? board.background, resolvedTheme);
    const patternId = `split-paper-${board.id}-${pg.id}`;
    return (
      <g key={pg.id} transform={`translate(0, ${offsetY})`} pointerEvents="none">
        <defs>
          <pattern
            id={patternId}
            x={0}
            y={0}
            width={paper.size}
            height={paper.size}
            patternUnits="userSpaceOnUse"
          >
            <PaperTiles pattern={paper.pattern} size={paper.size} />
          </pattern>
        </defs>
        <rect
          x={3}
          y={4}
          width={dim.width}
          height={dim.height}
          fill={resolvedTheme === "dark" ? "#000000" : "#a1a1aa"}
          opacity={resolvedTheme === "dark" ? 0.55 : 0.28}
          rx={2}
        />
        <rect
          x={0}
          y={0}
          width={dim.width}
          height={dim.height}
          fill={color || (resolvedTheme === "dark" ? "#18181b" : "#ffffff")}
          stroke={resolvedTheme === "dark" ? "#3f3f46" : "#d4d4d8"}
          strokeWidth={1 / camera.zoom}
          rx={2}
        />
        {pg.pdf && <PdfPageLayer pdf={pg.pdf} width={dim.width} height={dim.height} />}
        {paper.pattern !== "plain" && (
          <rect x={0} y={0} width={dim.width} height={dim.height} fill={`url(#${patternId})`} />
        )}
        {(pg.elements ?? []).map((el) => (
          <ElementRenderer key={el.id} element={el} isSelected={false} />
        ))}
      </g>
    );
  })}

  {!notebook &&
    (board.elements ?? []).map((el) => (
      <ElementRenderer key={el.id} element={el} isSelected={false} />
    ))}
</g>
</svg>

{onlySheetEmpty && (
  <p className="pointer-events-none absolute inset-x-0 bottom-16 text-center text-[11px] text-muted-foreground">
    Empty page – tap to start writing
  </p>
)}
</div>

```

```

    {/* Chrome floats: it must not take height the live canvas would not lose. */}
    <div className="pointer-events-none absolute left-3 right-3 top-3 z-10 flex items-start justify-between gap-2">
      <SplitPaneChip
        board={board}
        live={false}
        onPrevPage={() => goPage(-1)}
        onNextPage={() => goPage(1)}
      />
      <div className="pointer-events-auto flex shrink-0 items-center gap-1 rounded-full border border-border bg-white/90 p-0.5 shadow-md backdrop-blur-md dark:bg-zinc-900/90">
        <button
          type="button"
          onClick={() => focusSplitSide(mySide)}
          title="Write on this side"
          aria-label="Edit this pane"
          className="flex h-7 w-7 items-center justify-center rounded-full text-muted-foreground hover:bg-muted hover:text-foreground"
        >
          <PenLine size={14} />
        </button>
        <button
          type="button"
          onClick={swapSplit}
          title={stacked ? "Swap top and bottom" : "Swap left and right"}
          aria-label="Swap panes"
          className="flex h-7 w-7 items-center justify-center rounded-full text-muted-foreground hover:bg-muted hover:text-foreground"
        >
          {stacked ? <ArrowUpDown size={14} /> : <ArrowLeftRight size={14} />}
        </button>
        <button
          type="button"
          onClick={() => setSplitOrientation(stacked ? "side" : "stacked")}
          title={stacked ? "Show side by side" : "Show one above the other"}
          aria-label={stacked ? "Side by side" : "Top and bottom"}
          className="flex h-7 w-7 items-center justify-center rounded-full text-muted-foreground hover:bg-muted hover:text-foreground"
        >
          {stacked ? <Columns2 size={14} /> : <Rows2 size={14} />}
        </button>
        <button
          type="button"
          onClick={() => setSplitBoard(null)}
          title="Close split view"
          aria-label="Close split view"
          className="flex h-7 w-7 items-center justify-center rounded-full text-muted-foreground hover:bg-muted hover:text-foreground"
        >
          <X size={14} />
        </button>
      </div>
    </div>
  </div>
);
}

/**
 * The divider between the two panes: a vertical bar side by side, a
 * horizontal one when stacked. A 6px line with a wide grab area and a grip,
 * because a hairline cannot be hit with a finger. Double-tap evens it out.
 */
export function SplitDivider({
  containerRef,
  stacked = false,
  style,
}: {
  containerRef: React.RefObject<HTMLDivElement | null>;
  stacked?: boolean;
  style?: React.CSSProperties;
}) {
  const setSplitRatio = useWhiteboard((s) => s.setSplitRatio);
  const [dragging, setDragging] = React.useState(false);
  const lastTap = React.useRef(0);

  React.useEffect(() => {
    if (!dragging) return;
    // One resize per frame. The pointer reports far more often than that, and

```

```

// every resize re-lays-out both panes and the canvas - dragging used to
// stutter because each report did all of that (and a save) on its own.
let frame = 0;
let pending: number | null = null;
const flush = () => {
  frame = 0;
  if (pending !== null) setSplitRatio(pending);
  pending = null;
};
const onMove = (e: PointerEvent) => {
  const node = containerRef.current;
  if (!node) return;
  const box = node.getBoundingClientRect();
  pending = stacked
    ? dividerRatio(e.clientY - box.top, box.height)
    : dividerRatio(e.clientX - box.left, box.width);
  if (!frame) frame = requestAnimationFrame(flush);
};
const onUp = () => {
  if (frame) cancelAnimationFrame(frame);
  flush();
  setDragging(false);
};
window.addEventListener("pointermove", onMove);
window.addEventListener("pointerup", onUp);
window.addEventListener("pointercancel", onUp);
return () => {
  if (frame) cancelAnimationFrame(frame);
  window.removeEventListener("pointermove", onMove);
  window.removeEventListener("pointerup", onUp);
  window.removeEventListener("pointercancel", onUp);
};
}, [dragging, containerRef, setSplitRatio, stacked]);

return (
  <div
    role="separator"
    aria-orientation={stacked ? "horizontal" : "vertical"}
    aria-label="Resize split view (double-tap to even it out)"
    onPointerDown={(e) => {
      e.preventDefault();
      const now = Date.now();
      if (now - lastTap.current < 350) {
        lastTap.current = 0;
        setSplitRatio(0.5);
        return;
      }
      lastTap.current = now;
      setDragging(true);
    }}
    style={{ ...style, touchAction: "none" }}
    className={cn(
      "group relative z-[60] shrink-0 bg-border",
      stacked ? "h-1.5 w-full cursor-row-resize" : "w-1.5 cursor-col-resize"
    )}
  >
    { /* A wider invisible hit area than the line itself. */ }
    <span className={cn("absolute", stacked ? "inset-x-0 -top-2.5 -bottom-2.5" : "inset-y-0 -left-2.5 -right-2.5")} aria-hidden />
    <span
      aria-hidden
      className={cn(
        "absolute left-1/2 top-1/2 -translate-x-1/2 -translate-y-1/2 rounded-full transition-colors",
        stacked ? "h-1.5 w-12" : "h-12 w-1.5",
        dragging ? "bg-blue-500" : "bg-muted-foreground/40 group-hover:bg-blue-500"
      )}
    />
  </div>
);
}

```

```

=====
// === END FILE: src/components/whiteboard/SplitPane.tsx ===
=====

```

```

=====
// === START FILE: src/components/whiteboard/SplitPicker.tsx ===

```

```
// File 80 of 194 | Lines: 242 | Size: 9322 bytes
```

```
=====

"use client";

/**
 * "Split view" button and its picker.
 *
 * Both sides are chosen here. The earlier version only let you pick the RIGHT
 * document – the left was silently whatever you already had open – so pairing
 * two particular documents meant opening one first and then finding the other
 * in the sidebar, and nothing on screen said so.
 *
 * Each row also says whether the document has anything in it. Without that,
 * picking a blank board looks identical to the split being broken.
 */

import React from "react";
import { Columns2, Rows2, X } from "lucide-react";
import { useWhiteboard, useActiveBoard } from "@store/whiteboard";
import { allElements, getPages, isNotebook } from "@lib/whiteboard/notebook";
import type { Board, SplitOrientation } from "@lib/whiteboard/types";
import { Button } from "@components/ui/button";
import { Popover, PopoverContent, PopoverTrigger } from "@components/ui/popover";
import { cn } from "@lib/utils";

/** "12 items" / "4 pages, 2 drawn on" / "empty" – enough to pick by. */
function describe(b: Board): { text: string; empty: boolean } {
  if (isNotebook(b)) {
    const pages = getPages(b);
    const withContent = pages.filter(
      (p) => (p.elements?.length ?? 0) > 0 || p.pdf
    ).length;
    if (withContent === 0) {
      return { text: `${pages.length} blank page${pages.length === 1 ? "" : "s"}`, empty: true };
    }
    return { text: `${pages.length}p · ${withContent} with content`, empty: false };
  }
  const n = allElements(b).length;
  return { text: n === 0 ? "empty canvas" : `canvas · ${n} item${n === 1 ? "" : "s"}`, empty: n === 0 };
}

function BoardList({
  boards,
  selectedId,
  disabledId,
  onPick,
}: {
  boards: Board[];
  selectedId: string | null;
  disabledId: string | null;
  onPick: (id: string) => void;
}) {
  return (
    <div className="max-h-[168px] space-y-0.5 overflow-y-auto">
      {boards.map((b) => {
        const d = describe(b);
        const blocked = b.id === disabledId;
        return (
          <button
            key={b.id}
            onClick={() => !blocked && onPick(b.id)}
            disabled={blocked}
            title={blocked ? "Already on the other side" : b.name}
            className={cn(
              "flex w-full items-baseline gap-2 rounded-lg px-2 py-1 text-left transition-colors",
              b.id === selectedId
                ? "bg-primary/10 text-primary"
                : "text-foreground hover:bg-muted",
              blocked && "cursor-not-allowed opacity-35 hover:bg-transparent"
            )}
          >
            <span className="min-w-0 flex-1 truncate text-[11px]">{b.name}</span>
            <span
              className={cn(
                "shrink-0 text-[9px]",
                d.empty ? "text-muted-foreground/60 italic" : "text-muted-foreground"
              )}
            />
          </button>
        )
      })}
    </div>
  )
}
```

```

    >
    {d.text}
  </span>
</button>
);
}}
</div>
);
}

export function SplitPicker() {
  const boards = useWhiteboard((s) => s.boards);
  const activeBoard = useActiveBoard();
  const splitBoardId = useWhiteboard((s) => s.splitBoardId);
  const splitSide = useWhiteboard((s) => s.splitSide);
  const setSplitBoard = useWhiteboard((s) => s.setSplitBoard);
  const setSplitPair = useWhiteboard((s) => s.setSplitPair);
  const orientation = useWhiteboard((s) => s.splitOrientation);
  const setSplitOrientation = useWhiteboard((s) => s.setSplitOrientation);
  const [open, setOpen] = React.useState(false);
  // The layout is chosen FIRST, together with the documents, and can be
  // changed later from the pane's own buttons.
  const [pendingLayout, setPendingLayout] = React.useState<SplitOrientation>("side");

  const splitOpen = Boolean(splitBoardId);

  // Which document is physically on each side right now.
  const leftId = splitSide === "left" ? activeBoard?.id ?? null : splitBoardId;
  const rightId = splitSide === "left" ? splitBoardId : activeBoard?.id ?? null;

  // Staged choices, so picking a left that is already on the right does not
  // half-apply and leave the split in a state you did not ask for.
  const [pendingLeft, setPendingLeft] = React.useState<string | null>(null);
  const [pendingRight, setPendingRight] = React.useState<string | null>(null);

  React.useEffect(() => {
    if (open) {
      setPendingLeft(leftId ?? activeBoard?.id ?? null);
      setPendingRight(rightId);
      // An open split keeps its layout; a new one starts in the layout that
      // suits the screen - stacked on a tablet held upright.
      const portrait = typeof window !== "undefined" && window.innerHeight > window.innerWidth;
      setPendingLayout(splitOpen ? orientation : portrait ? "stacked" : orientation);
    }
    // eslint-disable-next-line react-hooks/exhaustive-deps
  }, [open]);

  const canApply =
    pendingLeft && pendingRight && pendingLeft !== pendingRight;

  return (
    <Popover open={open} onOpenChange={setOpen}>
      <PopoverTrigger asChild>
        <Button
          variant="ghost"
          size="icon"
          className={cn(
            "h-8 w-8 text-muted-foreground hover:text-foreground",
            splitOpen && "bg-primary/10 text-primary"
          )}
          title="Split view – two documents at once"
          aria-label="Split view"
        >
          {splitOpen && orientation === "stacked" ? <Rows2 size={15} /> : <Columns2 size={15} />}
        </Button>
      </PopoverTrigger>

      <PopoverContent align="end" className="w-80 p-2">
        <p className="mb-2 px-1 text-[12px] font-semibold">Split view</p>

        {/* Layout first: it decides what the two lists below are called. */}
        <div className="mb-2 grid grid-cols-2 gap-1 rounded-xl bg-muted/60 p-1" role="radiogroup" aria-label="Layout">
          {
            [
              { id: "side" as const, label: "Side by side", icon: Columns2 },
              { id: "stacked" as const, label: "Top & bottom", icon: Rows2 },
            ]
          }
        </div>
      </PopoverContent>
    </Popover>
  );
}

```

```

).map(({ id, label, icon: Icon }) => (
  <button
    key={id}
    type="button"
    role="radio"
    aria-checked={pendingLayout === id}
    onClick={() => {
      setPendingLayout(id);
      // With a split already open the layout applies at once: it
      // changes nothing about WHICH documents are shown.
      if (splitOpen) setSplitOrientation(id);
    }}
    className={cn(
      "flex flex-col items-center gap-1 rounded-lg py-2 text-[11px] font-medium transition",
      pendingLayout === id
        ? "bg-background text-foreground shadow-sm"
        : "text-muted-foreground hover:text-foreground"
    )}
  >
    <Icon size={18} className={pendingLayout === id ? "text-blue-600" : undefined} />
    {label}
  </button>
))}
</div>

{boards.length < 2 ? (
  <p className="px-1 py-3 text-center text-[11px] leading-snug text-muted-foreground">
    You need two documents. Make another board or notebook, or open a
    PDF, and you can pair them here.
  </p>
) : (
  <>
    <p className="mb-1 px-1 text-[10px] font-semibold uppercase tracking-wider text-muted-foreground">
      {pendingLayout === "stacked" ? "Top" : "Left"}
    </p>
    <BoardList
      boards={boards}
      selectedId={pendingLeft}
      disabledId={pendingRight}
      onPick={setPendingLeft}
    />

    <p className="mb-1 mt-2 px-1 text-[10px] font-semibold uppercase tracking-wider text-muted-foreground">
      {pendingLayout === "stacked" ? "Bottom" : "Right"}
    </p>
    <BoardList
      boards={boards}
      selectedId={pendingRight}
      disabledId={pendingLeft}
      onPick={setPendingRight}
    />

    <p className="mt-2 px-1 text-[10px] leading-snug text-muted-foreground">
      Both can be written on: tap a pane (or start writing in it) to make
      it the one your tools act on. The blue frame shows which one that is.
    </p>

    <div className="mt-2 flex gap-1">
      <Button
        size="sm"
        className="h-7 flex-1 text-[11px]"
        disabled={!canApply}
        onClick={() => {
          if (!canApply) return;
          setSplitPair(pendingLeft!, pendingRight!, pendingLayout);
          setOpen(false);
        }}
      >
        {splitOpen ? "Apply" : pendingLayout === "stacked" ? "Show top & bottom" : "Show side by side"}
      </Button>
      {splitOpen && (
        <Button
          variant="outline"
          size="sm"
          className="h-7 text-[11px]"
          onClick={() => {

```

```

        setSplitBoard(null);
        setOpen(false);
      }}
    >
    <X size={12} className="mr-1" /> Close
  </Button>
  )}
</div>
</>
)}
</PopoverContent>
</Popover>
);
}

=====
// === END FILE: src/components/whiteboard/SplitPicker.tsx ===
=====

=====
// === START FILE: src/components/whiteboard/StepperControl.tsx ===
// File 81 of 194 | Lines: 151 | Size: 4910 bytes
=====

"use client";

/**
 * -/+ steppers for numeric sliders
 * =====
 * Dragging a slider is good for getting roughly where you want and bad for
 * landing on an exact value – especially with a finger on a tablet, where the
 * thumb is under the fingertip you are trying to aim with. Every numeric
 * slider in the app therefore gets a -/+ pair around it.
 *
 * Holding a button repeats. Tapping + seventeen times to walk 7px → 24px is
 * the problem the buttons were added to solve, so they must not recreate it.
 */

import React from "react";
import { Minus, Plus } from "lucide-react";
import { cn } from "@lib/utills";

const HOLD_DELAY_MS = 400;
const HOLD_REPEAT_MS = 60;

export interface StepperProps {
  value: number;
  min: number;
  max: number;
  /** How much one press moves the value. Match the slider's own step. */
  step?: number;
  onChange: (next: number) => void;
  /**
   * Called once when a press (or a hold) ends, for controls that batch into
   * one undo entry rather than one per tick.
   */
  onCommit?: () => void;
}

function clamp(v: number, min: number, max: number): number {
  return Math.max(min, Math.min(max, v));
}

export function StepButton({
  direction,
  value,
  min,
  max,
  step = 1,
  onChange,
  onCommit,
  className,
}: StepperProps & { direction: -1 | 1; className?: string }) {
  const disabled = direction < 0 ? value <= min : value >= max;

  // The repeat timer closes over the render that started the hold, so without
  // this it would keep re-applying the step to the value as it was on press –

```

```

// holding + would move you exactly one step and then stall.
const latest = React.useRef({ value, onChange, onCommit });
React.useEffect(() => {
  latest.current = { value, onChange, onCommit };
});

const delayRef = React.useRef<ReturnType<typeof setTimeout> | null>(null);
const repeatRef = React.useRef<ReturnType<typeof setInterval> | null>(null);
const activeRef = React.useRef(false);

const stop = React.useCallback(() => {
  if (delayRef.current) clearTimeout(delayRef.current);
  if (repeatRef.current) clearInterval(repeatRef.current);
  delayRef.current = null;
  repeatRef.current = null;
  if (activeRef.current) {
    activeRef.current = false;
    latest.current.onCommit?.();
  }
}, []);

React.useEffect(() => stop, [stop]);

// Reaching min/max mid-hold disables the button, and a disabled button gets
// no pointerup – the timer would otherwise run forever against the clamp.
React.useEffect(() => {
  if (disabled) stop();
}, [disabled, stop]);

const apply = React.useCallback(() => {
  const { value: v, onChange: set } = latest.current;
  const next = clamp(v + direction * step, min, max);
  if (next !== v) set(next);
}, [direction, step, min, max]);

const begin = React.useCallback(() => {
  activeRef.current = true;
  apply();
  delayRef.current = setTimeout(() => {
    repeatRef.current = setInterval(apply, HOLD_REPEAT_MS);
  }, HOLD_DELAY_MS);
}, [apply]);

return (
  <button
    type="button"
    disabled={disabled}
    onPointerDown={(e) => {
      if (e.button !== 0) return;
      // Keeps the press from stealing focus or starting a text selection,
      // which on touch would turn a hold into a text-selection gesture.
      e.preventDefault();
      begin();
    }}
    onPointerUp={stop}
    onPointerLeave={stop}
    onPointerCancel={stop}
    // preventDefault above suppresses the synthetic click, and pointer events
    // never fire for keyboard activation, so the keyboard is handled here.
    onKeyDown={(e) => {
      if (e.key === "Enter" || e.key === " ") {
        e.preventDefault();
        apply();
        latest.current.onCommit?.();
      }
    }}
    aria-label={direction < 0 ? "Decrease" : "Increase"}
    title={direction < 0 ? "Decrease (hold to repeat)" : "Increase (hold to repeat)"}
    className={cn(
      "flex h-7 w-7 shrink-0 touch-none select-none items-center justify-center rounded-lg",
      "border border-border text-muted-foreground transition-colors",
      "hover:bg-muted hover:text-foreground active:bg-muted active:scale-95",
      "disabled:pointer-events-none disabled:opacity-40",
      className
    )}
  >
    {direction < 0 ? <Minus size={13} /> : <Plus size={13} />}
  </button>

```

```

    });
}

/**
 * Wraps a slider as `[-] slider [+]`. The slider keeps its own markup and
 * styling; this only adds the buttons and the row that holds them.
 */
export function StepperRow({
  children,
  className,
  ...stepper
}: StepperProps & { children: React.ReactNode; className?: string }) {
  return (
    <div className={cn("flex items-center gap-2", className)}>
      <StepButton direction={-1} {...stepper} />
      {children}
      <StepButton direction={1} {...stepper} />
    </div>
  );
}

=====
// === END FILE: src/components/whiteboard/StepperControl.tsx ===
=====

=====
// === START FILE: src/components/whiteboard/SyncPanel.tsx ===
// File 82 of 194 | Lines: 327 | Size: 12565 bytes
=====

"use client";

/**
 * Sync & Backup panel
 * =====
 * - Microsoft sign-in, then two-way sync of the whole workspace through the
 * user's own OneDrive (see lib/whiteboard/onedrive.ts).
 * - Local backup: download / restore a workspace JSON. This works with no
 * account at all, and is the safety net for moving between browsers, since
 * localStorage is per-browser.
 */

import React, { useCallback, useEffect, useRef, useState } from "react";
import { useWhiteboard } from "@store/whiteboard";
import {
  beginSignIn,
  completeSignInIfRedirected,
  downloadWorkspace,
  getSignedInAccount,
  isConfigured,
  isSignedIn,
  signOut,
  uploadWorkspace,
} from "@lib/whiteboard/onedrive";
import {
  parseWorkspace,
  serializeWorkspace,
  WorkspaceParseError,
} from "@lib/whiteboard/workspace";
import { isNativeApp, saveBlob } from "@lib/whiteboard/export";
import { Cloud, CloudOff, Download, Loader2, Logout, RefreshCw, Upload, X } from "lucide-react";
import { AccountSection } from "../AccountSection";
import { Button } from "@components/ui/button";
import { cn } from "@lib/utils";

type Status =
  | { kind: "idle" }
  | { kind: "busy"; message: string }
  | { kind: "ok"; message: string }
  | { kind: "error"; message: string };

function deviceLabel(): string {
  if (typeof navigator === "undefined") return "unknown device";
  const ua = navigator.userAgent;
  if (/Android/i.test(ua)) return "Android tablet/phone";
  if (/iPad|iPhone/i.test(ua)) return "iPad/iPhone";
  if (/Windows/i.test(ua)) return "Windows PC";

```

```

    if (/Mac/i.test(ua)) return "Mac";
    return "browser";
}

export function SyncPanel({ open, onClose }: { open: boolean; onClose: () => void }) {
    const getWorkspaceSnapshot = useWhiteboard((s) => s.getWorkspaceSnapshot());
    const replaceWorkspace = useWhiteboard((s) => s.replaceWorkspace());
    const mergeWorkspace = useWhiteboard((s) => s.mergeWorkspace());

    const [status, setStatus] = useState<Status>({ kind: "idle" });
    const [signedIn, setSignedIn] = useState(false);
    const [account, setAccount] = useState<{ name?: string; email?: string } | null>(null);
    const fileInputRef = useRef<HTMLInputElement>(null);

    const refreshAuthState = useCallback(() => {
        setSignedIn(isSignedIn());
        setAccount(getSignedInAccount() ?? null);
    }, []);

    // Finish the OAuth redirect on load – the app comes back to its own URL with
    // ?code=..., which must be exchanged before anything else works.
    useEffect(() => {
        let cancelled = false;
        completeSignInIfRedirected()
            .then((completed) => {
                if (cancelled) return;
                refreshAuthState();
                if (completed) setStatus({ kind: "ok", message: "Signed in to Microsoft." });
            })
            .catch((e: Error) => {
                if (!cancelled) setStatus({ kind: "error", message: e.message });
            });
        refreshAuthState();
        return () => {
            cancelled = true;
        };
    }, [refreshAuthState]);

    if (!open) return null;

    const configured = isConfigured();

    const doSync = async () => {
        setStatus({ kind: "busy", message: "Syncing with OneDrive..." });
        try {
            const remote = await downloadWorkspace();
            let summary: string;
            if (!remote) {
                summary = "First sync – uploaded this device's boards.";
            } else {
                const report = mergeWorkspace(remote.snapshot);
                summary =
                    `Merged: ${report.added} new, ${report.updatedFromRemote} updated from OneDrive, ` +
                    `${report.keptLocal} kept from this device.`;
            }
            // Upload the merged result so both devices converge on the same document.
            await uploadWorkspace(getWorkspaceSnapshot(), deviceLabel());
            setStatus({ kind: "ok", message: summary });
        } catch (e) {
            setStatus({ kind: "error", message: (e as Error).message });
        }
    };

    const doSignIn = async () => {
        setStatus({ kind: "busy", message: "Opening Microsoft sign-in..." });
        try {
            await beginSignIn();
        } catch (e) {
            setStatus({ kind: "error", message: (e as Error).message });
        }
    };

    const doSignOut = () => {
        signOut();
        refreshAuthState();
        setStatus({ kind: "ok", message: "Signed out. Your boards stay on this device." });
    };
}

```

```

const doBackup = async () => {
  const json = serializeWorkspace(getWorkspaceSnapshot(), deviceLabel());
  const blob = new Blob([json], { type: "application/json" });
  const stamp = new Date().toISOString().slice(0, 10);
  setStatus({ kind: "busy", message: "Preparing backup..." });
  try {
    // saveBlob, not a hand-rolled <a download>: inside the APK that click is
    // swallowed and the backup silently never happens.
    await saveBlob(blob, `pronotes-backup-${stamp}.json`);
    setStatus({
      kind: "ok",
      message: isNativeApp() ? "Backup ready – pick where to save it." : "Backup file downloaded.",
    });
  } catch (e) {
    setStatus({ kind: "error", message: `Could not save the backup: ${(e as Error).message}` });
  }
};

const doRestore = async (file: File) => {
  setStatus({ kind: "busy", message: "Reading backup..." });
  try {
    const snapshot = parseWorkspace(await file.text());
    const boards = snapshot.boards.length;
    // Restoring REPLACES everything, so make the user confirm with the real
    // number in front of them – this is not undoable.
    const ok = window.confirm(
      `Replace everything on this device with the backup?\n\n` +
      `The backup has ${boards} board${boards === 1 ? "" : "s"}.` +
      `Your current boards on this device will be lost.`
    );
    if (!ok) {
      setStatus({ kind: "idle" });
      return;
    }
    replaceWorkspace(snapshot);
    setStatus({ kind: "ok", message: `Restored ${boards} board${boards === 1 ? "" : "s"}.` });
  } catch (e) {
    setStatus({
      kind: "error",
      message:
        e instanceof WorkspaceParseError ? e.message : `Could not restore: ${(e as Error).message}`,
    });
  }
};

const busy = status.kind === "busy";

return (
  <div
    data-wb-modal
    className="fixed inset-0 z-[90] flex items-center justify-center bg-black/50 backdrop-blur-sm"
    onClick={onClose}
  >
    <div
      className="w-full max-w-md rounded-2xl border border-border bg-white shadow-2xl dark:bg-zinc-900"
      onClick={(e) => e.stopPropagation()}
    >
      <div className="flex items-center justify-between border-b border-border px-5 py-3">
        <div className="flex items-center gap-2">
          <Cloud size={18} className="text-primary" />
          <h2 className="text-base font-semibold">Sync & Backup</h2>
        </div>
        <Button variant="ghost" size="icon" className="h-7 w-7" onClick={onClose}>
          <X size={16} />
        </Button>
      </div>

      <div className="space-y-4 p-5">
        {/* ---- Account (email + password, self-service sign-up) ---- */}
        <section className="space-y-2">
          <h3 className="text-xs font-semibold uppercase tracking-wider text-muted-foreground">
            Your account
          </h3>
          <AccountSection
            deviceLabel={deviceLabel()}
            onStatus={setStatus}
            busy={status.kind === "busy"}
          />
        </section>
      </div>
    </div>
  </div>
);

```

```

</section>

<div className="h-px bg-border" />

{/* ---- OneDrive ---- */}
<section className="space-y-2">
  <h3 className="text-xs font-semibold uppercase tracking-wider text-muted-foreground">
    Or use your Microsoft account
  </h3>

  {!configured ? (
    <div className="rounded-lg border border-amber-500/40 bg-amber-500/10 p-3 text-xs leading-relaxed">
      <p className="font-medium text-amber-700 dark:text-amber-400">
        Sign-in is not set up yet.
      </p>
      <p className="mt-1 text-muted-foreground">
        Add your free Microsoft app ID as{" "}
        <code className="font-semibold">NEXT_PUBLIC_MS_CLIENT_ID</code> and rebuild. See{" "}
        <code>ONEDRIVE_SETUP.md</code> – it takes about two minutes and costs nothing.
      </p>
    </div>
  ) : signedIn ? (
    <>
      <div className="rounded-lg border border-border p-3 text-xs">
        <div className="font-medium">{account?.name || "Signed in"}</div>
        {account?.email && (
          <div className="text-muted-foreground">{account.email}</div>
        )}
      </div>
      <div className="flex gap-2">
        <Button size="sm" className="flex-1" onClick={doSync} disabled={busy}>
          {busy ? (
            <Loader2 size={14} className="mr-1.5 animate-spin" />
          ) : (
            <RefreshCw size={14} className="mr-1.5" />
          )}
          Sync now
        </Button>
        <Button size="sm" variant="outline" onClick={doSignOut} disabled={busy}>
          <LogOut size={14} className="mr-1.5" />
          Sign out
        </Button>
      </div>
      <p className="text-[11px] leading-snug text-muted-foreground">
        Boards are merged one by one, newest edit wins. Only a board you
        changed on two devices since the last sync has to pick a winner.
      </p>
    </>
  ) : (
    <>
      <Button size="sm" className="w-full" onClick={doSignIn} disabled={busy}>
        <Cloud size={14} className="mr-1.5" />
        Sign in with Microsoft
      </Button>
      <p className="text-[11px] leading-snug text-muted-foreground">
        Your boards are stored in one file in your own OneDrive. ProNotes
        can only see its own folder, never the rest of your files.
      </p>
    </>
  )}
</section>

<div className="h-px bg-border" />

{/* ---- Local file backup ---- */}
<section className="space-y-2">
  <h3 className="text-xs font-semibold uppercase tracking-wider text-muted-foreground">
    Backup file
  </h3>
  <div className="flex gap-2">
    <Button size="sm" variant="outline" className="flex-1" onClick={() => void doBackup()} disabled={
      busy}>
      <Download size={14} className="mr-1.5" />
      Save backup
    </Button>
    <Button
      size="sm"

```

```

        variant="outline"
        className="flex-1"
        onClick={() => fileInputRef.current?.click()}
        disabled={busy}
      >
        <Upload size={14} className="mr-1.5" />
        Restore
      </Button>
    </div>
    <input
      ref={fileInputRef}
      type="file"
      accept="application/json,.json"
      className="hidden"
      onChange={(e) => {
        const file = e.target.files?.[0];
        // Reset so picking the SAME file twice still fires onChange.
        e.target.value = "";
        if (file) void doRestore(file);
      }}
    />
    <p className="text-[11px] leading-snug text-muted-foreground">
      Boards live in this browser only. A backup file moves them to another
      browser, another PC, or your tablet – no account needed.
    </p>
  </section>

  {status.kind !== "idle" && (
    <div
      className={cn(
        "rounded-lg border p-2.5 text-xs",
        status.kind === "error" && "border-destructive/40 bg-destructive/10 text-destructive",
        status.kind === "ok" && "border-emerald-500/40 bg-emerald-500/10 text-emerald-700 dark:text-emerald-400",
        status.kind === "busy" && "border-border bg-muted text-muted-foreground"
      )}
    >
      <span className="flex items-center gap-1.5">
        {status.kind === "busy" && <Loader2 size={12} className="animate-spin" />}
        {status.kind === "error" && <CloudOff size={12} />}
        {status.message}
      </span>
    </div>
  )}
</div>
</div>
</div>
);
}

```

```

=====
// === END FILE: src/components/whiteboard/SyncPanel.tsx ===
=====

```

```

=====
// === START FILE: src/components/whiteboard/ThemeToggle.tsx ===
// File 83 of 194 | Lines: 99 | Size: 3362 bytes
=====

```

```

"use client";

import React from "react";
import { useTheme } from "next-themes";
import { Sun, Moon, Laptop, Check } from "lucide-react";
import {
  DropdownMenu,
  DropdownMenuContent,
  DropdownMenuItem,
  DropdownMenuTrigger,
} from "@components/ui/dropdown-menu";
import { Tooltip, TooltipContent, TooltipTrigger } from "@components/ui/tooltip";
import { cn } from "@lib/utls";

interface ThemeToggleProps {
  className?: string;
  size?: "sm" | "default" | "icon";
  variant?: "ghost" | "outline" | "default";
}

```

```

}

export function ThemeToggle({ className, size = "icon", variant = "ghost" }: ThemeToggleProps) {
  const { theme, setTheme, resolvedTheme } = useTheme();
  const [mounted, setMounted] = React.useState(false);

  React.useEffect(() => {
    setMounted(true);
  }, []);

  if (!mounted) {
    return (
      <button
        className={cn(
          "p-1.5 rounded-lg text-muted-foreground hover:text-foreground hover:bg-muted transition-colors",
          className
        )}
        aria-label="Toggle theme"
      >
        <Sun size={16} />
      </button>
    );
  }

  const currentIcon = resolvedTheme === "dark" ? <Moon size={16} /> : <Sun size={16} />;

  return (
    <DropdownMenu>
      <Tooltip>
        <TooltipTrigger asChild>
          <DropdownMenuTrigger asChild>
            <button
              className={cn(
                "p-1.5 rounded-lg text-muted-foreground hover:text-foreground hover:bg-muted transition-colors",
                "flex items-center justify-center",
                className
              )}
              aria-label="Toggle theme (Ctrl+Shift+D)"
            >
              {currentIcon}
            </button>
          </DropdownMenuTrigger>
        </TooltipTrigger>
        <TooltipContent side="top">
          Theme ({theme ? theme.charAt(0).toUpperCase() + theme.slice(1) : "System"})
        </TooltipContent>
      </Tooltip>
      <DropdownMenuContent align="end" className="w-36 z-50">
        <DropdownMenuItem
          onClick={() => setTheme("light")}
          className={cn("flex items-center justify-between text-xs cursor-pointer", theme === "light" && "font-
            -semibold text-primary")}
        >
          <div className="flex items-center gap-2">
            <Sun size={14} />
            <span>Light</span>
          </div>
          {theme === "light" && <Check size={12} className="text-primary" />}
        </DropdownMenuItem>
        <DropdownMenuItem
          onClick={() => setTheme("dark")}
          className={cn("flex items-center justify-between text-xs cursor-pointer", theme === "dark" && "font-
            -semibold text-primary")}
        >
          <div className="flex items-center gap-2">
            <Moon size={14} />
            <span>Dark</span>
          </div>
          {theme === "dark" && <Check size={12} className="text-primary" />}
        </DropdownMenuItem>
        <DropdownMenuItem
          onClick={() => setTheme("system")}
          className={cn("flex items-center justify-between text-xs cursor-pointer", theme === "system" && "font-
            -semibold text-primary")}
        >
          <div className="flex items-center gap-2">
            <Laptop size={14} />
            <span>System</span>
          </div>
        </DropdownMenuItem>
      </DropdownMenuContent>
    </DropdownMenu>
  );
}

```

```

    </div>
    {theme === "system" && <Check size={12} className="text-primary" />}
  </DropdownMenuItem>
</DropdownMenuContent>
</DropdownMenu>
);
}

=====
// === END FILE: src/components/whiteboard/ThemeToggle.tsx ===
=====

=====
// === START FILE: src/components/whiteboard/ToolProperties.tsx ===
// File 84 of 194 | Lines: 940 | Size: 36962 bytes
=====

"use client";

/**
 * Contextual tool properties
 * =====
 * Follows the pattern every serious note app uses (Samsung Notes, Concepts,
 * GoodNotes): the tool row only SELECTS a tool, and a second row shows the
 * properties of whatever is selected – colour, thickness – with a ■ that opens
 * the full panel for everything else.
 *
 * The point is that the common controls (colour, size) are one tap away
 * instead of hidden in a popover, while rarely-changed options stay out of
 * sight until asked for.
 */

import React from "react";
import { useWhiteboard } from "@store/whiteboard";
import {
  FONT_FAMILIES,
  LASER_COLORS,
  PEN_COLORS,
  PEN_TYPE_PRESETS,
  PEN_TYPES,
  STROKE_TOOL_IDS,
  STICKY_COLORS,
  getToolColor,
  getToolSize,
  setToolColor,
  setToolSize,
  type DashStyle,
  type FillStyle,
  type LaserMode,
  type StickyColor,
  type StrokeStyle,
  type ToolType,
  type PressureCurve,
  type PenFeel,
  isVerticalDock,
  penFeelFor,
  penUsesPressure,
} from "@lib/whiteboard/types";
import { buildStrokePath, calibratePressure } from "@lib/whiteboard/stroke";
import { ColorRow, Swatch } from "../ColorSwatches";
import { flyoutSide } from "../DockControls";
import { Popover, PopoverContent, PopoverTrigger } from "@components/ui/popover";
import { StepperRow } from "../StepperControl";
import { Settings2, X } from "lucide-react";
import { Button } from "@components/ui/button";
import { cn } from "@lib/utils";

const STROKE_TOOLS: ToolType[] = STROKE_TOOL_IDS as ToolType[];
const SHAPE_TOOLS: ToolType[] = ["rectangle", "ellipse", "diamond", "triangle", "star", "cloud"];
const LINE_TOOLS: ToolType[] = ["line", "arrow"];
const TEXT_TOOLS: ToolType[] = ["text", "sticky"];

/** Tools with nothing to configure – the bar hides entirely for these. */
const BARE_TOOLS: ToolType[] = ["select", "lasso", "pan", "image", "textselect"];

// These mirror the DashStyle / FillStyle unions in types.ts, which are types
// rather than runtime values.

```

```

const DASH_STYLES: DashStyle[] = ["solid", "dashed", "dotted"];
const FILL_STYLES: FillStyle[] = ["transparent", "solid", "semi", "hatch"];

export function toolHasProperties(tool: ToolType): boolean {
  return !BARE_TOOLS.includes(tool);
}

/** The laser's three looks, with what each is for. */
const LASER_MODES: Array< { id: LaserMode; label: string; hint: string }> = [
  { id: "dot", label: "Dot", hint: "Just a glowing dot where you point" },
  { id: "trail", label: "Trail", hint: "The dot leaves a short tail that fades at once" },
  { id: "draw", label: "Draw", hint: "Circle or underline; the lines fade after you lift the pen" },
];

/** "Draw" mode: how long the lines stay after the pen lifts. */
const LASER_FADE_CHOICES = [
  { value: 800, label: "Short" },
  { value: 1500, label: "Normal" },
  { value: 3000, label: "Long" },
  { value: 6000, label: "6 s" },
];

function sizeRangeFor(tool: ToolType): { min: number; max: number; presets: number[] } {
  if (tool === "laser") return { min: 4, max: 40, presets: [6, 10, 16, 24] };
  if (tool === "eraser") return { min: 4, max: 96, presets: [8, 16, 32, 64] };
  if (TEXT_TOOLS.includes(tool)) return { min: 10, max: 96, presets: [14, 18, 24, 32, 48] };
  if (STROKE_TOOLS.includes(tool)) return { min: 1, max: 64, presets: [2, 4, 8, 16, 24] };
  return { min: 1, max: 32, presets: [1, 2, 4, 8, 12] };
}

// -----
// Contextual bar (row above the tool dock)
// -----

export function ToolPropertiesBar({ onOpenPanel }: { onOpenPanel: () => void }) {
  const tool = useWhiteboard((s) => s.tool);
  const style = useWhiteboard((s) => s.style);
  const setStyle = useWhiteboard((s) => s.setStyle);
  const dockPos = useWhiteboard((s) => s.toolbarPosition);
  const vertical = isVerticalDock(dockPos);
  const editingId = useWhiteboard((s) => s.editingId);
  const collapsed = useWhiteboard((s) => s.toolPropsBarCollapsed);

  if (!toolHasProperties(tool)) return null;
  // Folded away by the chevron pinned at the end of the tool row.
  if (collapsed) return null;
  // While typing in a text box this bar would edit the text TOOL's default -
  // for the next box - not the one being typed in, so pressing + read "21px"
  // while the box stayed 17px. The box's own bar is the one place now.
  if (editingId) return null;

  const { min, max } = sizeRangeFor(tool);
  const isSticky = tool === "sticky";
  const stickyColorsList = Object.keys(STICKY_COLORS).map((k) => STICKY_COLORS[k as StickyColor].bg);
  const color = isSticky ? STICKY_COLORS[style.stickyColor]?.bg ?? "#fef9c3" : getToolColor(style, tool);
  const size = getToolSize(style, tool);

  const onColorChange = (c: string) => {
    if (isSticky) {
      const match = (Object.keys(STICKY_COLORS) as StickyColor[]).find(
        (k) => STICKY_COLORS[k].bg.toLowerCase() === c.toLowerCase()
      );
      if (match) {
        setStyle({ stickyColor: match });
        return;
      }
    }
    setStyle(setToolColor(style, tool, c));
  };

  const showColor = tool !== "eraser";

  return (
    <div
      className={cn(
        "flex items-center gap-2 rounded-2xl border border-border/80 bg-white/95 p-1.5 shadow-lg backdrop-blur",
        "-xl dark:bg-zinc-900/95",
        vertical ? "max-h-full flex-col overflow-y-auto" : "max-w-full overflow-x-auto"
      )}
    >

```

```

    })
  >
  { /* Thickness: a live slider, right here. Five fixed presets meant any
    other size needed the panel; dragging is one gesture and covers the
    whole range. The dot previews the ACTUAL width in the current colour. */
  <div className={cn("flex shrink-0 items-center gap-2", vertical ? "flex-col pt-0.5" : "pl-0.5")}>
    <span
      className="shrink-0 rounded-full border border-black/10 dark:border-white/15"
      style={{
        width: Math.max(4, Math.min(20, size)),
        height: Math.max(4, Math.min(20, size)),
        backgroundColor: tool === "eraser" ? "transparent" : color,
        borderStyle: tool === "eraser" ? "dashed" : "solid",
      }}
      title={` ${size}px` }
    />
    <StepperRow
      value={size}
      min={min}
      max={max}
      onChange={(next) => setStyle(setToolSize(style, tool, next))}
    />
    <input
      type="range"
      min={min}
      max={max}
      value={size}
      onChange={(e) => setStyle(setToolSize(style, tool, Number(e.target.value)))}
      aria-label="Thickness"
      title={`Thickness - ${size}px`}
      className={cn(
        "h-1.5 cursor-pointer appearance-none rounded-lg bg-muted accent-primary",
        vertical ? "w-16" : "w-24"
      )}
    />
  </StepperRow>
  <span className="w-9 shrink-0 text-[11px] font-semibold tabular-nums text-muted-foreground">
    {size}px
  </span>
</div>

{tool === "laser" && (
  <>
    <span className={cn("shrink-0 bg-border", vertical ? "h-px w-5" : "h-5 w-px")} />
    <div className={cn("flex shrink-0 gap-0.5 rounded-lg bg-muted p-0.5", vertical ? "flex-col" : "items
      -center")}>
      {LASER_MODES.map((m) => (
        <button
          key={m.id}
          onClick={() => setStyle({ laserMode: m.id })}
          title={m.hint}
          className={cn(
            "rounded-md px-2 py-1 text-[11px] font-medium transition-colors",
            (style.laserMode ?? "trail") === m.id
              ? "bg-background shadow-sm"
              : "text-muted-foreground hover:text-foreground"
          )}
        >
          {m.label}
        </button>
      ))}
    </div>
  </>
)}

{tool === "eraser" && (
  <>
    <span className={cn("shrink-0 bg-border", vertical ? "h-px w-5" : "h-5 w-px")} />
    <div className={cn("flex shrink-0 gap-0.5 rounded-lg bg-muted p-0.5", vertical ? "flex-col" : "items
      -center")}>
      {[
        { id: "pixel", label: "Pixel" },
        { id: "object", label: "Whole" },
      ] as const}.map((m) => (
        <button
          key={m.id}
          onClick={() => setStyle({ eraserMode: m.id })}
          title={

```

```

        m.id === "pixel"
        ? "Rub out only the ink you touch; text boxes and pictures are left alone"
        : "Delete the whole element on contact"
    }
    className={cn(
      "rounded-md px-2 py-1 text-[11px] font-medium transition-colors",
      style.eraserMode === m.id
        ? "bg-background shadow-sm"
        : "text-muted-foreground hover:text-foreground"
    )}
  >
    {m.label}
  </button>
))}
</div>
</>
)}

{showColor && (
  <>
    <span className={cn("shrink-0 bg-border", vertical ? "h-px w-5" : "h-5 w-px")} />
    {vertical ? (
      // Six swatches in a column would make the dock taller than the
      // window, so the colours open outward over the canvas instead.
      <Popover>
        <PopoverTrigger asChild>
          <button
            title="Colour"
            aria-label="Colour"
            className="flex h-7 w-7 shrink-0 items-center justify-center rounded-lg border border-border
            transition-colors hover:bg-muted"
          >
            <span
              className="h-4 w-4 rounded-full border border-black/15 dark:border-white/20"
              style={{ backgroundColor: color }}
            />
          </button>
        </PopoverTrigger>
        <PopoverContent side={flyoutSide(dockPos)} align="center" className="w-auto p-2">
          <ColorRow
            colors={isSticky ? stickyColorsList : tool === "laser" ? LASER_COLORS : PEN_COLORS}
            value={color}
            onChange={onColorChange}
          />
        </PopoverContent>
      </Popover>
    ) : (
      <ColorRow
        compact
        colors={isSticky ? stickyColorsList : tool === "laser" ? LASER_COLORS : PEN_COLORS.slice(0, 6)}
        value={color}
        onChange={onColorChange}
      />
    )}
  </>
)}

<span className={cn("shrink-0 bg-border", vertical ? "h-px w-5" : "h-5 w-px")} />
<button
  onClick={onOpenPanel}
  title={` ${tool === "laser" ? "Laser" : PEN_TYPE_PRESETS[tool as StrokeStyle]?.label ?? "Tool"} setting
  s`}
  className="flex h-7 w-7 shrink-0 items-center justify-center rounded-lg text-muted-foreground transiti
  on-colors hover:bg-muted hover:text-foreground"
>
  <Settings2 size={15} />
</button>
</div>
);
}

// -----
// Full panel (opened from the ■)
// -----

/**
 * A tiny sample squiggle whose shape hints at the pen: tapered pens get a
 * stroke that narrows, even pens get a constant one.

```

```

*/
function penPreviewPath(style: StrokeStyle): string {
  const p = PEN_TYPE_PRESETS[style];
  return p.endTaper > 0 || p.thinning > 0.3
    ? "M2,13 C10,3 18,15 26,7 S40,3 44,6"
    : "M2,9 C10,4 18,14 26,9 S40,7 44,9";
}

// -----
// Pen panel - compact, in the manner of GoodNotes / Samsung Notes
// -----
//
// One small card: a live preview line, the pen types, thickness, colour, and
// a folded "Writing feel" section. The old panel was a near-full-screen sheet
// of engineering terms with a scratchpad that did not write like the page
// (different input path), which made it look like the pen was broken.

const SMOOTHING_CHOICES = [
  { value: 0, label: "Off" },
  { value: 25, label: "Low" },
  { value: 50, label: "Normal" },
  { value: 80, label: "High" },
] as const;

const PRESSURE_CHOICES: { value: PressureCurve; label: string; hint: string }[] = [
  { value: "off", label: "Off", hint: "Same width however hard you press." },
  { value: "soft", label: "Light", hint: "Full width with a light touch." },
  { value: "standard", label: "Normal", hint: "Width follows your pressure." },
  { value: "firm", label: "Firm", hint: "Press harder for a thicker line." },
];

/**
 * The preview line: a handwriting-like wave drawn by the SAME path builder
 * and pressure curve as the page, with a light-heavy-light pressure run like
 * a real stroke (0.2 to 0.6, the range measured from a Galaxy Tab S9 S Pen).
 */
function PenPreview({
  tool,
  color,
  size,
  smoothing,
  sensitivity,
  curve,
}: {
  tool: StrokeStyle;
  color: string;
  size: number;
  smoothing: number;
  sensitivity: number;
  curve: PressureCurve;
}) {
  const preset = PEN_TYPE_PRESETS[tool] ?? PEN_TYPE_PRESETS.gel;
  const d = React.useMemo(() => {
    const pts: Array<[number, number, number]> = [];
    const N = 90;
    for (let i = 0; i <= N; i++) {
      const t = i / N;
      const raw = 0.2 + 0.4 * Math.sin(t * Math.PI);
      pts.push([12 + t * 256, 30 + Math.sin(t * Math.PI * 5) * 11, calibratePressure(raw, sensitivity, curve)]);
    }
  });
  return buildStrokePath({
    id: "preview",
    type: "stroke",
    x: 0,
    y: 0,
    rotation: 0,
    opacity: 1,
    createdAt: 0,
    updatedAt: 0,
    points: pts,
    color,
    strokeWidth: size,
    strokeStyle: tool,
    smoothing,
  });
}, [tool, color, size, smoothing, sensitivity, curve]);
return (

```

```

    <svg viewBox="0 0 280 60" className="h-14 w-full rounded-xl border border-border/70 bg-white dark:bg-zinc-950">
      <path
        d={d}
        fill={color}
        opacity={preset.opacity ?? 1}
        style={preset.mixBlendMode === "multiply" ? { mixBlendMode: "multiply" } : undefined}
      />
    </svg>
  );
}

function Choice<T extends string | number>({
  value,
  current,
  label,
  onPick,
}): {
  value: T;
  current: T;
  label: string;
  onPick: (v: T) => void;
} {
  return (
    <button
      type="button"
      onClick={() => onPick(value)}
      className={cn(
        "flex-1 rounded-lg border py-1.5 text-[11px] font-medium transition-colors",
        current === value
          ? "border-primary bg-primary/10 text-primary"
          : "border-border text-muted-foreground hover:bg-muted"
      )}
    >
      {label}
    </button>
  );
}

function PenPanel({ onClose }: { onClose: () => void }) {
  const tool = useWhiteboard((s) => s.tool) as StrokeStyle;
  const style = useWhiteboard((s) => s.style);
  const setStyle = useWhiteboard((s) => s.setStyle);
  const setTool = useWhiteboard((s) => s.setTool);
  const touchSettings = useWhiteboard((s) => s.touchSettings);
  const setTouchSettings = useWhiteboard((s) => s.setTouchSettings);

  const color = getToolColor(style, tool);
  const size = getToolSize(style, tool);
  const { min, max } = sizeRangeFor(tool);
  // Each pen keeps its own feel; editing it here changes only this pen.
  const { smoothing, sensitivity, curve } = penFeelFor(touchSettings, tool);
  const setFeel = (patch: PenFeel) =>
    setTouchSettings({
      penFeel: { ...touchSettings.penFeel, [tool]: { ...touchSettings.penFeel?.[tool], ...patch } },
    });
  const usesPressure = penUsesPressure(tool);
  // What this pen does instead, where pressure has no effect.
  const noPressureNote =
    tool === "brush"
      ? "The brush gets thinner the faster you write."
      : "Same width on every stroke, however hard you press.";
  const curveHint = PRESSURE_CHOICES.find((c) => c.value === curve)?.hint ?? "";
  // Ballpoint and fineliner strokes still exist; they show as "Pen".
  const shownTool: StrokeStyle = tool === "ballpoint" || tool === "fineliner" ? "gel" : tool;
  // The closest named smoothing step, so a value set elsewhere still shows.
  const smoothingStep = SMOOTHING_CHOICES.reduce((a, c) =>
    Math.abs(c.value - smoothing) < Math.abs(a.value - smoothing) ? c : a
  ).value;

  return (
    <div
      data-wb-modal
      className="fixed inset-0 z-[85] flex items-end justify-center bg-black/10 pb-28 sm:items-center sm:pb-0"
      onClick={onClose}
    >
      <div
        className="max-h-[80vh] w-[20rem] max-w-[calc(100vw-2rem)] space-y-3 overflow-y-auto rounded-2xl borde

```

```

    r border-border bg-white p-3 shadow-2xl dark:bg-zinc-900"
    onClick={(e) => e.stopPropagation()}
  >
  <div className="flex items-center justify-between">
    <h3 className="text-sm font-semibold">{PEN_TYPE_PRESETS[tool]?.label ?? "Pen"}</h3>
    <Button variant="ghost" size="icon" className="h-7 w-7" onClick={onClose} aria-label="Close pen settings">
      <X size={15} />
    </Button>
  </div>

  <PenPreview
    tool={tool}
    color={color}
    size={size}
    smoothing={smoothing}
    sensitivity={sensitivity}
    curve={curve}
  />

  <div className="grid grid-cols-3 gap-1">
    {PEN_TYPES.map((st) => {
      const p = PEN_TYPE_PRESETS[st];
      return (
        <button
          key={st}
          type="button"
          onClick={() => setTool(st as ToolType)}
          title={p.hint}
          className={cn(
            "flex flex-col items-center gap-0.5 rounded-lg border px-1 py-1.5 transition-colors",
            shownTool === st ? "border-primary bg-primary/10" : "border-transparent hover:bg-muted"
          )}
        >
          <svg width="34" height="14" viewBox="0 0 46 18" aria-hidden>
            <path
              d={penPreviewPath(st)}
              fill="none"
              stroke={color}
              strokeLinecap={p.cap === "flat" ? "butt" : "round"}
              strokeWidth={p.cap === "flat" ? 7 : 3}
              opacity={p.opacity}
            />
          </svg>
          <span className="w-full truncate text-center text-[10px] leading-tight">{p.label}</span>
        </button>
      );
    })}
  </div>

  <div>
    <div className="mb-1 flex items-center justify-between">
      <span className="text-xs font-medium">Thickness</span>
      <span className="text-[11px] font-semibold tabular-nums text-muted-foreground">{size}px</span>
    </div>
    <StepperRow value={size} min={min} max={max} onChange={(v) => setStyle(setToolSize(style, tool, v))>
      >
      <input
        type="range"
        min={min}
        max={max}
        value={size}
        aria-label="Thickness"
        onChange={(e) => setStyle(setToolSize(style, tool, Number(e.target.value)))}
        className="h-1.5 w-full cursor-pointer appearance-none rounded-lg bg-muted accent-primary"
      />
    </StepperRow>
  </div>

  <div>
    <div className="mb-1 text-xs font-medium">Colour</div>
    <ColorRow colors={PEN_COLORS} value={color} onChange={(c) => setStyle(setToolColor(style, tool, c))>
      />
    </div>

  <details className="group rounded-xl border border-border/70 px-2.5 py-2">
    <summary className="cursor-pointer select-none text-xs font-medium">
      Writing feel <span className="font-normal text-muted-foreground">(this pen only)</span>
    </summary>
  </details>

```

```

</summary>
<div className="mt-2 space-y-3">
  <div>
    <div className="mb-1 text-[11px] text-muted-foreground">Smoothing</div>
    <div className="flex gap-1">
      {SMOOTHING_CHOICES.map((c) => (
        <Choice
          key={c.value}
          value={c.value as number}
          current={smoothingStep}
          label={c.label}
          onPick={(v) => setFeel({ smoothing: v })}
        />
      ))}
    </div>
  </div>
  <div>
    <div className="mb-1 text-[11px] text-muted-foreground">Pressure</div>
    {usesPressure ? (
      <>
        <div className="flex gap-1">
          {PRESSURE_CHOICES.map((c) => (
            <Choice
              key={c.value}
              value={c.value}
              current={curve}
              label={c.label}
              onPick={(v) => setFeel({ curve: v })}
            />
          ))}
        </div>
        <p className="mt-1 text-[10px] text-muted-foreground">{curveHint}</p>
      </>
    ) : (
      <p className="text-[10px] text-muted-foreground">{noPressureNote}</p>
    )}
  </div>
  <label className="flex cursor-pointer items-center justify-between gap-3 text-[11px]">
    <span>Hold at the end of a line to straighten it</span>
    <input
      type="checkbox"
      checked={touchSettings.penAutoStraighten !== false}
      onChange={(e) => setTouchSettings({ penAutoStraighten: e.target.checked })}
      className="h-4 w-4 accent-primary"
    />
  </label>
  <label className="flex cursor-pointer items-center justify-between gap-3 text-[11px]">
    <span>Show a dot where the pen will touch</span>
    <input
      type="checkbox"
      checked={touchSettings.penHoverIndicator !== false}
      onChange={(e) => setTouchSettings({ penHoverIndicator: e.target.checked })}
      className="h-4 w-4 accent-primary"
    />
  </label>
</div>
</div>
</div>
</div>
);
}

/** A small animated-looking sample of the laser in its current look. */
function LaserPreview({ mode, color, size }: { mode: LaserMode; color: string; size: number }) {
  const r = Math.max(2, Math.min(14, size / 2));
  return (
    <div className="flex h-16 items-center justify-center rounded-xl bg-zinc-900">
      <svg width="220" height="56" viewBox="0 0 220 56" aria-hidden>
        <defs>
          <filter id="laser-glow" x="-50%" y="-50%" width="200%" height="200%">
            <feGaussianBlur stdDeviation={Math.max(1.5, r * 0.8)} />
          </filter>
          <linearGradient id="laser-tail" x1="0" x2="1">
            <stop offset="0" stopColor={color} stopOpacity="0" />
            <stop offset="1" stopColor={color} stopOpacity="1" />
          </linearGradient>
        </defs>
        {mode === "trail" && (

```

```

    <path d="M40 40 Q 90 8 150 28" fill="none" stroke="url(#laser-tail)" strokeWidth={r * 1.4} strokeLinecap="round" />
  )}
  {mode === "draw" && (
    <>
      <ellipse cx="110" cy="28" rx="70" ry="18" fill="none" stroke={color} strokeWidth={r * 1.2} filter="url(#laser-glow)" />
      <ellipse cx="110" cy="28" rx="70" ry="18" fill="none" stroke={color} strokeWidth={r * 1.2} />
      <ellipse cx="110" cy="28" rx="70" ry="18" fill="none" stroke="white" strokeOpacity="0.9" strokeWidth={Math.max(1, r * 0.4)} />
    </>
  )}
  {mode !== "draw" && (
    <>
      <circle cx="150" cy="28" r={r * 2.4} fill={color} opacity="0.45" filter="url(#laser-glow)" />
      <circle cx="150" cy="28" r={r} fill={color} />
      <circle cx="150" cy="28" r={r * 0.42} fill="white" opacity="0.92" />
    </>
  )}
</svg>
</div>
);
}

```

```

function LaserPanel({ onClose }: { onClose: () => void }) {
  const style = useWhiteboard((s) => s.style);
  const setStyle = useWhiteboard((s) => s.setStyle);
  const mode = style.laserMode ?? "trail";
  const color = getToolColor(style, "laser");
  const size = getToolSize(style, "laser");
  const { min, max, presets } = sizeRangeFor("laser");
  const fade = style.laserFadeMs ?? 1500;

  return (
    <div
      data-wb-modal
      className="fixed inset-0 z-[85] flex items-end justify-center bg-black/10 pb-28 sm:items-center sm:pb-0"
      onClick={onClose}
    >
      <div
        className="max-h-[80vh] w-[20rem] max-w-[calc(100vw-2rem)] space-y-3 overflow-y-auto rounded-2xl border border-border bg-white p-3 shadow-2xl dark:bg-zinc-900"
        onClick={(e) => e.stopPropagation()}
      >
        <div className="flex items-center justify-between">
          <h3 className="text-sm font-semibold">Laser pointer</h3>
          <Button variant="ghost" size="icon" className="h-7 w-7" onClick={onClose} aria-label="Close laser settings">
            <X size={15} />
          </Button>
        </div>

        <LaserPreview mode={mode} color={color} size={size} />

        <div>
          <div className="mb-1 text-xs font-medium">Type</div>
          <div className="grid grid-cols-3 gap-1">
            {LASER_MODES.map((m) => (
              <button
                key={m.id}
                type="button"
                onClick={() => setStyle({ laserMode: m.id })}
                className={cn(
                  "rounded-lg border px-1.5 py-1.5 text-left transition-colors",
                  mode === m.id ? "border-primary bg-primary/10 text-primary" : "border-border hover:bg-muted"
                )}
              >
                <span className="block text-[11px] font-semibold">{m.label}</span>
                <span className="block text-[10px] leading-tight text-muted-foreground">{m.hint}</span>
              </button>
            )})}
          </div>
        </div>

        <div>
          <div className="mb-1 flex items-center justify-between">
            <span className="text-xs font-medium">Size</span>
            <span className="text-[11px] font-semibold tabular-nums text-muted-foreground">{size}px</span>
          </div>

```

```

    </div>
    <div className="mb-2 flex gap-1">
      {presets.map((p) => (
        <Choice key={p} value={p} current={size} label={` ${p} `} onPick={(v) => setStyle(setToolSize(style, "laser", v))} />
      ))}
    </div>
    <StepperRow value={size} min={min} max={max} onChange={(v) => setStyle(setToolSize(style, "laser", v))}>
      <input
        type="range"
        min={min}
        max={max}
        value={size}
        aria-label="Laser size"
        onChange={(e) => setStyle(setToolSize(style, "laser", Number(e.target.value)))}
        className="h-1.5 w-full cursor-pointer appearance-none rounded-lg bg-muted accent-primary"
      />
    </StepperRow>
  </div>

  <div>
    <div className="mb-1 text-xs font-medium">Colour</div>
    <ColorRow colors={LASER_COLORS} value={color} onChange={(c) => setStyle(setToolColor(style, "laser", c))} />
  </div>

  {mode === "draw" && (
    <div>
      <div className="mb-1 text-xs font-medium">Lines stay for</div>
      <div className="flex gap-1">
        {LASER_FADE_CHOICES.map((c) => (
          <Choice key={c.value} value={c.value} current={fade} label={c.label} onPick={(v) => setStyle({
            laserFadeMs: v })} />
        ))}
      </div>
    </div>
  )}

  <p className="text-[10px] leading-snug text-muted-foreground">
    The laser never writes on the page: nothing it shows is saved or exported. Hover the S Pen
    over the screen to point without touching.
  </p>
</div>
</div>
);
}

export function ToolPropertiesPanel({ open, onClose }: { open: boolean; onClose: () => void }) {
  const tool = useWhiteboard((s) => s.tool);
  const style = useWhiteboard((s) => s.style);
  const setStyle = useWhiteboard((s) => s.setStyle);

  if (!open || !toolHasProperties(tool)) return null;
  if (STROKE_TOOLS.includes(tool)) return <PenPanel onClose={onClose} />;
  if (tool === "laser") return <LaserPanel onClose={onClose} />;

  const { min, max } = sizeRangeFor(tool);
  const color = getToolColor(style, tool);
  const size = getToolSize(style, tool);

  const isStroke = STROKE_TOOLS.includes(tool);
  const isShape = SHAPE_TOOLS.includes(tool);
  const isLine = LINE_TOOLS.includes(tool);
  const isText = TEXT_TOOLS.includes(tool);

  const title = isStroke
    ? PEN_TYPE_PRESETS[tool as StrokeStyle].label
    : tool.charAt(0).toUpperCase() + tool.slice(1);

  return (
    <div
      data-wb-modal
      className="fixed inset-0 z-[85] flex items-end justify-center bg-black/30 pb-28 backdrop-blur-[2px] sm:items-center sm:pb-0"
      onClick={onClose}
    >
      <div

```

```

className={cn(
  "overflow-y-auto rounded-2xl border border-border bg-white p-4 shadow-2xl dark:bg-zinc-900 transitio
    n-all",
  isStroke ? "w-full max-w-md sm:max-w-lg max-h-[85vh]" : "w-full max-w-xs max-h-[70vh]"
)}
onClick={(e) => e.stopPropagation()}
>
<div className="mb-3 flex items-center justify-between">
  <div className="flex items-center gap-2">
    <h3 className="text-sm font-semibold">{title}</h3>
  </div>
  <Button variant="ghost" size="icon" className="h-7 w-7" onClick={onClose}>
    <X size={15} />
  </Button>
</div>

<div className="space-y-4">
  </* Thickness with quick preset pills */>
  <div>
    <div className="mb-1.5 flex items-center justify-between">
      <span className="text-xs font-medium">{isText ? "Font size" : "Thickness"}</span>
      <span className="rounded-md bg-muted px-2 py-0.5 text-xs font-semibold tabular-nums">
        {size}px
      </span>
    </div>
    {isStroke && (
      <div className="mb-2 flex gap-1">
        {[2, 4, 8, 16, 24].map((sz) => (
          <button
            key={sz}
            type="button"
            onClick={() => setStyle(setToolSize(style, tool, sz))}
            className={cn(
              "flex-1 rounded-md border py-1 text-[11px] font-semibold transition-colors",
              size === sz
                ? "border-primary bg-primary/10 text-primary"
                : "border-border text-muted-foreground hover:bg-muted"
            )}
          >
            {sz}px
          </button>
        ))}
      </div>
    )}
    <StepperRow value={size} min={min} max={max} onChange={(v) => setStyle(setToolSize(style, tool, v))}>
      <input
        type="range"
        min={min}
        max={max}
        value={size}
        aria-label={isText ? "Font size" : "Thickness"}
        onChange={(e) => setStyle(setToolSize(style, tool, Number(e.target.value)))}
        className="h-1.5 w-full cursor-pointer appearance-none rounded-lg bg-muted accent-primary"
      />
    </StepperRow>
  </div>

  {tool === "eraser" && (
    <div>
      <div className="mb-1.5 text-xs font-medium">Eraser mode</div>
      <div className="flex gap-1.5">
        {[
          { id: "pixel", label: "Pixel", hint: "Rub out ink" },
          { id: "object", label: "Whole object", hint: "Delete on contact" },
        ] as const}.map((m) => (
          <button
            key={m.id}
            onClick={() => setStyle({ eraserMode: m.id })}
            className={cn(
              "flex-1 rounded-lg border px-2 py-1.5 text-left transition-colors",
              style.eraserMode === m.id
                ? "border-primary bg-primary/10 text-primary"
                : "border-border hover:bg-muted"
            )}
          >
            <span className="block text-[11px] font-medium">{m.label}</span>
            <span className="block text-[10px] text-muted-foreground">{m.hint}</span>
          </button>
        ))}
      </div>
    </div>
  )}

```

```

        </button>
      )}}
    </div>
    <p className="mt-1.5 text-[11px] leading-snug text-muted-foreground">
      Pixel rubs out ink only: drawn shapes and lines go whole, and text boxes, notes and
      pictures are never touched. Use Whole object to delete those.
    </p>
  </div>
)}

{tool !== "eraser" && (
  <div>
    <div className="mb-1.5 text-xs font-medium">Ink Colour</div>
    <ColorRow
      colors={PEN_COLORS}
      value={color}
      onChange={(c) => setStyle(setToolColor(style, tool, c))}
    />
  </div>
)}

{/* Shape-only: fill and outline style */}
{(isShape || isLine) && (
  <>
    <div>
      <div className="mb-1.5 text-xs font-medium">Line style</div>
      <div className="flex gap-1.5">
        {DASH_STYLES.map((d) => (
          <button
            key={d}
            onClick={() => setStyle({ dashStyle: d })}
            className={cn(
              "flex-1 rounded-lg border px-2 py-1.5 text-[11px] capitalize transition-colors",
              style.dashStyle === d
                ? "border-primary bg-primary/10 text-primary"
                : "border-border hover:bg-muted"
            )}
          >
            {d}
          </button>
        ))}
      </div>
    </div>

    {isShape && (
      <>
        <div>
          <div className="mb-1.5 text-xs font-medium">Fill</div>
          <ColorRow
            colors={PEN_COLORS}
            value={style.shapeFillColor}
            onChange={(c) => setStyle({ shapeFillColor: c, fillColor: c })}
            swatchSize={20}
          />
        </div>
        <div>
          <div className="mb-1.5 text-xs font-medium">Fill style</div>
          <div className="flex gap-1.5">
            {FILL_STYLES.map((f) => (
              <button
                key={f}
                onClick={() => setStyle({ fillStyle: f })}
                className={cn(
                  "flex-1 rounded-lg border px-2 py-1.5 text-[11px] capitalize transition-colors",
                  style.fillStyle === f
                    ? "border-primary bg-primary/10 text-primary"
                    : "border-border hover:bg-muted"
                )}
              >
                {f}
              </button>
            ))}
          </div>
        </div>
      </>
    )}
  </>
)}
})

```

```

    { /* Text-only */ }
    { isText && (
      <div>
        <div className="mb-1.5 text-xs font-medium">Font</div>
        <div className="grid grid-cols-2 gap-1.5">
          { FONT_FAMILIES.map((f) => (
            <button
              key={f.value}
              onClick={() => setStyle({ fontFamily: f.value })}
              className={cn(
                "truncate rounded-lg border px-2 py-1.5 text-[11px] transition-colors",
                style.fontFamily === f.value
                  ? "border-primary bg-primary/10 text-primary"
                  : "border-border hover:bg-muted"
              )}
              style={{ fontFamily: f.value }}
            >
              {f.label}
            </button>
          )}
        </div>
      </div>
    )}

    { tool === "sticky" && (
      <div>
        <div className="mb-1.5 text-xs font-medium">Note colour</div>
        <div className="flex flex-wrap gap-1.5">
          { (Object.keys(STICKY_COLORS) as StickyColor[]).map((sc) => (
            <Swatch
              key={sc}
              color={STICKY_COLORS[sc].bg}
              title={sc}
              active={style.stickyColor === sc}
              onSelect={() => setStyle({ stickyColor: sc })}
            />
          )}
        </div>
      </div>
    )}
  </div>
</div>
</div>
</div>
);
}

```

```

=====
// === END FILE: src/components/whiteboard/ToolProperties.tsx ===
=====

```

```

=====
// === START FILE: src/components/whiteboard/Toolbar.tsx ===
// File 85 of 194 | Lines: 995 | Size: 39301 bytes
=====

```

```
"use client";
```

```

/**
 * Modern Modular Toolbar & Tool Dock (v4 – Native Touchscreen & Obsidian Canvas Grade)
 * =====
 * Features:
 * - Grouped modular dock with popover flyouts for Pens, Shapes, Connectors, and Touchscreen Mode
 * - First-Class Touchscreen Mode Switcher (Touch Draw, Stylus + Palm Rejection, Touch Navigation)
 * - Live tool indicators & persistent per-tool color/size configuration
 * - Quick access to Canvas Zoom, Undo/Redo, Search, Graph View, and Theme Toggle
 */

```

```

import React, { useEffect, useRef, useState } from "react";
import {
  Hand,
  Pen,
  Highlighter,
  Paintbrush,
  Brush,
  Feather,
  Pencil,

```

```

Eraser,
Square,
Circle,
Diamond,
Triangle,
Star,
Cloud,
Minus,
ArrowRight,
Type,
StickyNote,
Link2,
ZoomIn,
ZoomOut,
Maximize,
Network,
Search,
ChevronDown,
ChevronUp,
ChevronLeft,
ChevronRight,
Tablet,
Check,
Sparkles,
Download,
Grid,
Map,
HelpCircle,
Keyboard,
SlidersHorizontal,
} from "lucide-react";
import { ThemeToggle } from "./ThemeToggle";
import { PANEL_W, NotebookPageControls } from "./NotebookPages";
import { ToolPropertiesBar, ToolPropertiesPanel, toolHasProperties } from "./ToolProperties";
import { ImageInsertButton } from "./ImageInsertSheet";
import { DockGrip, dockSizeClasses, dockStackClasses, flyoutSide } from "./DockControls";
import { isVerticalDock, TOOLBAR_ITEMS, type ToolbarItemId } from "@lib/whiteboard/types";
import { LaserIcon } from "./LaserPointer";
import { ShapeAssistIcon } from "./ShapeAssist";
import { ToolbarCustomizer } from "./ToolbarSettings";
import { TextSelectIcon } from "./PdfTextSelect";
import { useWhiteboard } from "@store/whiteboard";
import {
  PEN_COLORS,
  STROKE_WIDTHS,
  STICKY_COLORS,
  BUILTIN_LINK_TYPES,
  LINK_TYPE_COLORS,
  LINK_TYPE_LABELS,
  PEN_TYPE_PRESETS,
  PEN_TYPES,
  isStrokeTool,
  getToolColor,
  getToolSize,
  setToolColor,
  setToolSize,
  type StrokeStyle,
  type ToolType,
  type ConnectionRouting,
  type BuiltinLinkType,
  type StickyColor,
  type TouchInputMode,
} from "@lib/whiteboard/types";
import { Popover, PopoverContent, PopoverTrigger } from "@components/ui/popover";
import { SelectToolButton } from "./SelectToolButton";
import { useLongPress } from "@hooks/use-long-press";
import { Tooltip, TooltipContent, TooltipProvider, TooltipTrigger } from "@components/ui/tooltip";
import { Separator } from "@components/ui/separator";
import { StepperRow } from "./StepperControl";
import { cn } from "@lib/utils";

interface ToolbarProps {
  onOpenGraph: () => void;
  onOpenSearch: () => void;
}

/**

```

```

* Zoom in/out, percentage and fit. A standalone pill so it can live inside
* the dock row – or, when the dock itself is on top, detached at the bottom
* of the page where it never hangs over the canvas under the tools.
* Hidden entirely via Settings > Canvas when you zoom by gesture alone.
*/
/**
* Zoom readout, bottom-left of the canvas.
*
* It used to ride inside the tool dock, which cost the tool row ~150px of
* width it could not spare – on a narrow pane the row wrapped and the page
* controls were pushed onto a line of their own – and it needed a special case
* that detached it to the bottom of the screen whenever the dock sat on top.
* Pinned to its own corner it is the same control in every dock position, and
* the tool row gets its width back.
*
* It shows itself while the zoom is CHANGING and fades out about a second
* later: the percentage matters while you are pinching and never again, so it
* has no business occupying a corner permanently. It stays up while the
* pointer is on it, so the buttons are still usable once it has appeared, and
* while hidden it is `pointer-events-none` – an invisible pill that swallowed
* clicks in the corner of the canvas would be worse than the space it saved.
*/
export function ZoomDock() {
  const zoom = useWhiteboard((s) => s.camera.zoom);
  const zoomTo = useWhiteboard((s) => s.zoomTo);
  const zoomToFit = useWhiteboard((s) => s.zoomToFit);
  const zoomDockOpen = useWhiteboard((s) => s.zoomDockOpen);
  const pagePanelOpen = useWhiteboard((s) => s.pagePanelOpen);

  const [changing, setChanging] = useState(false);
  const [hovered, setHovered] = useState(false);
  const firstRender = useRef(true);

  useEffect(() => {
    // Mounting is not a zoom change; without this it flashes on every load.
    if (firstRender.current) {
      firstRender.current = false;
      return;
    }
    setChanging(true);
    const t = setTimeout(() => setChanging(false), 1400);
    return () => clearTimeout(t); // each change restarts the countdown
  }, [zoom]);

  if (!zoomDockOpen) return null;
  const show = changing || hovered;

  return (
    <div
      onPointerEnter={() => setHovered(true)}
      onPointerLeave={() => setHovered(false)}
      // Opacity is an inline style, not `opacity-0`/`opacity-100`: this build
      // does not generate an `opacity-100` rule, so the class flipped without
      // the pill ever becoming visible. Inline wins outright and needs nothing
      // generated.
      // Beside the page panel when it is open, not under it.
      style={{ opacity: show ? 1 : 0, left: pagePanelOpen ? PANEL_W + 12 : undefined }}
      className={cn(
        // Under the tool dock (z-30): where a narrow pane makes them meet, a
        // tap aimed at a tool must reach the tool, not "Zoom out".
        "absolute bottom-3 left-3 z-20 flex select-none items-center gap-0.5 rounded-lg border border-border b
          g-white/95 p-0.5 shadow-lg backdrop-blur-md transition-opacity duration-200 dark:bg-zinc-900/95",
        !show && "pointer-events-none"
      )}
    >
      <button
        onClick={() => zoomTo(zoom * 0.85)}
        className="rounded-md p-1 text-muted-foreground hover:bg-muted hover:text-foreground"
        title="Zoom out"
        aria-label="Zoom out"
      >
        <ZoomOut size={13} />
      </button>

      <button
        onClick={() => zoomTo(1)}
        className="rounded-md px-1 py-0.5 text-[10px] font-semibold tabular-nums hover:bg-muted"
        title="Reset zoom to 100%"

```

```

      aria-label="Reset zoom to 100%"
    >
      {Math.round(zoom * 100)}%
    </button>

    <button
      onClick={() => zoomTo(zoom * 1.15)}
      className="rounded-md p-1 text-muted-foreground hover:bg-muted hover:text-foreground"
      title="Zoom in"
      aria-label="Zoom in"
    >
      <ZoomIn size={13} />
    </button>

    <button
      onClick={() => zoomToFit()}
      className="rounded-md p-1 text-muted-foreground hover:bg-muted hover:text-foreground"
      title="Fit view to content (Shift+1)"
      aria-label="Fit view to content"
    >
      <Maximize size={13} />
    </button>
  </div>
);
}

/**
 * Lets the tool bar scroll along its length by swipe or drag - finger, S Pen
 * or mouse alike - with a little glide after a flick. Handled here rather than
 * by native scrolling because the pen and the mouse do not scroll a box by
 * dragging it, and mixing native and hand-rolled scrolling fights itself.
 *
 * A press that moves more than a few pixels is a scroll: the button it began
 * on does not also get clicked. The edges fade where tools are hidden.
 */
function useDragScroll(vertical: boolean, contentKey: string) {
  const ref = useRef<HTMLDivElement>(null);
  const [edges, setEdges] = useState({ start: false, end: false });

  useEffect(() => {
    const el = ref.current;
    if (!el) return;
    const pos = () => (vertical ? el.scrollTop : el.scrollLeft);
    const setPos = (v: number) => {
      if (vertical) el.scrollTop = v;
      else el.scrollLeft = v;
    };
    const update = () => {
      const max = vertical ? el.scrollHeight - el.clientHeight : el.scrollWidth - el.clientWidth;
      const next = { start: pos() > 1, end: pos() < max - 1 };
      setEdges(e => (e.start === next.start && e.end === next.end ? e : next));
    };
    update();
    el.addEventListener("scroll", update, { passive: true });
    const ro = new ResizeObserver(update);
    ro.observe(el);
    for (const child of Array.from(el.children)) ro.observe(child);

    let drag: { id: number; start: number; from: number; moved: boolean; lastAt: number; last: number; v: number; slop: number } | null = null;
    let blocker: ((ev: Event) => void) | null = null;
    let glide = 0;
    const coord = (e: PointerEvent) => (vertical ? e.clientY : e.clientX);

    const down = (e: PointerEvent) => {
      if (e.pointerType === "mouse" && e.button !== 0) return;
      cancelAnimationFrame(glide);
      // A click blocker left over from the last swipe must not eat this tap
      // (Chrome sends no click at the end of a touch or pen drag at all).
      if (blocker) {
        el.removeEventListener("click", blocker, { capture: true });
        blocker = null;
      }
      // Nothing to scroll: every press is simply a press, however it wobbles.
      const scrollable = vertical ? el.scrollHeight > el.clientHeight + 1 : el.scrollWidth > el.clientWidth + 1;
      if (!scrollable) return;
      const c = coord(e);

```

```

    drag = {
      id: e.pointerId,
      start: c,
      from: pos(),
      moved: false,
      lastAt: e.timeStamp,
      last: c,
      v: 0,
      slop: e.pointerType === "touch" ? 12 : 8,
    };
  };
  const move = (e: PointerEvent) => {
    if (!drag || e.pointerId !== drag.id) return;
    const c = coord(e);
    const d = c - drag.start;
    if (!drag.moved) {
      if (Math.abs(d) < drag.slop) return;
      drag.moved = true;
      // From here the bar owns the pointer: the button under the finger
      // loses it (and with it any pending long press or click).
      try {
        el.setPointerCapture(e.pointerId);
      } catch {
        /* pointer already gone */
      }
    }
    const dt = Math.max(1, e.timeStamp - drag.lastAt);
    drag.v = 0.8 * ((c - drag.last) / dt) + 0.2 * drag.v;
    drag.last = c;
    drag.lastAt = e.timeStamp;
    setPos(drag.from - d);
  };
  const up = (e: PointerEvent) => {
    if (!drag || e.pointerId !== drag.id) return;
    const { moved, lastAt } = drag;
    // Held still before letting go: no glide.
    const v = e.type === "pointercancel" || e.timeStamp - lastAt > 80 ? 0 : drag.v;
    drag = null;
    if (!moved) return;
    // The press was a scroll: swallow the click it may end with (a mouse
    // does send one). Removed by the next press, whatever happens.
    const block = (ev: Event) => {
      ev.stopPropagation();
      ev.preventDefault();
      el.removeEventListener("click", block, { capture: true });
      if (blocker === block) blocker = null;
    };
    blocker = block;
    el.addEventListener("click", block, { capture: true });
    // Glide on after a flick, slowing to a stop.
    let vel = v * 16; // px per frame
    const step = () => {
      if (Math.abs(vel) < 0.5) return;
      setPos(pos() - vel);
      vel *= 0.92;
      glide = requestAnimationFrame(step);
    };
    glide = requestAnimationFrame(step);
  };

  el.addEventListener("pointerdown", down);
  el.addEventListener("pointermove", move);
  el.addEventListener("pointerup", up);
  el.addEventListener("pointercancel", up);
  return () => {
    cancelAnimationFrame(glide);
    if (blocker) el.removeEventListener("click", blocker, { capture: true });
    el.removeEventListener("scroll", update);
    ro.disconnect();
    el.removeEventListener("pointerdown", down);
    el.removeEventListener("pointermove", move);
    el.removeEventListener("pointerup", up);
    el.removeEventListener("pointercancel", up);
  };
}, [vertical, contentKey]);

// Fade the edge where more tools are waiting.
const fade = 28;

```

```

const dir = vertical ? "to bottom" : "to right";
const mask =
  edges.start || edges.end
    ? `linear-gradient(${dir}, ${edges.start ? "transparent" : "black"} 0, black ${edges.start ? fade : 0}px
      , black calc(100% - ${edges.end ? fade : 0}px), ${edges.end ? "transparent" : "black"} 100%)`
    : undefined;
const style: React.CSSProperties = {
  // The bar handles its own dragging; the browser must not also pan.
  touchAction: "none",
  maskImage: mask,
  WebkitMaskImage: mask,
};
return { ref, style };
}

export function Toolbar({ onOpenGraph, onOpenSearch }: Partial<ToolbarProps>) {
  const tool = useWhiteboard((s) => s.tool);
  const setTool = useWhiteboard((s) => s.setTool);
  const style = useWhiteboard((s) => s.style);
  const setStyle = useWhiteboard((s) => s.setStyle);
  const toolbarPosition = useWhiteboard((s) => s.toolbarPosition);
  const toolPropsCollapsed = useWhiteboard((s) => s.toolPropsBarCollapsed);
  const toggleToolPropsBar = useWhiteboard((s) => s.toggleToolPropsBar);
  const pagePanelOpen = useWhiteboard((s) => s.pagePanelOpen);
  const lastStrokeTool = useWhiteboard((s) => s.lastStrokeTool);
  const [toolPanelOpen, setToolPanelOpen] = useState(false);
  const penLongPress = useLongPress(() => {
    if (!isStrokeTool(tool)) setTool(lastStrokeTool);
    setToolPanelOpen(true);
  });
  const laserLongPress = useLongPress(() => {
    setTool("laser");
    setToolPanelOpen(true);
  });
  const toolbarItems = useWhiteboard((s) => s.toolbarItems);
  const shapeRecognition = useWhiteboard((s) => s.shapeRecognition);
  const toggleShapeRecognition = useWhiteboard((s) => s.toggleShapeRecognition);
  const scroller = useDragScroll(isVerticalDock(toolbarPosition), toolbarItems.join());

  const isPenTool = isStrokeTool(tool);
  const isShapeTool = ["rectangle", "ellipse", "diamond", "triangle", "star", "cloud", "line", "arrow"].includes(tool);

  // Pen type icon lookup
  // The pen flyout edits whichever stroke tool is active. Reading/writing
  // penColor/penSize unconditionally meant picking a colour while the
  // highlighter (or marker/brush/calligraphy/pencil) was selected changed the
  // PEN and left the active tool untouched – the canvas reads each tool's own
  // field via getToolColor/getToolSize.
  // The pen the button represents: the one in hand, or the one you last used.
  const penFlyoutTool: ToolType = isPenTool ? tool : lastStrokeTool;
  const activeInkColor = getToolColor(style, penFlyoutTool);
  const activeInkSize = getToolSize(style, penFlyoutTool);
  const setInkColor = (c: string) => setStyle(setToolColor(style, penFlyoutTool, c));
  const setInkSize = (sz: number) => setStyle(setToolSize(style, penFlyoutTool, sz));

  const getPenIcon = (st: StrokeStyle) => {
    switch (st) {
      case "highlighter": return Highlighter;
      case "marker": return Paintbrush;
      case "brush": return Brush;
      case "calligraphy": return Feather;
      case "pencil": return Pencil;
      case "pen":
      default: return Pen;
    }
  };

  // Shape icon lookup
  const getShapeIcon = (t: ToolType) => {
    switch (t) {
      case "ellipse": return Circle;
      case "diamond": return Diamond;
      case "triangle": return Triangle;
      case "star": return Star;
      case "cloud": return Cloud;
      case "line": return Minus;
    }
  };

```

```

    case "arrow": return ArrowRight;
    case "rectangle":
    default: return Square;
  }
};

const dockPos = toolbarPosition;
const vertical = isVerticalDock(dockPos);
const flyout = flyoutSide(dockPos);

// Every button the tool bar can hold. Which ones show, and in what order,
// is the user's choice (the bar's Customize button); a thin divider is drawn
// wherever two neighbours belong to different groups.
const toolbarButtons: Record<ToolbarItemId, React.ReactNode> = {
  select: <SelectToolButton vertical={vertical} />,
  pan: (
    <Tooltip>
      <TooltipTrigger asChild>
        <button
          onClick={() => setTool("pan")}
          aria-label="Pan Canvas"
          className={cn(
            "p-2.5 rounded-xl transition-all",
            tool === "pan"
              ? "bg-primary text-primary-foreground shadow-sm"
              : "hover:bg-muted text-muted-foreground hover:text-foreground"
          )}
        >
          <Hand size={18} />
        </button>
      </TooltipTrigger>
      <TooltipContent side={flyout}>Pan Canvas (H / Space+Drag)</TooltipContent>
    </Tooltip>
  ),
  textselect: (
    <Tooltip>
      <TooltipTrigger asChild>
        <button
          onClick={() => setTool("textselect")}
          aria-label="Select text"
          className={cn(
            "p-2.5 rounded-xl transition-all",
            tool === "textselect"
              ? "bg-primary text-primary-foreground shadow-sm"
              : "hover:bg-muted text-muted-foreground hover:text-foreground"
          )}
        >
          <TextSelectIcon size={18} />
        </button>
      </TooltipTrigger>
      <TooltipContent side={flyout}>
        Select text (I)
        <span className="block text-[10px] opacity-70">Copy, highlight or underline the text of PDF, Word an
          d PowerPoint pages</span>
      </TooltipContent>
    </Tooltip>
  ),
  pen: (
    <
      /* Pen – SELECTION only. Its colour, thickness and pen family live in
       the contextual properties row and its ■ panel, so the same controls
       no longer exist in two places. */
      <Tooltip>
        <TooltipTrigger asChild>
          <button
            {...penLongPress.handlers}
            aria-label="Pen tool"
            onClick={() => {
              // A long press already opened the picker; don't also re-select.
              if (penLongPress.firedRef.current) return;
              setTool(isPenTool ? tool : lastStrokeTool);
            }}
            // Double-click the tool you are already holding to open its
            // settings – the same gesture desktop apps use for tool options.
            // Selecting a pen and choosing WHICH pen are different intents,
            // so a single click must not open a panel over the canvas.
            onDoubleClick={() => {
              if (!isPenTool) setTool(lastStrokeTool);
            }}
          >
            <PenIcon size={18} />
          </button>
        </TooltipTrigger>
        <TooltipContent side={flyout}>
          Pen (P)
          <span className="block text-[10px] opacity-70">Draw a line or shape with the
            pen tool. Select a color and thickness from the contextual
            properties row.
          </span>
        </TooltipContent>
      </Tooltip>
    >

```

```

        setToolPanelOpen(true);
    }}
    className={cn(
      "relative flex items-center p-2.5 rounded-xl transition-all",
      isPenTool
        ? "bg-primary text-primary-foreground shadow-sm"
        : "hover:bg-muted text-muted-foreground hover:text-foreground"
    )}
  >
    {React.createElement(getPenIcon((isPenTool ? tool : lastStrokeTool) as StrokeStyle), { size: 18 })}
    <span
      className="absolute bottom-1 right-1 w-2 h-2 rounded-full border border-background"
      style={{ backgroundColor: activeInkColor }}
    />
  </button>
</TooltipTrigger>
<TooltipContent side={flyout}>
  {PEN_TYPE_PRESETS[(isPenTool ? tool : lastStrokeTool) as StrokeStyle]?.label ?? "Pen"} (P)
  <span className="block text-[10px] opacity-70">Double-click for pen types</span>
</TooltipContent>
</Tooltip>
</>
),
eraser: (
  </* Eraser */>
  <Tooltip>
    <TooltipTrigger asChild>
      <button
        onClick={() => setTool("eraser")}
        aria-label="Eraser tool"
        // Same gesture as the pen: double-click the tool to open its
        // settings, so the rule is "double-click any tool" rather than a
        // one-off for one button.
        onDoubleClick={() => {
          setTool("eraser");
          setToolPanelOpen(true);
        }}
        className={cn(
          "p-2.5 rounded-xl transition-all",
          tool === "eraser"
            ? "bg-primary text-primary-foreground shadow-sm"
            : "hover:bg-muted text-muted-foreground hover:text-foreground"
        )}
      >
        <Eraser size={18} />
      </button>
    </TooltipTrigger>
    <TooltipContent side={flyout}>Eraser (E)</TooltipContent>
  </Tooltip>
</>
),
laser: (
  <Tooltip>
    <TooltipTrigger asChild>
      <button
        onClick={() => setTool("laser")}
        aria-label="Laser pointer"
        // Double-click (or long-press on the tablet) for its type, size and colour.
        onDoubleClick={() => {
          setTool("laser");
          setToolPanelOpen(true);
        }}
        {...laserLongPress.handlers}
        className={cn(
          "relative p-2.5 rounded-xl transition-all",
          tool === "laser"
            ? "bg-primary text-primary-foreground shadow-sm"
            : "hover:bg-muted text-muted-foreground hover:text-foreground"
        )}
      >
        <LaserIcon size={18} />
        <span
          className="absolute bottom-1 right-1 w-2 h-2 rounded-full border border-background"
          style={{ backgroundColor: getToolColor(style, "laser"), boxShadow: `0 0 4px ${getToolColor(style, "laser")}` }}
        />

```

```

    </button>
  </TooltipTrigger>
  <TooltipContent side={flyout}>
    Laser pointer (K)
    <span className="block text-[10px] opacity-70">Points without writing. Double-click for its settings
    </span>
  </TooltipContent>
</Tooltip>
),
autoshape: (
  <Tooltip>
    <TooltipTrigger asChild>
      <button
        onClick={() => {
          // Turning it on means you are about to draw a shape: pick up
          // the pen if something else is in hand.
          if (!shapeRecognition && !isStrokeTool(tool)) setTool(lastStrokeTool);
          toggleShapeRecognition();
        }}
        aria-label="Shape recognition"
        aria-pressed={shapeRecognition}
        className={cn(
          "relative p-2.5 rounded-xl transition-all",
          shapeRecognition
            ? "bg-primary/12 text-primary ring-1 ring-primary/40"
            : "hover:bg-muted text-muted-foreground hover:text-foreground"
        )}
      >
        <ShapeAssistIcon size={18} />
        {shapeRecognition && (
          <span className="absolute right-1 top-1 h-1.5 w-1.5 rounded-full bg-primary" />
        )}
      </button>
    </TooltipTrigger>
    <TooltipContent side={flyout}>
      Shape recognition: {shapeRecognition ? "on" : "off"}
      <span className="block text-[10px] opacity-70">
        Draw a circle, line or box with the pen and it becomes the exact shape
      </span>
    </TooltipContent>
  </Tooltip>
),
shapes: (
  <
    { /* Shapes Flyout */ }
    <Popover>
      <PopoverTrigger asChild>
        <button
          onClick={() => {
            if (!isShapeTool) setTool("rectangle");
          }}
          aria-label="Shapes tool"
          className={cn(
            "flex items-center gap-1 p-2.5 rounded-xl transition-all",
            isShapeTool
              ? "bg-primary text-primary-foreground shadow-sm"
              : "hover:bg-muted text-muted-foreground hover:text-foreground"
            )}
        >
          {React.createElement(getShapeIcon(isShapeTool ? tool : "rectangle"), { size: 18 })}
          <ChevronDown size={12} className="opacity-60" />
        </button>
      </PopoverTrigger>
      <PopoverContent side={flyout} className="w-56 p-2" align="center">
        <div className="px-2 py-1 text-xs font-semibold text-muted-foreground uppercase tracking-wider">
          Shapes & Primitives
        </div>
        <div className="grid grid-cols-4 gap-1 p-1">
          {[
            { t: "rectangle" as ToolType, icon: Square, label: "Rectangle (R)" },
            { t: "ellipse" as ToolType, icon: Circle, label: "Ellipse (O)" },
            { t: "diamond" as ToolType, icon: Diamond, label: "Diamond (D)" },
            { t: "triangle" as ToolType, icon: Triangle, label: "Triangle" },
            { t: "star" as ToolType, icon: Star, label: "Star" },
            { t: "cloud" as ToolType, icon: Cloud, label: "Cloud" },
            { t: "line" as ToolType, icon: Minus, label: "Line (L)" },
            { t: "arrow" as ToolType, icon: ArrowRight, label: "Arrow (A)" },
          ].map(({ t, icon: Icon, label }) => (

```

```

        <button
          key={t}
          onClick={() => setTool(t)}
          title={label}
          className={cn(
            "p-2.5 rounded-lg flex items-center justify-center transition-colors",
            tool === t ? "bg-primary text-primary-foreground" : "hover:bg-muted text-foreground"
          )}
        >
          <Icon size={18} />
        </button>
      )}
    </div>

    { /* Shape Border Width Slider */ }
    <div className="space-y-1.5 pt-2 border-t border-border/50">
      <div className="flex items-center justify-between">
        <span className="text-[11px] font-medium text-muted-foreground">Border Width</span>
        <span className="text-[11px] font-bold tabular-nums px-1.5 py-0.2 rounded bg-primary/10 text-primary">
          {style.strokeWidth || 2} px
        </span>
      </div>
      <StepperRow
        value={style.strokeWidth || 2}
        min={1}
        max={32}
        onChange={({strokeWidth}) => setStyle({ strokeWidth })}
      >
        <input
          type="range"
          min="1"
          max="32"
          step="1"
          value={style.strokeWidth || 2}
          aria-label="Border width"
          onChange={(e) => {
            const sz = Number(e.target.value);
            setStyle({ strokeWidth: sz });
          }}
          className="w-full h-1.5 bg-muted rounded-lg appearance-none cursor-pointer accent-primary"
        />
      </StepperRow>
    </div>

    { /* Shape Border Colors */ }
    <div className="space-y-1 pt-1">
      <span className="text-[11px] text-muted-foreground">Border Color</span>
      <div className="grid grid-cols-5 gap-1">
        {PEN_COLORS.slice(0, 10).map((c) => (
          <button
            key={c}
            onClick={() => setStyle({ shapeStrokeColor: c, shapeFillColor: c, strokeColor: c })}
            className={cn(
              "w-5 h-5 rounded-full border transition-transform",
              style.shapeStrokeColor === c ? "scale-110 border-foreground ring-1 ring-primary" : "border-border"
            )}
            style={{ backgroundColor: c }}
          />
        ))}
      </div>
    </div>
  </PopoverContent>
</Popover>
</>
),
connect: (
  <
    { /* Smart Connector Tool */ }
    <Popover>
      <PopoverTrigger asChild>
        <button
          onClick={() => setTool("connect")}
          aria-label="Smart connector tool"
          className={cn(
            "flex items-center gap-1 p-2.5 rounded-xl transition-all",
            tool === "connect"

```

```

      ? "bg-primary text-primary-foreground shadow-sm ring-2 ring-primary/40"
      : "hover:bg-muted text-muted-foreground hover:text-foreground"
    )}
  >
    <Link2 size={18} />
    <ChevronDown size={12} className="opacity-60" />
  </button>
</PopoverTrigger>
<PopoverContent side={flyout} className="w-72 p-3 space-y-3" align="center">
  <div className="text-xs font-semibold text-muted-foreground uppercase tracking-wider">
    Smart Connectors & Relationships
  </div>

  {/* Routing style */}
  <div className="space-y-1.5">
    <span className="text-xs text-muted-foreground">Arrow Routing</span>
    <div className="grid grid-cols-3 gap-1">
      {[["curved", "orthogonal", "straight"] as ConnectionRouting[]].map((r) => (
        <button
          key={r}
          onClick={() => {
            setStyle({ connectionRouting: r });
            setTool("connect");
          }}
          className={cn(
            "py-1.5 px-2 rounded-lg border text-xs capitalize transition-colors font-medium",
            style.connectionRouting === r
              ? "border-primary bg-primary/10 text-primary"
              : "border-border hover:bg-muted text-foreground"
          )}
        >
          {r}
        </button>
      ))}
    </div>
  </div>

  {/* Semantic Relationship Types */}
  <div className="space-y-1.5">
    <span className="text-xs text-muted-foreground">Relationship Type</span>
    <div className="space-y-1 max-h-44 overflow-y-auto pr-1">
      {BUILTIN_LINK_TYPES.map((lt) => (
        <button
          key={lt}
          onClick={() => {
            setStyle({ connectionLinkType: lt });
            setTool("connect");
          }}
          className={cn(
            "w-full text-left px-2.5 py-1.5 rounded-lg text-xs flex items-center justify-between transition-colors",
            style.connectionLinkType === lt ? "bg-muted font-semibold" : "hover:bg-muted/60"
          )}
        >
          <div className="flex items-center gap-2">
            <span
              className="w-2.5 h-2.5 rounded-full"
              style={{ backgroundColor: LINK_TYPE_COLORS[lt] }}
            />
            <span>{LINK_TYPE_LABELS[lt]}</span>
          </div>
          {style.connectionLinkType === lt && <Check size={12} className="text-primary" />}
        </button>
      ))}
    </div>
  </div>
</PopoverContent>
</Popover>
</>
),
text: (
  <>
    {/* Text Tool */}
    <Tooltip>
      <TooltipTrigger asChild>
        <button
          onClick={() => setTool("text")}
          aria-label="Text tool"

```

```

    // Same gesture as the pen: double-click the tool to open its
    // settings, so the rule is "double-click any tool" rather than a
    // one-off for one button.
    onDoubleClick={() => {
      setTool("text");
      setToolPanelOpen(true);
    }}
    className={cn(
      "p-2.5 rounded-xl transition-all",
      tool === "text"
        ? "bg-primary text-primary-foreground shadow-sm"
        : "hover:bg-muted text-muted-foreground hover:text-foreground"
    )}
  >
    <Type size={18} />
  </button>
</TooltipTrigger>
<TooltipContent side={flyout}>Text & Markdown (T)</TooltipContent>
</Tooltip>
</>
),
sticky: (
  <>
    { /* Sticky Note Tool */ }
    <Tooltip>
      <TooltipTrigger asChild>
        <button
          onClick={() => setTool("sticky")}
          aria-label="Sticky note tool"
          // Same gesture as the pen: double-click the tool to open its
          // settings, so the rule is "double-click any tool" rather than a
          // one-off for one button.
          onDoubleClick={() => {
            setTool("sticky");
            setToolPanelOpen(true);
          }}
          className={cn(
            "p-2.5 rounded-xl transition-all",
            tool === "sticky"
              ? "bg-primary text-primary-foreground shadow-sm"
              : "hover:bg-muted text-muted-foreground hover:text-foreground"
          )}
        >
          <StickyNote size={18} />
        </button>
      </TooltipTrigger>
      <TooltipContent side={flyout}>Sticky Note (N)</TooltipContent>
    </Tooltip>
  </>
),
image: <ImageInsertButton flyout={flyout} />,
};
const groupOf = (id: ToolbarItemId) => TOOLBAR_ITEMS.find((t) => t.id === id)?.group;

return (
  <TooltipProvider delayDuration={300}>
    { /* Main Floating Tool Dock */ }
    { /* Both bottom docks live in ONE wrapping row.
      They used to be independently positioned (tools centered, zoom pinned
      right), which is fine on a wide window but overlaps as soon as the
      window narrows – the zoom dock covered the right-hand tools and made
      them unclickable. In a shared flex row they sit side by side when
      there is room and stack when there is not, so they can never collide
      at any window size. */ }
    <div
      // Measured by the favourite pens bar, which must never cover the
      // toolbar (see the exclusion in its clamp).
      data-wb-toolbar
      className={cn(
        "absolute z-30 flex w-max gap-2 transition-[left] duration-150",
        dockStackClasses(dockPos),
        dockSizeClasses(dockPos)
      )}
      style={dockPos === "left" && pagePanelOpen ? { left: PANEL_W + 16 } : undefined}
    >
      { /* Row 1: properties of the ACTIVE tool (colour, thickness, settings).
        Selecting a tool and configuring it are separate jobs, so they get

```

```

        separate rows – the pattern Samsung Notes / Concepts use. */}
<ToolPropertiesBar onOpenPanel={() => setToolPanelOpen(true)} />

<div
  className={cn(
    "flex items-center justify-center gap-1.5",
    // Never wrap: the page controls stay on the LEFT of the tools, on
    // the same line. A wrapping row dropped them onto a line of their
    // own between the colour bar and the tools; now the tools pill
    // (min-w-0 below) shrinks and scrolls instead.
    vertical ? "min-h-0 max-h-full flex-col" : "max-w-full flex-nowrap"
  )}
>
  {/* Which page you are on, on the left of the tools, compact. */}
  <NotebookPageControls />
  {/* The tools pill and its chevron travel together: `min-w-0` lets the
    pill shrink and scroll while the chevron keeps its place at the
    end. Without this group the chevron is just another flex child of
    a wrapping row, and on a narrow pane it drops onto its own line. */}
  <div className={cn("flex min-w-0 max-w-full items-center gap-1.5", vertical && "min-h-0 max-h-full flex-col")}>
    {/* One line of tools, never two: when they do not fit, swipe or
      drag the bar sideways (up and down on a side dock). The edges fade
      where more tools are waiting. */}
    <div
      ref={scroller.ref}
      className={cn(
        "flex items-center gap-1.5 p-1.5 bg-white/95 dark:bg-zinc-900/95 backdrop-blur-xl shadow-2xl border
          r border-border/80 rounded-2xl select-none",
        "[scrollbar-width:none] [&::-webkit-scrollbar]:hidden [&*]:shrink-0",
        vertical ? "max-h-full min-h-0 flex-col overflow-y-auto" : "min-w-0 max-w-full flex-nowrap overflow-x-auto"
      )}
      style={scroller.style}
    >
      <DockGrip />
      {toolbarItems.map((id, i) => (
        <React.Fragment key={id}>
          {i > 0 && groupOf(toolbarItems[i - 1]) !== groupOf(id) && (
            <Separator
              orientation={vertical ? "horizontal" : "vertical"}
              className={cn("shrink-0", vertical ? "w-6 h-px my-0.5" : "h-6 w-px mx-0.5")}
            />
          )}
          {toolbarButtons[id]}
        </React.Fragment>
      )]}
    </div>

    {/* Customise the tool bar: which tools it holds and in what order.
      The LAST item of the bar itself - it belongs to the bar it
      changes, and on a narrow screen you drag the bar to its end to
      reach it, like any other tool. */}
    <Separator
      orientation={vertical ? "horizontal" : "vertical"}
      className={cn("shrink-0", vertical ? "w-6 h-px my-0.5" : "h-6 w-px mx-0.5")}
    />
    <Popover>
      <Tooltip>
        <TooltipTrigger asChild>
          <PopoverTrigger asChild>
            <button
              aria-label="Customize tool bar"
              className="p-2.5 rounded-xl text-muted-foreground transition-all hover:bg-muted hover:text-foreground data-[state=open]:bg-primary data-[state=open]:text-primary-foreground"
            >
              <SlidersHorizontal size={18} />
            </button>
          </PopoverTrigger>
        </TooltipTrigger>
        <TooltipContent side={flyout}>Customize tool bar</TooltipContent>
      </Tooltip>
      <PopoverContent
        side={flyout}
        align="end"
        collisionPadding={12}
        className="w-80 max-w-[calc(100vw-2rem)] p-0"
      >
        <div className="border-b border-border/70 px-3 py-2.5">

```

```

      <div className="text-sm font-semibold">Tool bar</div>
      <div className="text-[11px] text-muted-foreground">Choose the tools and their order</div>
    </div>
    <div className="max-h-[min(60vh,28rem)] overflow-y-auto py-2">
      <ToolbarCustomizer />
    </div>
  </PopoverContent>
</Popover>

</div>

{/* Fold the active tool's properties row away.
  It lives OUTSIDE the scrolling pill on purpose: everything in that
  pill slides when the tools overflow, and a control that scrolls off
  the end is a control you cannot reach exactly when the bar is
  crowded enough to need it. Pinned here it is always in the same
  place, and the arrow points at what the press will do. */}
{toolHasProperties(tool) && (
  <Tooltip>
    <TooltipTrigger asChild>
      <button
        onClick={toggleToolPropsBar}
        aria-label={toolPropsCollapsed ? "Show tool options" : "Hide tool options"}
        aria-expanded={!toolPropsCollapsed}
        className="flex shrink-0 items-center justify-center rounded-2xl border border-border/80 bg-wh
          ite/95 p-2 text-muted-foreground shadow-2xl backdrop-blur-xl transition-colors hover:bg-muted
          hover:text-foreground dark:bg-zinc-900/95"
        >
          {vertical
            ? (toolPropsCollapsed
              ? <ChevronRight size={16} />
              : <ChevronLeft size={16} />)
            : (toolPropsCollapsed
              ? <ChevronUp size={16} />
              : <ChevronDown size={16} />)}
          </button>
        </TooltipTrigger>
        <TooltipContent side={flyout}>
          {toolPropsCollapsed ? "Show tool options" : "Hide tool options"}
        </TooltipContent>
      </Tooltip>
    )}

  </div>

</div>
</div>

{/* Its own corner, in every dock position. */}
<ZoomDock />

<ToolPropertiesPanel open={toolPanelOpen} onClose={() => setToolPanelOpen(false)} />
</TooltipProvider>
);
}

=====
// === END FILE: src/components/whiteboard/Toolbar.tsx ===
=====

=====
// === START FILE: src/components/whiteboard/ToolbarSettings.tsx ===
// File 86 of 194 | Lines: 176 | Size: 6249 bytes
=====

"use client";

/**
 * The tool bar's own "Customize" panel: choose which buttons it holds and in
 * what order. Removing one only hides the button - the tool, its settings and its
 * shortcut key all stay - and "Reset" brings back the full default bar.
 */

import React from "react";
import {
  ArrowDown,

```

```

    ArrowUp,
    Eraser,
    Hand,
    Image as ImageIcon,
    Link2,
    MinusCircle,
    MousePointer2,
    Pen,
    PlusCircle,
    RotateCcw,
    Shapes,
    StickyNote,
    Type,
  } from "lucide-react";
import { useWhiteboard } from "@store/whiteboard";
import { ALL_TOOLBAR_ITEMS, DEFAULT_TOOLBAR_ITEMS, TOOLBAR_ITEMS, type ToolbarItemId } from "@lib/whiteboard/types";
import { LaserIcon } from "../LaserPointer";
import { ShapeAssistIcon } from "../ShapeAssist";
import { TextSelectIcon } from "../PdfTextSelect";
import { cn } from "@lib/utils";

const ICONS: Record<ToolbarItemId, React.ReactNode> = {
  select: <MousePointer2 size={15} />,
  pan: <Hand size={15} />,
  textselect: <TextSelectIcon size={15} />,
  pen: <Pen size={15} />,
  eraser: <Eraser size={15} />,
  laser: <LaserIcon size={15} />,
  autoshape: <ShapeAssistIcon size={15} />,
  shapes: <Shapes size={15} />,
  connect: <Link2 size={15} />,
  text: <Type size={15} />,
  sticky: <StickyNote size={15} />,
  image: <ImageIcon size={15} />,
};

const labelOf = (id: ToolbarItemId) => TOOLBAR_ITEMS.find((t) => t.id === id)?.label ?? id;

function IconButton({
  onClick,
  disabled,
  label,
  children,
  tone = "plain",
}: {
  onClick: () => void;
  disabled?: boolean;
  label: string;
  children: React.ReactNode;
  tone?: "plain" | "remove" | "add";
}) {
  return (
    <button
      type="button"
      onClick={onClick}
      disabled={disabled}
      title={label}
      aria-label={label}
      className={cn(
        "flex h-8 w-8 shrink-0 items-center justify-center rounded-lg transition-colors disabled:pointer-event
          s-none disabled:opacity-25",
        tone === "remove" && "text-red-500 hover:bg-red-500/10",
        tone === "add" && "text-emerald-600 hover:bg-emerald-500/10 dark:text-emerald-400",
        tone === "plain" && "text-muted-foreground hover:bg-muted hover:text-foreground"
      )}
    >
      {children}
    </button>
  );
}

export function ToolbarCustomizer() {
  const items = useWhiteboard((s) => s.toolbarItems);
  const setItems = useWhiteboard((s) => s.setToolbarItems);
  const hidden = ALL_TOOLBAR_ITEMS.filter((id) => !items.includes(id));
  const isDefault = items.join() === DEFAULT_TOOLBAR_ITEMS.join();

```

```

const move = (i: number, by: -1 | 1) => {
  const j = i + by;
  if (j < 0 || j >= items.length) return;
  const next = [...items];
  [next[i], next[j]] = [next[j], next[i]];
  setItems(next);
};
const remove = (id: ToolbarItemId) => setItems(items.filter((x) => x !== id));
// An added button goes back to its place in the default order, as near as
// the current order allows, rather than always to the end.
const add = (id: ToolbarItemId) => {
  const order = ALL_TOOLBAR_ITEMS;
  const next = [...items];
  const after = order.slice(0, order.indexOf(id)).reverse().find((x) => next.includes(x));
  next.splice(after ? next.indexOf(after) + 1 : 0, 0, id);
  setItems(next);
};

return (
  <div className="space-y-2 px-2">
    <div className="overflow-hidden rounded-xl border border-border">
      {items.map((id, i) => (
        <div
          key={id}
          className={cn("flex items-center gap-2 bg-background px-2 py-1", i > 0 && "border-t border-border/70")}
        >
          <span className="flex h-7 w-7 shrink-0 items-center justify-center rounded-lg bg-muted text-foreground">
            {ICONS[id]}
          </span>
          <span className="min-w-0 flex-1 truncate text-[13px]">{labelOf(id)}</span>
          <IconButton onClick={() => move(i, -1)} disabled={i === 0} label={`Move ${labelOf(id)} earlier`}
            <ArrowUp size={15} />
          </IconButton>
          <IconButton onClick={() => move(i, 1)} disabled={i === items.length - 1} label={`Move ${labelOf(id)} later`}
            <ArrowDown size={15} />
          </IconButton>
          <IconButton
            onClick={() => remove(id)}
            // The last button stays: an empty tool bar could not be put back
            // from itself.
            disabled={items.length === 1}
            label={`Remove ${labelOf(id)} from the tool bar`}
            tone="remove"
          >
            <MinusCircle size={16} />
          </IconButton>
        </div>
      )})}
    </div>

    {hidden.length > 0 && (
      <div>
        <div className="px-1 pb-1 text-[11px] text-muted-foreground">Not in the tool bar</div>
        <div className="overflow-hidden rounded-xl border border-dashed border-border">
          {hidden.map((id, i) => (
            <div
              key={id}
              className={cn("flex items-center gap-2 px-2 py-1", i > 0 && "border-t border-dashed border-border/70")}
            >
              <span className="flex h-7 w-7 shrink-0 items-center justify-center rounded-lg text-muted-foreground">
                {ICONS[id]}
              </span>
              <span className="min-w-0 flex-1 truncate text-[13px] text-muted-foreground">{labelOf(id)}</span>
              <IconButton onClick={() => add(id)} label={`Add ${labelOf(id)} to the tool bar`} tone="add">
                <PlusCircle size={16} />
              </IconButton>
            </div>
          )})}
        </div>
      </div>
    )}
  )}

```

```

<div className="flex items-center justify-between gap-2 px-1">
  <p className="text-[11px] leading-snug text-muted-foreground">
    Removing a button only hides it; its keyboard shortcut still works.
  </p>
  <button
    type="button"
    onClick={() => setItems([...DEFAULT_TOOLBAR_ITEMS])}
    disabled={isDefault}
    className="flex shrink-0 items-center gap-1 rounded-lg px-2 py-1 text-[11px] font-medium text-primar
      y transition-colors hover:bg-primary/10 disabled:pointer-events-none disabled:opacity-40"
  >
    <RotateCcw size={12} />
    Reset
  </button>
</div>
</div>
);
}

```

```

// === END FILE: src/components/whiteboard/ToolbarSettings.tsx ===

```

```

// === START FILE: src/components/whiteboard/TutorialModal.tsx ===
// File 87 of 194 | Lines: 348 | Size: 12659 bytes

```

```

"use client";

/**
 * Tutorial Modal
 * =====
 * A first-run onboarding modal that explains how to use the whiteboard,
 * especially the Connect feature (which was confusing users).
 *
 * Shows on first load (when no tutorial-seen flag is in localStorage).
 * Can be reopened via the Help button (?).
 */

import React, { useEffect, useState } from "react";
import { useWhiteboard } from "@store/whiteboard";
import {
  X,
  MousePointer2,
  Pen,
  Type,
  StickyNote,
  Link2,
  Network,
  Search,
  ChevronRight,
  ChevronLeft,
  Sparkles,
} from "lucide-react";
import { Button } from "@components/ui/button";

interface TutorialStep {
  title: string;
  icon: React.ComponentType<{ size?: number; className?: string }>;
  content: React.ReactNode;
}

const STEPS: TutorialStep[] = [
  {
    title: "Welcome to ProNotes",
    icon: Sparkles,
    content: (
      <div className="space-y-3 text-sm">
        <p>
          ProNotes is an infinite canvas for notes, ideas, and visual knowledge graphs –
          inspired by Microsoft Whiteboard + Obsidian Canvas.
        </p>
        <p className="text-muted-foreground">
          Let's walk through the key features. You can reopen this tutorial
          anytime with the <strong>?</strong> button (top-right).
        </p>
      </div>
    )
  }
]

```

```

    </div>
  ),
},
{
  title: "1. Draw & Write",
  icon: Pen,
  content: (
    <div className="space-y-3 text-sm">
      <p>
        <strong>Pen (P):</strong> Draw freehand with pressure-sensitive smoothing.
      </p>
      <p>
        <strong>Text (T):</strong> Click anywhere on the canvas to add text.
        Double-click empty space to quick-add text.
      </p>
      <p>
        <strong>Sticky Note (N):</strong> Add colored sticky notes.
      </p>
      <p className="text-muted-foreground">
        Each tool remembers its own color and size – switch between them
        without losing your settings.
      </p>
    </div>
  ),
},
{
  title: "2. How to CONNECT Elements",
  icon: Link2,
  content: (
    <div className="space-y-3 text-sm">
      <p className="font-medium text-primary">
        This is the key feature – here's how to connect things:
      </p>
      <ol className="space-y-2 list-decimal list-inside">
        <li>
          Press <kbd className="px-1.5 py-0.5 rounded border bg-muted text-xs">C</kbd>{" "}
          or click the <Link2 className="inline" size={14} /> <strong>Connect</strong> tool
          in the bottom toolbar (chain icon).
        </li>
        <li>
          <strong>Click and hold</strong> on the first element (e.g., a sticky note).
        </li>
        <li>
          <strong>Drag</strong> to the second element – a dashed preview line follows
          your cursor.
        </li>
        <li>
          <strong>Release</strong> on the target element → a colored arrow is created!
        </li>
      </ol>
      <p className="text-muted-foreground">
        The arrow's color depends on the <strong>link type</strong> you choose
        in the style bar (Relates = blue, Depends = red, etc.).
      </p>
    </div>
  ),
},
{
  title: "3. Link Types & Colors",
  icon: Network,
  content: (
    <div className="space-y-3 text-sm">
      <p>Each connection has a <strong>type</strong> that determines its color:</p>
      <div className="space-y-1.5">
        <div className="flex items-center gap-2">
          <span className="w-4 h-1.5 rounded-full" style={{ backgroundColor: "#3b82f6" }} />
          <span><strong>Relates to</strong> – general relationship</span>
        </div>
        <div className="flex items-center gap-2">
          <span className="w-4 h-1.5 rounded-full" style={{ backgroundColor: "#ef4444" }} />
          <span><strong>Depends on</strong> – "needs this first"</span>
        </div>
        <div className="flex items-center gap-2">
          <span className="w-4 h-1.5 rounded-full" style={{ backgroundColor: "#8b5cf6" }} />
          <span><strong>Leads to</strong> – sequence / flow order</span>
        </div>
        <div className="flex items-center gap-2">
          <span className="w-4 h-1.5 rounded-full" style={{ backgroundColor: "#8b5cf6" }} />

```

```

        <span><strong>Flows into</strong> – data / process flow</span>
      </div>
    </div>
    <p className="text-muted-foreground">
      You can also <strong>create your own link type</strong>: click "+ New type"
      in the style bar, enter a name + color. Custom types appear in the legend.
    </p>
    <p className="text-muted-foreground">
      Every arrow shows a <strong>label pill</strong> with the relationship name.
    </p>
  </div>
),
},
{
  title: "4. Folders, Boards & Graph",
  icon: Network,
  content: (
    <div className="space-y-3 text-sm">
      <p>
        <strong>Folders</strong> contain boards. Every board is automatically a
        "child" of its folder.
      </p>
      <p>
        Press <code className="px-1.5 py-0.5 rounded border bg-muted text-xs">Ctrl+G</code>{" "}
        to open the <strong>Graph View</strong> – see all boards and folders as a
        network:
      </p>
      <ul className="space-y-1 list-disc list-inside text-muted-foreground">
        <li>Squares = folders, Circles = boards</li>
        <li>Dashed lines = folder contains board</li>
        <li>Solid colored lines = wikilinks</li>
      </ul>
      <p>
        Press <code className="px-1.5 py-0.5 rounded border bg-muted text-xs">Ctrl+K</code>{" "}
        to <strong>search</strong> across all boards.
      </p>
    </div>
  ),
},
{
  title: "5. Wikilinks ([[link]])",
  icon: Type,
  content: (
    <div className="space-y-3 text-sm">
      <p>
        Besides visual arrows, you can create <strong>text links</strong> inside
        any text or sticky note:
      </p>
      <p>
        Type <code className="px-1 py-0.5 rounded bg-muted text-xs">[[Board Name]]</code>{" "}
        – it becomes a clickable blue link.
      </p>
      <p>
        Or select a text element and click the <strong>"Link"</strong> button
        that appears above it → pick a board → the wikilink is inserted automatically.
      </p>
      <p className="text-muted-foreground">
        Clicking a wikilink navigates to that board (creates it if it doesn't exist).
      </p>
    </div>
  ),
},
{
  title: "You're ready!",
  icon: Sparkles,
  content: (
    <div className="space-y-3 text-sm">
      <p className="font-medium">Quick reference:</p>
      <div className="grid grid-cols-2 gap-2 text-xs">
        <div className="flex items-center gap-1.5"><code className="px-1.5 py-0.5 rounded border bg-muted">V</code>
          <code>/kbd> Select</code></div>
        <div className="flex items-center gap-1.5"><code className="px-1.5 py-0.5 rounded border bg-muted">P</code>
          <code>/kbd> Pen</code></div>
        <div className="flex items-center gap-1.5"><code className="px-1.5 py-0.5 rounded border bg-muted">T</code>
          <code>/kbd> Text</code></div>
        <div className="flex items-center gap-1.5"><code className="px-1.5 py-0.5 rounded border bg-muted">N</code>
          <code>/kbd> Sticky</code></div>
        <div className="flex items-center gap-1.5"><code className="px-1.5 py-0.5 rounded border bg-muted">C</code>

```

```

      /kbd> Connect</div>
      <div className="flex items-center gap-1.5"><kbd className="px-1.5 py-0.5 rounded border bg-muted">Ctrl+G</kbd> Graph</div>
      <div className="flex items-center gap-1.5"><kbd className="px-1.5 py-0.5 rounded border bg-muted">Ctrl+K</kbd> Search</div>
      <div className="flex items-center gap-1.5"><kbd className="px-1.5 py-0.5 rounded border bg-muted">Ctrl+Z</kbd> Undo</div>
    </div>
    <p className="text-muted-foreground pt-2">
      Reopen this tutorial anytime with <kbd className="px-1.5 py-0.5 rounded border bg-muted text-xs">?</kbd>
    </p>
  </div>
),
},
];

const TUTORIAL_SEEN_KEY = "z-whiteboard-tutorial-seen";

export function TutorialModal() {
  const [open, setOpen] = useState(false);
  const [step, setStep] = useState(0);

  // Check if first run (no tutorial-seen flag) OR if the flag was cleared
  // by the Help button (check on mount and also listen for storage changes)
  React.useEffect(() => {
    if (typeof window === "undefined") return;
    // getItem itself can throw (private mode, disabled storage): showing the
    // tutorial again beats crashing the first-run path.
    let seen: string | null = null;
    try {
      seen = window.localStorage.getItem(TUTORIAL_SEEN_KEY);
    } catch {
      /* fall through with seen === null, i.e. show the tutorial */
    }
    if (!seen) {
      setOpen(true);
    }
    // Listen for the Help button clearing the flag
    const onStorage = (e: StorageEvent) => {
      if (e.key === TUTORIAL_SEEN_KEY && e.newValue === null) {
        setOpen(true);
        setStep(0);
      }
    };
    window.addEventListener("storage", onStorage);
    return () => window.removeEventListener("storage", onStorage);
  }, []);

  // Settings > Help > "Show the tutorial again" reopens it on demand. Without
  // this the tutorial was unreachable after the first dismissal.
  useEffect(() => {
    const reopen = () => {
      setStep(0);
      setOpen(true);
    };
    window.addEventListener("wb:open-tutorial", reopen);
    return () => window.removeEventListener("wb:open-tutorial", reopen);
  }, []);

  // NOTE: "?" intentionally does NOT open this tutorial – the canvas binds
  // "?" to the Keyboard Shortcuts modal. Opening both on one keypress stacked
  // two dialogs (tutorial on top, shortcuts hidden underneath).

  const handleClose = () => {
    setOpen(false);
    if (typeof window !== "undefined") {
      try {
        window.localStorage.setItem(TUTORIAL_SEEN_KEY, "1");
      } catch {
        /* remembering dismissal is a convenience, never a requirement */
      }
    }
  };

  if (!open) return null;

  const currentStep = STEPS[step];

```

```

const Icon = currentStep.icon;
const isLast = step === STEPS.length - 1;

return (
  <div data-wb-modal
    className="fixed inset-0 z-[100] flex items-center justify-center bg-black/50 backdrop-blur-sm">
    <div className="w-full max-w-lg bg-white dark:bg-zinc-900 rounded-2xl shadow-2xl border border-border overflow-hidden">
      {/* Header */}
      <div className="flex items-center justify-between px-5 py-3 border-b border-border bg-muted/30">
        <div className="flex items-center gap-2">
          <Icon size={18} className="text-primary" />
          <span className="text-sm font-medium">{currentStep.title}</span>
        </div>
        <div className="flex items-center gap-2">
          <span className="text-xs text-muted-foreground tabular-nums">
            {step + 1} / {STEPS.length}
          </span>
          <Button variant="ghost" size="icon" className="h-7 w-7 onClick={handleClose}>
            <X size={16} />
          </Button>
        </div>
      </div>

      {/* Content */}
      <div className="px-5 py-5 min-h-[200px]">
        {currentStep.content}
      </div>

      {/* Progress dots */}
      <div className="flex items-center justify-center gap-1.5 pb-3">
        {STEPS.map((_, i) => (
          <button
            key={i}
            className={cn(
              "h-1.5 rounded-full transition-all",
              i === step ? "w-6 bg-primary" : "w-1.5 bg-muted-foreground/30"
            )}
            onClick={() => setStep(i)}
          />
        ))}
      </div>

      {/* Footer */}
      <div className="flex items-center justify-between px-5 py-3 border-t border-border bg-muted/30">
        <Button
          variant="ghost"
          size="sm"
          disabled={step === 0}
          onClick={() => setStep(step - 1)}
        >
          <ChevronLeft size={14} className="mr-1" />
          Back
        </Button>
        {isLast ? (
          <Button size="sm" onClick={handleClose}>
            <Sparkles size={14} className="mr-1" />
            Get started
          </Button>
        ) : (
          <Button size="sm" onClick={() => setStep(step + 1)}>
            Next
            <ChevronRight size={14} className="ml-1" />
          </Button>
        )}
      </div>
    </div>
  );
};

// Inline cn to avoid import cycle in this file
function cn(...classes: (string | false | undefined)[]): string {
  return classes.filter(Boolean).join(" ");
}

=====
// === END FILE: src/components/whiteboard/TutorialModal.tsx ===

```

```
=====
// === START FILE: src/components/whiteboard/WhiteboardApp.tsx ===
// File 88 of 194 | Lines: 724 | Size: 31767 bytes
=====
```

```
"use client";

/**
 * Whiteboard App (v4 – Professional Grade Canvas & Knowledge Architecture)
 * =====
 * Top-level whiteboard application shell. Composes:
 * - Sidebar (folder + board tree) – collapsible
 * - BacklinksPanel (Obsidian-style links & backlinks)
 * - Canvas (infinite drawing surface with snap-to-grid)
 * - Toolbar (modern bottom dock with tools, quick export, and zoom controls)
 * - Minimap Navigator (real-time canvas radar with interactive pan)
 * - ExportModal (vector SVG and 2x Retina PNG exporter)
 * - KeyboardShortcutsModal (searchable shortcuts categorized overlay)
 * - ThemeToggle (dark / light / system theme switching)
 * - PropertiesPanel (contextual HUD with opacity slider and styling)
 * - Inline text editor (overlay)
 * - Graph View modal (network visualization)
 * - Search modal (global search)
 */

import React, { useCallback, useEffect, useState, useRef } from "react";
import { useWhiteboard } from "@store/whiteboard";
import { useTheme } from "next-themes";
import { WhiteboardCanvas } from "../WhiteboardCanvas";
import { Toolbar } from "../Toolbar";
import { Sidebar } from "../Sidebar";
import { PenDiagnostics } from "../PenDiagnostics";
import { BacklinksPanel } from "../BacklinksPanel";
import { SelectionActions } from "../SelectionActions";
import { ImageActions } from "../ImageActions";
import { CaptureHud } from "../CaptureHud";
import { CapturePanel } from "../CapturePanel";
import { ImageStudio } from "../ImageStudio";
import { LockedModeBar } from "../LockedModeBar";
import { PropertiesPanel } from "../PropertiesPanel";
import { InlineTextEditor } from "../InlineTextEditor";
import { GraphView } from "../GraphView";
import { SearchModal } from "../SearchModal";
import { LinkLegend } from "../LinkLegend";
import { TutorialModal } from "../TutorialModal";
import { LinkManager } from "../LinkManager";
import { Minimap } from "../Minimap";
import { FavoritePensBar } from "../FavoritePensBar";
import { NotebookPagePanelHost } from "../NotebookPages";
import { FileDropZone } from "../FileDropZone";
import { getBoardElements, isNotebook } from "@lib/whiteboard/notebook";
import { SplitPane, SplitDivider, SplitLiveOverlay } from "../SplitPane";
import { SplitPicker } from "../SplitPicker";
import { SyncPanel } from "../SyncPanel";
import { SettingsPanel } from "../SettingsPanel";
import { ExportModal } from "../ExportModal";
import { InsertPagesModal } from "../InsertPagesModal";
import { KeyboardShortcutsModal } from "../KeyboardShortcutsModal";
import { ThemeToggle } from "../ThemeToggle";
import {
  PanelLeftOpen,
  Map as MapIcon,
  HelpCircle,
  Share2,
  FilePlus2,
  Keyboard,
  Grid,
  Cloud,
  Search,
  Network,
  Settings as SettingsIcon,
  CheckSquare,
  PenLine,
  Type,
  StickyNote,

```

```

    Shapes,
    Image as ImageIcon,
    Link2,
    Undo2,
    Redo2,
  } from "lucide-react";
  import { Button } from "@components/ui/button";
  import {
    DropdownMenu,
    DropdownMenuContent,
    DropdownMenuItem,
    DropdownMenuSeparator,
    DropdownMenuTrigger,
  } from "@components/ui/dropdown-menu";
  import { cn } from "@lib/utils";
  import type { Viewport } from "@lib/whiteboard/geometry";
  import type { SearchScope } from "@lib/whiteboard/search";

  /** Undo and redo, in the top bar beside Select / Links / Search. */
  function HeaderUndoRedo() {
    const undo = useWhiteboard((s) => s.undo);
    const redo = useWhiteboard((s) => s.redo);
    const canUndo = useWhiteboard((s) => s.canUndo());
    const canRedo = useWhiteboard((s) => s.canRedo());
    const btn = "h-8 w-8 text-muted-foreground hover:text-foreground disabled:opacity-30";
    return (
      <>
        <Button variant="ghost" size="icon" className={btn} onClick={undo} disabled={!canUndo} title="Undo (Ctrl
          +Z / 2-finger tap)" aria-label="Undo">
          <Undo2 size={15} />
        </Button>
        <Button variant="ghost" size="icon" className={btn} onClick={redo} disabled={!canRedo} title="Redo (Ctrl
          +Y / 3-finger tap)" aria-label="Redo">
          <Redo2 size={15} />
        </Button>
      </>
    );
  }

  export function WhiteboardApp() {
    const init = useWhiteboard((s) => s.init);
    const isReady = useWhiteboard((s) => s.isReady);
    const sidebarOpen = useWhiteboard((s) => s.sidebarOpen);
    const splitBoardId = useWhiteboard((s) => s.splitBoardId);
    const splitRatio = useWhiteboard((s) => s.splitRatio);
    const splitSide = useWhiteboard((s) => s.splitSide);
    const stacked = useWhiteboard((s) => s.splitOrientation === "stacked");
    const splitRowRef = useRef<HTMLDivElement>(null);
    const setSidebarOpen = useWhiteboard((s) => s.setSidebarOpen);
    const findOrCreateBoardByName = useWhiteboard((s) => s.findOrCreateBoardByName);
    const setActiveBoard = useWhiteboard((s) => s.setActiveBoard);
    const selectedIds = useWhiteboard((s) => s.selectedIds);
    const selectAll = useWhiteboard((s) => s.selectAll);
    const clearSelection = useWhiteboard((s) => s.clearSelection);
    const selectByType = useWhiteboard((s) => s.selectByType);
    const invertSelection = useWhiteboard((s) => s.invertSelection);

    const { theme, setTheme, resolvedTheme } = useTheme();

    const [viewport, setViewport] = useState<Viewport>({ width: 1200, height: 800 });
    const [graphOpen, setGraphOpen] = useState(false);
    // Search opens in a scope: Ctrl+F finds in the open note, while Ctrl+K and
    // the header button search everywhere (see SearchModal). Each request gets
    // a fresh key, so asking again while the find bar is up reopens the panel
    // in the requested scope.
    const [search, setSearch] = useState<{ scope: SearchScope; key: number; query?: string; mode?: "panel" | "bar" } | null>(null);
    const searchKeyRef = useRef(0);
    const openSearch = useCallback((scope: SearchScope, query?: string, mode?: "panel" | "bar") => {
      searchKeyRef.current += 1;
      setSearch({ scope, key: searchKeyRef.current, query, mode });
    }, []);
    // "Find in note" on a PDF text selection: search this note for that text.
    useEffect(() => {
      const onOpen = (e: Event) => {
        const q = (e as CustomEvent<{ query?: string }>).detail?.query;
        // Straight to the find bar on the matches, not a panel over them.
        openSearch("note", typeof q === "string" ? q : undefined, typeof q === "string" && q ? "bar" : "panel");
      };
    });
  }

```

```

};
window.addEventListener("wb:open-search", onOpen);
return () => window.removeEventListener("wb:open-search", onOpen);
}, [openSearch]);
const [showBacklinks, setShowBacklinks] = useState(false);
const [syncOpen, setSyncOpen] = useState(false);
const [settingsOpen, setSettingsOpen] = useState(false);

// Home-screen shortcuts from manifest.json ("New Whiteboard", "Knowledge
// Graph"). The manifest has always pointed at `/?action=new` and
// `/?view=graph`, but nothing read those parameters, so both shortcuts just
// opened the app. Handled once, after the workspace has loaded (a new board
// made before that would be overwritten by the load), and then stripped from
// the address so a reload does not create a second board.
const createBoardAction = useWhiteboard((s) => s.createBoard);
const shortcutHandledRef = useRef(false);
useEffect(() => {
  if (!isReady || shortcutHandledRef.current || typeof window === "undefined") return;
  shortcutHandledRef.current = true;
  const params = new URLSearchParams(window.location.search);
  const action = params.get("action");
  const view = params.get("view");
  if (!action && !view) return;
  if (action === "new") createBoardAction("Untitled Board", null);
  if (view === "graph") setGraphOpen(true);
  params.delete("action");
  params.delete("view");
  const rest = params.toString();
  window.history.replaceState(null, "", window.location.pathname + (rest ? `?${rest}` : "") + window.location.hash);
}, [isReady, createBoardAction]);
// Off by default: it is a colour key, not a tool, and it occupied a large
// block of the canvas permanently. Turn it on in Settings > Canvas.
// Persisted in the store (like the minimap) so it survives reloads.
const showLegend = useWhiteboard((s) => s.legendOpen);
const toggleLegend = useWhiteboard((s) => s.toggleLegend);
const [linkManagerOpen, setLinkManagerOpen] = useState(false);
const [exportOpen, setExportOpen] = useState(false);
const [insertPagesOpen, setInsertPagesOpen] = useState(false);
// Inserting pages only means anything in a paged document, so the button
// stays out of a whiteboard's toolbar rather than opening to explain itself.
const isNotebookBoard = useWhiteboard((s) => isNotebook(s.boards.find((b) => b.id === s.activeBoardId)));
const [shortcutsOpen, setShortcutsOpen] = useState(false);
const containerRef = useRef<HTMLDivElement>(null);

// Initialize store from localStorage on mount
useEffect(() => {
  init();
}, [init]);

// Track viewport size for the inline text editor and tools.
//
// `isReady` is a dependency for a reason: until the workspace loads, this
// component renders a "Loading" placeholder, so containerRef is null on the
// first run and the effect returned early. With only the sidebar/backlinks
// flags as deps it never ran again, and the size stayed at the 1200x800
// default until the sidebar was toggled - so the text editor, the selection
// action bar and the minimap were all positioned for a canvas that was not
// there (on a tablet the text editor's bar ran off the screen).
useEffect(() => {
  if (!containerRef.current) return;
  const updateSize = () => {
    if (containerRef.current) {
      const rect = containerRef.current.getBoundingClientRect();
      if (rect.width > 0 && rect.height > 0) {
        setViewport({ width: rect.width, height: rect.height });
      }
    }
  };
  updateSize();

  const observer = new ResizeObserver((entries) => {
    for (const entry of entries) {
      const { width, height } = entry.contentRect;
      if (width > 0 && height > 0) {
        setViewport({ width, height });
      }
    }
  });
}

```

```

    });

    observer.observe(containerRef.current);
    window.addEventListener("resize", updateSize);
    return () => {
      observer.disconnect();
      window.removeEventListener("resize", updateSize);
    };
  }, [sidebarOpen, showBacklinks, isReady]);

// Listen for wikilink navigation events
useEffect(() => {
  const handler = (e: Event) => {
    const detail = (e as CustomEvent).detail as { target: string };
    const id = findOrCreateBoardByName(detail.target);
    setActiveBoard(id);
  };
  window.addEventListener("wb:navigate-link", handler);
  return () => window.removeEventListener("wb:navigate-link", handler);
}, [findOrCreateBoardByName, setActiveBoard]);

// Listen for auto-fit height events from text elements
useEffect(() => {
  const handler = (e: Event) => {
    const detail = (e as CustomEvent).detail as { id: string; height: number };
    const state = useWhiteboard.getState();
    const board = state.boards.find((b) => b.id === state.activeBoardId);
    if (!board) return;
    // On a notebook the element is on the active page, not on the canvas
    // field – so a growing text box never found itself and stopped
    // expanding as you typed.
    const el = getBoardElements(board).find((x) => x.id === detail.id);
    if (!el || (el.type !== "text" && el.type !== "sticky")) return;
    state.updateElement(detail.id, { height: detail.height } as never, false);
  };
  window.addEventListener("wb:auto-fit-height", handler);
  return () => window.removeEventListener("wb:auto-fit-height", handler);
}, []);

// Listen for custom modal events
useEffect(() => {
  const onOpenExport = () => setExportOpen(true);
  const onOpenInsertPages = () => setInsertPagesOpen(true);
  const onOpenShortcuts = () => setShortcutsOpen(true);
  const onOpenSettings = () => setSettingsOpen(true);
  const onToggleTheme = () => {
    const nextTheme = resolvedTheme === "dark" ? "light" : "dark";
    setTheme(nextTheme);
  };

  window.addEventListener("wb:open-export", onOpenExport);
  window.addEventListener("wb:open-insert-pages", onOpenInsertPages);
  window.addEventListener("wb:open-shortcuts", onOpenShortcuts);
  window.addEventListener("wb:open-settings", onOpenSettings);
  window.addEventListener("wb:toggle-theme", onToggleTheme);
  return () => {
    window.removeEventListener("wb:open-export", onOpenExport);
    window.removeEventListener("wb:open-shortcuts", onOpenShortcuts);
    window.removeEventListener("wb:open-settings", onOpenSettings);
    window.removeEventListener("wb:toggle-theme", onToggleTheme);
  };
}, [resolvedTheme, setTheme]);

// Global keyboard shortcuts for search and graph
useEffect(() => {
  const onKeyDown = (e: KeyboardEvent) => {
    // Never hijack typing: skip while an element is being edited inline or
    // while focus is in any input / textarea / contentEditable host.
    const editing = useWhiteboard.getState().editingId;
    if (editing) return;
    const target = e.target as HTMLElement | null;
    if (
      target &&
      (target.tagName === "INPUT" ||
        target.tagName === "TEXTAREA" ||
        target.isContentEditable)
    ) {
      return;
    }
  };

```

```

}

// Shortcuts must not fire under open modal overlays (Radix dialogs
// focus a plain DIV, so the input guard above is not enough).
if (document.querySelector('[role="dialog"][data-state="open"], [data-wb-modal]')) {
  return;
}

// Ctrl+K / Cmd+K: search everywhere
if ((e.ctrlKey || e.metaKey) && !e.shiftKey && e.code === "KeyK") {
  e.preventDefault();
  openSearch("everywhere");
}

// Ctrl+F / Cmd+F: find in the open note - what Ctrl+F means in every
// editor. With nothing open there is no note to find in.
if ((e.ctrlKey || e.metaKey) && !e.shiftKey && e.code === "KeyF") {
  e.preventDefault();
  openSearch(useWhiteboard.getState().activeBoardId ? "note" : "everywhere");
}

// Ctrl+G / Cmd+G: graph view (plain only – Ctrl+Shift+G is Snap-to-Grid)
if ((e.ctrlKey || e.metaKey) && !e.shiftKey && e.code === "KeyG") {
  e.preventDefault();
  setGraphOpen(true);
}

// Ctrl+B / Cmd+B: toggle backlinks panel
if ((e.ctrlKey || e.metaKey) && !e.shiftKey && e.code === "KeyB") {
  e.preventDefault();
  setShowBacklinks((v) => !v);
}

// Ctrl+L / Cmd+L: link manager (plain only – Ctrl+Shift+L is Lock)
if ((e.ctrlKey || e.metaKey) && !e.shiftKey && e.code === "KeyL") {
  e.preventDefault();
  setLinkManagerOpen(true);
}

// Ctrl+Shift+I / Cmd+Shift+I: select all ink strokes
if ((e.ctrlKey || e.metaKey) && e.shiftKey && e.code === "KeyI") {
  e.preventDefault();
  selectByType("stroke");
}

// Ctrl+Shift+T / Cmd+Shift+T: select all text cards
if ((e.ctrlKey || e.metaKey) && e.shiftKey && e.code === "KeyT") {
  e.preventDefault();
  selectByType("text");
}

// Ctrl+Shift+V / Cmd+Shift+V: invert selection
if ((e.ctrlKey || e.metaKey) && e.shiftKey && e.code === "KeyV") {
  e.preventDefault();
  invertSelection();
}

// Escape: deselect all if elements are selected
if (e.key === "Escape" && selectedIds.length > 0) {
  e.preventDefault();
  clearSelection();
}
};
window.addEventListener("keydown", onKeyDown);
return () => window.removeEventListener("keydown", onKeyDown);
}, [selectByType, invertSelection, clearSelection, selectedIds.length, openSearch]);

// Listen for "open link manager" events from the sidebar button
useEffect(() => {
  const handler = () => setLinkManagerOpen(true);
  window.addEventListener("wb:open-link-manager", handler);
  return () => window.removeEventListener("wb:open-link-manager", handler);
}, []);

// Storage health banner: quota-degraded saves, total save failure, recovery,
// and multi-tab clobbering – all previously silent.
const [storageWarning, setStorageWarning] = useState<string | null>(null);
useEffect(() => {
  const onDegraded = () => setStorageWarning("Storage full – images were NOT saved (boards, strokes and text were). Delete unused boards or large images.");
  const onFailed = () => setStorageWarning("Saving failed – your latest changes may be lost on reload.");
  const onRecovered = () => setStorageWarning(null);
  const onPressure = () =>
    setStorageWarning(
      "Storage is almost full – export a backup (Settings → Sync → Save backup), then delete unused boards."
    );
};

```

```

const onImageInline = () =>
  setStorageWarning(
    "A picture couldn't go into this device's image storage, so it was saved inside the board instead. A few more like that can fill up storage - export a backup if you add many images."
  );
const onExternal = () =>
  setStorageWarning("This whiteboard was just saved from another tab. Keep working in ONE tab to avoid losing changes.");
window.addEventListener("wb:save-degraded", onDegraded);
window.addEventListener("wb:save-failed", onFailed);
window.addEventListener("wb:save-recovered", onRecovered);
window.addEventListener("wb:storage-pressure", onPressure);
window.addEventListener("wb:external-change", onExternal);
window.addEventListener("wb:image-inline", onImageInline);
return () => {
  window.removeEventListener("wb:image-inline", onImageInline);
  window.removeEventListener("wb:save-degraded", onDegraded);
  window.removeEventListener("wb:save-failed", onFailed);
  window.removeEventListener("wb:save-recovered", onRecovered);
  window.removeEventListener("wb:storage-pressure", onPressure);
  window.removeEventListener("wb:external-change", onExternal);
};
}, []);

if (!isReady) {
  return (
    <div className="w-screen h-screen flex items-center justify-center bg-background">
      <div className="text-muted-foreground text-sm">Loading whiteboard...</div>
    </div>
  );
}

return (
  <div className="w-screen h-screen flex overflow-hidden bg-background text-foreground">
    { /* Sidebar */ }
    { sidebarOpen && <Sidebar /> }

    { /* Raw pointer readout, off unless switched on in Settings. */ }
    <PenDiagnostics />

    { /* Backlinks panel */ }
    { showBacklinks && <BacklinksPanel onClose={() => setShowBacklinks(false)} /> }

    { /* Storage health banner */ }
    { storageWarning && (
      <div
        data-wb-modal
        className="absolute top-2 left-1/2 -translate-x-1/2 z-[120] flex items-center gap-3 max-w-[90vw] px-4 py-2 rounded-lg bg-amber-500/95 text-amber-950 text-xs font-medium shadow-lg"
        role="alert"
      >
        <span>{storageWarning}</span>
        <button
          className="px-2 py-0.5 rounded bg-amber-900/15 hover:bg-amber-900/25 font-semibold"
          onClick={() => setStorageWarning(null)}
        >
          Dismiss
        </button>
      </div>
    ) }

    { /* Editable pane + optional reference pane. The row is what the divider measures against when you drag it. */ }
    <div ref={splitRowRef} className={cn("flex min-h-0 min-w-0 flex-1", splitBoardId && stacked && "flex-col")}>
      { /* The editable canvas and the reference pane are REORDERED rather than swapped in the tree: moving them would remount the canvas and lose its measured viewport mid-gesture. `order` keeps both mounted and just changes which side each occupies. */ }
      <div
        ref={containerRef}
        // The floating pen bar measures this box to keep itself out of the sidebar and the backlinks panel. Measuring the pane rather than assuming a sidebar width means a split view narrows it too.
        data-wb-canvas-area
        // @container: the dock's page controls go compact in a narrow pane (upright tablet with the sidebar open) - see NotebookPageControls.
        className="@container relative min-h-0 min-w-0 overflow-hidden"
      >

```

```

style={
  splitBoardId
    ? {
      order: splitSide === "left" ? 0 : 2,
      // Minus half the 6px divider each, or the two panes plus the
      // divider overflow the row and the far pane loses its edge.
      flex: `0 0 calc(${(splitSide === "left" ? splitRatio : 1 - splitRatio) * 100}% - 3px)`,
    }
    : { flex: "1 1 0%" }
}
>
{/* Which half your tools act on: framed, and named. Only in split view. */}
<SplitLiveOverlay />

{/* Top-left controls (when sidebar is hidden) */}
{!sidebarOpen && (
  <div className="absolute top-4 left-4 z-30 flex items-center gap-2">
    <Button
      variant="outline"
      size="icon"
      className="h-9 w-9 bg-white/95 dark:bg-zinc-900/95 backdrop-blur-md shadow-md"
      onClick={() => setSidebarOpen(true)}
      title="Show sidebar"
    >
    <PanelLeftOpen size={18} />
  </Button>
</div>
)}

{/* Top-right header — DOCUMENT actions (things you do to this board).
View controls live in the bottom-right dock and one-time settings
live in Settings, so no control appears in two places.
In a narrow pane (split view, or the sidebar open on an upright
tablet) the labels become icons and the gaps tighten, so the whole
bar fits and undo at its left end is never cut off. */}
<div className="absolute top-4 right-4 z-30 flex max-w-[calc(100%-2rem)] items-center gap-1.5 @max-[46rem]:gap-0.5 overflow-x-auto [scrollbar-width:none] [&::-webkit-scrollbar]:hidden p-1 bg-white/95 dark:bg-zinc-900/95 backdrop-blur-md shadow-md border border-border/80 rounded-xl select-none">
  {/* Undo / redo live here, with the other board actions: inside the
  tool pill they wrapped onto a second row of their own. */}
  <HeaderUndoRedo />
  <div className="h-4 w-px bg-border/80 mx-0.5" />
  {/* Pro Selection Tools */}
  <DropdownMenu>
    <DropdownMenuTrigger asChild>
      <Button
        variant={selectedIds.length > 0 ? "secondary" : "ghost"}
        size="sm"
        className={cn(
          "h-8 px-2 text-xs font-medium gap-1.5 transition-colors",
          selectedIds.length > 0
            ? "text-primary bg-primary/10 border border-primary/25"
            : "text-muted-foreground hover:text-foreground"
        )}
        title="Canvas selection tools (Select all, by type, invert)"
      >
      <CheckSquare size={14} className={selectedIds.length > 0 ? "text-primary" : ""} />
      <span className="hidden sm:inline @max-[46rem]:hidden">
        {selectedIds.length > 0 ? `Selected (${selectedIds.length})` : "Select"}
      </span>
      {selectedIds.length > 0 && (
        <span className="sm:hidden @max-[46rem]:inline text-[10px] font-bold">
          {selectedIds.length}
        </span>
      )}
    </Button>
  </DropdownMenuTrigger>
  <DropdownMenuContent align="end" className="w-56 text-xs">
    <div className="px-2 py-1 text-[10px] font-semibold uppercase tracking-wider text-muted-foreground">
      Canvas Selection
    </div>
    <DropdownMenuItem onClick={() => selectAll()}>
      <span className="flex-1">Select All</span>
      <span className="text-[10px] text-muted-foreground">Ctrl+A</span>
    </DropdownMenuItem>
    {selectedIds.length > 0 && (
      <DropdownMenuItem onClick={() => clearSelection()}>

```

```

        <span className="flex-1">Deselect All</span>
        <span className="text-[10px] text-muted-foreground">Esc</span>
      </DropdownMenuItem>
    )}
    <DropdownMenuItem onClick={() => invertSelection()}>
      <span className="flex-1">Invert Selection</span>
      <span className="text-[10px] text-muted-foreground">Ctrl+Shift+V</span>
    </DropdownMenuItem>
    <DropdownMenuSeparator />
    <div className="px-2 py-1 text-[10px] font-semibold uppercase tracking-wider text-muted-foreground">
      Select by Type
    </div>
    <DropdownMenuItem onClick={() => selectByType("stroke")}>
      <PenLine size={13} className="mr-2 text-primary" />
      <span className="flex-1">All Ink Strokes</span>
      <span className="text-[10px] text-muted-foreground">Ctrl+Shift+I</span>
    </DropdownMenuItem>
    <DropdownMenuItem onClick={() => selectByType("text")}>
      <Type size={13} className="mr-2 text-blue-500" />
      <span className="flex-1">All Text Notes</span>
      <span className="text-[10px] text-muted-foreground">Ctrl+Shift+T</span>
    </DropdownMenuItem>
    <DropdownMenuItem onClick={() => selectByType("sticky")}>
      <StickyNote size={13} className="mr-2 text-amber-500" />
      <span className="flex-1">All Sticky Notes</span>
    </DropdownMenuItem>
    <DropdownMenuItem onClick={() => selectByType("shape")}>
      <Shapes size={13} className="mr-2 text-indigo-500" />
      <span className="flex-1">All Shapes</span>
    </DropdownMenuItem>
    <DropdownMenuItem onClick={() => selectByType("image")}>
      <ImageIcon size={13} className="mr-2 text-emerald-500" />
      <span className="flex-1">All Images</span>
    </DropdownMenuItem>
    <DropdownMenuItem onClick={() => selectByType("connection")}>
      <Link2 size={13} className="mr-2 text-purple-500" />
      <span className="flex-1">All Connectors</span>
    </DropdownMenuItem>
  </DropdownMenuContent>
</DropdownMenu>

<Button
  variant="ghost"
  size="sm"
  className={cn(
    "h-8 px-2.5 text-xs font-medium",
    showBacklinks ? "text-primary" : "text-muted-foreground hover:text-foreground"
  )}
  onClick={() => setShowBacklinks(!showBacklinks)}
  title="Links & backlinks for this board (Ctrl+B)"
  aria-label="Links"
>
  <Link2 size={15} className="hidden @max-[46rem]:block" />
  <span className="@max-[46rem]:hidden">Links</span>
</Button>

<Button
  variant="ghost"
  size="icon"
  className="h-8 w-8 text-muted-foreground hover:text-foreground"
  // Everywhere, like Ctrl+K: it sits beside the graph, the other
  // whole-workspace tool, and is how you reach a note that is not
  // open. "This note" is one tap away in the panel, and Ctrl+F.
  onClick={() => openSearch("everywhere")}
  title="Search (Ctrl+K) · find in this note (Ctrl+F)"
  aria-label="Search"
>
  <Search size={15} />
</Button>

<Button
  variant="ghost"
  size="icon"
  className="h-8 w-8 text-muted-foreground hover:text-foreground"
  onClick={() => setGraphOpen(true)}
  title="Knowledge graph (Ctrl+G)"
>

```

```

    <Network size={15} />
  </Button>

  <SplitPicker />

  <div className="h-4 w-px bg-border/80 mx-0.5" />
    {isNotebookBoard && (
      <Button
        variant="ghost"
        size="icon"
        className="h-8 w-8 text-muted-foreground hover:text-foreground"
        onClick={() => setInsertPagesOpen(true)}
        title="Insert pages from another notebook or a PDF"
      >
        <FilePlus2 size={15} />
      </Button>
    )}

    <Button
      variant="ghost"
      size="icon"
      className="h-8 w-8 text-muted-foreground hover:text-foreground"
      onClick={() => setExportOpen(true)}
      title="Export or share (Ctrl+E)"
    >
      <Share2 size={15} />
    </Button>

    <Button
      variant="ghost"
      size="icon"
      className="h-8 w-8 text-muted-foreground hover:text-foreground"
      onClick={() => setSyncOpen(true)}
      title="Sync & Backup"
    >
      <Cloud size={16} />
    </Button>

    <Button
      variant="ghost"
      size="icon"
      className="h-8 w-8 text-muted-foreground hover:text-foreground"
      onClick={() => setSettingsOpen(true)}
      title="Settings"
    >
      <SettingsIcon size={16} />
    </Button>
  </div>

  {/* Canvas */}
  <WhiteboardCanvas />

  {/* Toolbar bottom dock */}
  <Toolbar
    onOpenGraph={() => setGraphOpen(true)}
    onOpenSearch={() => openSearch("everywhere")}
  />

  {/* Minimap Radar */}
  <Minimap viewport={viewport} />

  {/* Favourite pens (movable, reorderable pin bar) */}
  <FavoritePensBar />

  {/* Notebook page navigation – renders nothing on an infinite canvas */}
  <NotebookPagePanelHost />

  {/* Drop a PDF or images anywhere on the window */}
  <FileDropZone />

  {/* Floating actions and HUD */}
  <SelectionActions viewport={viewport} />
  <ImageActions viewport={viewport} />
  <CaptureHud />
  <LockedModeBar />
  <PropertiesPanel />
  <LinkLegend visible={showLegend} onToggle={toggleLegend} />

```

```

<InlineTextEditor viewport={viewport} />

{/* Modals */}
<ImageStudio />
<CapturePanel />
<ExportModal open={exportOpen} onClose={() => setExportOpen(false)} />
<InsertPagesModal open={insertPagesOpen} onClose={() => setInsertPagesOpen(false)} />
<KeyboardShortcutsModal open={shortcutsOpen} onClose={() => setShortcutsOpen(false)} />
<GraphView open={graphOpen} onClose={() => setGraphOpen(false)} />
{search && (
  <SearchModal key={search.key} initialScope={search.scope} initialQuery={search.query} initialMode={s
    earch.mode} onClose={() => setSearch(null)} />
)}
<LinkManager open={linkManagerOpen} onClose={() => setLinkManagerOpen(false)} />
<SyncPanel open={syncOpen} onClose={() => setSyncOpen(false)} />
<SettingsPanel
  open={settingsOpen}
  onClose={() => setSettingsOpen(false)}
  legendVisible={showLegend}
  onToggleLegend={toggleLegend}
/>
<TutorialModal />
</div>

{/* Reference pane. Modals stay inside the editable pane above so they
  are never squeezed by the split. */}
{splitBoardId && (
  <>
    <SplitDivider containerRef={splitRowRef} stacked={stacked} style={{ order: 1 }} />
    <div
      className="min-h-0 min-w-0"
      style={{
        order: splitSide === "left" ? 2 : 0,
        flex: `0 0 calc(${(splitSide === "left" ? 1 - splitRatio : splitRatio) * 100}% - 3px)`,
      }}
    >
      <SplitPane />
    </div>
  </>
)}
</div>
</div>
);
}

```

```

=====
// === END FILE: src/components/whiteboard/WhiteboardApp.tsx ===
=====

```

```

=====
// === START FILE: src/components/whiteboard/WhiteboardCanvas.tsx ===
// File 89 of 194 | Lines: 4614 | Size: 188043 bytes
=====

```

```
"use client";
```

```

/**
 * Whiteboard Infinite Canvas (v4.2 – Clean Touchscreen & Single-Dot Precision)
 * =====
 * Key Fixes:
 * - Single-tap dots (e.g. for letter dots, i/j, Hebrew niqqud, Arabic dots, periods)
 *   are preserved with exact radius and color (no more vanishing!).
 * - Removed annoying pinging ripple balls on tap.
 * - Multi-touch gestures: 2-finger pinch to zoom & 2-finger pan.
 * - 2-finger tap = Undo, 3-finger tap = Redo.
 * - Accurate container-relative coordinate tracking.
 */

```

```

import React, { useCallback, useEffect, useLayoutEffect, useRef, useState, useMemo } from "react";
import { LaserLayer, laser } from "../LaserPointer";
import { ShapeNoteChip, type ShapeNote } from "../ShapeAssist";
import { joinLine, lineBetween, recognizeShape, shapeLabel, type JoinTarget } from "@lib/whiteboard/shapeReco
  gnition";
import { usePdfTextSelect } from "../PdfTextSelect";
import { flushSync } from "react-dom";
import { nanoid } from "nanoid";
import { useWhiteboard, useActiveBoard } from "@store/whiteboard";

```

```
import {
  DEFAULT_STYLE,
  BOARD_PATTERN_MAX_SIZE,
  BOARD_PATTERN_MIN_SIZE,
  isStrokeTool,
  getToolColor,
  getToolSize,
  type ArrowElement,
  type ConnectionElement,
  type EllipseElement,
  type ImageElement,
  type LineElement,
  type RectangleElement,
  type DiamondElement,
  type TriangleElement,
  type StarElement,
  type CloudElement,
  type StrokeElement,
  type StickyElement,
  type TextElement,
  type WhiteboardElement,
  type ToolType,
  type ResizeHandle,
  type NodeAnchor,
  type Camera,
  PEN_TYPE_PRESETS,
  type StrokeStyle,
} from "@lib/whiteboard/types";
import {
  getElementBounds,
  getElementCenter,
  rotatePoint,
  getNodeAnchorPorts,
  getResolvedAnchorPoint,
  getConnectionPath,
  hitTestElement,
  HIT_THRESHOLD,
  isElementFullyInPolygon,
  isElementInPolygon,
  isPointInPolygon,
  screenToWorld,
  worldToScreen,
  type Viewport,
} from "@lib/whiteboard/geometry";
import {
  buildStrokePath,
  calibratePressure,
  computePressureFromSpeed,
  eraseFromStroke,
  settledStrokePoints,
  isStrokeStraight,
  pruneTerminalHooks,
  straightenStrokePoints,
  trimHoldTail,
} from "@lib/whiteboard/stroke";
import { ElementRenderer, StrokeFilterDefs } from "../ElementRenderer";
import { SelectionOverlay } from "../SelectionOverlay";
import { CanvasContextMenu } from "../CanvasContextMenu";
import { GRID_SIZE, snapToGrid, snapPointToGrid } from "@lib/whiteboard/grid";
import {
  backgroundForPage,
  clampBoxToSheet,
  clampNotebookCamera,
  minNotebookZoom,
  clampToSheet,
  getActivePage,
  getActivePageIndex,
  getBoardElements,
  getPages,
  isNotebook,
  isOnActiveSheet,
  paperForPage,
  sheetIndexAt,
  PAGE_GAP,
  pageSizeOf,
} from "@lib/whiteboard/notebook";
import { PaperTiles, paperColorFor } from "../PaperPattern";
import { PdfPageLayer } from "../PdfPageLayer";
```

```

import { placeImage, lockAspect, snapRotation, tapShapeRect, TAP_DRAG_SLOP_PX } from "@lib/whiteboard/placement";
import { effectiveSelectFilter, passesSelectFilter, penFeelFor, pickableNow } from "@lib/whiteboard/types";
import { deferSavesWhileWriting } from "@lib/whiteboard/persistence";
import {
  CAPTURE_MIN_SIDE,
  boundsOfPoints,
  clampCaptureRegion,
  polygonArea,
  rectFromCorners,
  type CaptureShape,
} from "@lib/whiteboard/capture";
import { screenshotShapeFor, useCapture } from "@store/capture";
import { appClipForPaste } from "@lib/whiteboard/appClipboard";
import { CaptureMarquee } from "../CaptureMarquee";

/** How far (screen px) a held pen may drift and still count as holding still. */
const HOLD_SLOP_PX = 6;
import { assetIdOf, isAssetRef, loadImage, storeImage } from "@lib/whiteboard/imagestore";
import { isUndoCombo, isRedoCombo } from "@lib/whiteboard/shortcuts";
import { stylusEraseTransition } from "@lib/whiteboard/input";
import {
  classifyTouch,
  contactLooksLikePalm,
  exceedsLongPressSlop,
  shouldSelectOnTouchTap,
  LONG_PRESS_MS,
  PEN_FAST_BYPASS_MS,
} from "@lib/whiteboard/palm";
import type { TouchVerdict } from "@lib/whiteboard/palm";
import {
  cameraPanForScroll,
  createWheelDeviceState,
  observeWheel,
  resolveWheelAction,
  wheelSourceOf,
} from "@lib/whiteboard/input";
import { useTheme } from "next-themes";

type InteractionMode =
  | { kind: "idle" }
  | { kind: "drawing"; elementId: string; tool: string }
  | { kind: "panning"; lastX: number; lastY: number }
  | { kind: "erasing" }
  | { kind: "selecting"; startX: number; startY: number; worldX: number; worldY: number }
  | { kind: "lasso-selecting"; points: Array<{ x: number; y: number }>; screenPoints: Array<{ x: number; y: number }>; strict: boolean }
  /** Drawing the area to capture: a box (first and last point) or a loop.
   * Tied to ONE pointer, so a palm lifting mid-drag cannot end it. */
  | {
    kind: "capturing";
    shape: CaptureShape;
    pointerId: number;
    /** Begun by a finger: a pen arriving takes it over (a palm got there first). */
    byTouch?: boolean;
    points: Array<{ x: number; y: number }>;
  }
  | { kind: "moving"; startWorldX: number; startWorldY: number; originalElements: WhiteboardElement[] }
  | { kind: "resizing"; handle: ResizeHandle; originalElement: WhiteboardElement; startWorldX: number; startWorldY: number }
  | { kind: "rotating"; originalElement: WhiteboardElement; startAngle: number; centerX: number; centerY: number }
  | { kind: "creating-shape"; elementId: string; startX: number; startY: number }
  | {
    kind: "connecting";
    sourceElementId: string;
    sourceAnchor: NodeAnchor;
    currentWorldX: number;
    currentWorldY: number;
    targetElementId?: string;
    targetAnchor?: NodeAnchor;
  };

/**
 * How far outside a notebook sheet a press still counts as aimed at the page.
 *
 * World units, so it shrinks on screen as you zoom out — which is correct: at

```

```

* low zoom the whole page is small and the surrounding desk is large, and you
* want the desk to pan, not to absorb strokes.
*/
const SHEET_GRAB_MARGIN = 24;

// -----
// Fling (momentum) – platform values, not invented ones
// -----
// iOS UIScrollView: velocity decays by DecelerationRate.normal = 0.998 per
// millisecond, so a throw travels about  $v_0 * 0.998 / (1 - 0.998) = 499 * v_0$ .
// Android ViewConfiguration: a fling starts at 50 dp/s and is capped at
// 8000 dp/s. Android's VelocityTracker only looks at the last 100 ms of
// movement and treats a finger that has not moved for 40 ms as STOPPED.
//
// That last rule is what the first attempt here got wrong. It smoothed the
// velocity across the whole drag, so a drag you had already brought to a stop
// still launched when the finger lifted, and every ordinary scroll overshot.
// With the 40 ms rule, stop-then-lift scrolls exactly where you put it, and
// only a real throw coasts. (A WebView CSS pixel is one dp.)
const FLING_DECEL_PER_MS = 0.998;
const FLING_MIN_V = 0.05; // px/ms = 50 dp/s
const FLING_MAX_V = 8; // px/ms = 8000 dp/s
const FLING_STOP_V = 0.02; // px/ms: settled
const VELOCITY_HORIZON_MS = 100;
const POINTER_STOPPED_MS = 40;

/**
 * Where the camera has to be so the paper keeps the screen.
 * Shared by the settle-guard and the fling, so a throw stops at exactly the
 * edge the guard would enforce instead of fighting it frame by frame.
 *
 * The rule lives in notebook.ts (`clampNotebookCamera`): at most
 * `DESK_MARGIN_PX` of bare desk past the paper on any side. The earlier rule
 * here asked only that 120 px of the sheet stay somewhere on screen, which let
 * the page be shoved against one edge with most of a screenful of grey beside
 * it - a whole empty half-screen on a tablet in portrait.
 */
function clampCameraToSheet(
  cam: { x: number; y: number; zoom: number },
  vp: { width: number; height: number },
  page: { width: number; height: number },
  doc?: { pageCount: number; activeIndex: number }
): { x: number; y: number } {
  return clampNotebookCamera(cam, vp, page, doc);
}

interface LiveDraftStroke {
  id: string;
  originX: number;
  originY: number;
  color: string;
  strokeWidth: number;
  strokeStyle: StrokeStyle;
  smoothing: number;
  createdAt: number;
  points: Array<number, number, number>;
}

export function WhiteboardCanvas() {
  const svgRef = useRef<SVGSVGElement>(null);
  const containerRef = useRef<HTMLDivElement>(null);

  const [viewport, setViewport] = useState<Viewport>({ width: 1200, height: 800 });
  const [interaction, setInteraction] = useState<InteractionMode>({ kind: "idle" });
  const [draftElement, setDraftElement] = useState<WhiteboardElement | null>(null);
  const [rubberBand, setRubberBand] = useState<{ x: number; y: number; w: number; h: number } | null>(null);
  const [lassoPoints, setLassoPoints] = useState<Array<{ x: number; y: number }>>>([]);
  const [spaceHeld, setSpaceHeld] = useState(false);

  // --- Direct DOM Live Inking Overlay Refs (R1-DEF-01, R1-DEF-02, R1-DEF-04, D5) ---
  const liveDraftStrokeRef = useRef<LiveDraftStroke | null>(null);
  const liveDraftGroupRef = useRef<SVGGElement | null>(null);
  const liveDraftPathRef = useRef<SVGPathElement | null>(null);
  const draftStrokeRafRef = useRef<number | null>(null);
  const drawingPointerIdRef = useRef<number | null>(null);
  const autoStraightenAnchorRef = useRef<{ x: number; y: number } | null>(null);
  const predictedPointsRef = useRef<Array<number, number, number>>>([]);
  /** Last real (non-zero) pen pressure of the stroke in progress. */

```

```

const lastPenPressureRef = useRef(0.5);
const rejectedPalmsRef = useRef<Set<number>>(new Set());

// Multi-touch tracking (finger pointers only – mouse/pen are never "fingers")
const activeTouchesRef = useRef<Map<number, { x: number; y: number; startX: number; startY: number }>>(new Map());
const pinchStartDistanceRef = useRef<number | null>(null);
/** True once a gesture has been judged a pinch rather than a scroll. */
const pinchEngagedRef = useRef(false);
const pinchStartZoomRef = useRef<number>(1);
const lastTouchMidRef = useRef<{ x: number; y: number } | null>(null);
const touchStartTimeRef = useRef<number>(0);
const maxTouchCountRef = useRef<number>(0);
const touchMovedRef = useRef<boolean>(false);
const lastTapRef = useRef<{ time: number; x: number; y: number }>({ time: 0, x: 0, y: 0 });
const lastPointerPosRef = useRef<{ clientX: number; clientY: number } | null>(null);
const penActiveRef = useRef<boolean>(false);
/**
 * When the stylus was last in contact. Palm suppression outlives the pen –
 * the hand rests on the glass BEFORE the nib lands and stays there while you
 * lift between words, so a during-contact-only guard never sees it.
 */
const lastPenAtRef = useRef<number>(0);
/**
 * The selection a touch-started transform replaced, so that a stylus landing
 * a moment later can retract the whole thing: a hand that turns out to have
 * been a palm should leave no trace, not merely stop.
 */
const touchInteractionRef = useRef<{ pointerId: number; priorSelection: string[] } | null>(null);
/** Armed long-press-to-select (finger only). */
const longPressRef = useRef<{
  timer: ReturnType<typeof setTimeout>;
  pointerId: number;
  elementId: string;
  startX: number;
  startY: number;
} | null>(null);
/**
 * Set when a long press has selected an object: the NEXT move event promotes
 * the pan into a drag, using that event's own world point as the origin so
 * the object cannot jump by however far the finger wobbled while holding.
 */
const pendingMoveRef = useRef<{ pointerId: number; elementId: string } | null>(null);
/**
 * Whether the canvas context menu may open. Radix opens it on a 700ms long
 * press for every pointer that is not a mouse, and its trigger sits on this
 * same <svg> – so a resting palm, or a stylus paused mid-thought, opened the
 * "Bring Forward / Send Backward / Delete" menu on its own. Disarming is
 * state rather than a ref because Radix reads the prop, and its own effect
 * cancels the pending timer the moment the prop flips.
 */
const [contextMenuArmed, setContextMenuArmed] = useState(true);
/** The coasting fling's frame, if one is running. */
const momentumRafRef = useRef<number | null>(null);
const contextMenuArmedRef = useRef(true);
/** Tool to hand back when the stylus button is released. */
const stylusEraseRestoreRef = useRef<ToolType | null>(null);
// Previous eraser position, so a drag erases the swept path, not just points.
const lastErasePosRef = useRef<{ x: number; y: number } | null>(null);
// Read by the wheel listener, which is attached once.
const cameraRef = useRef<Camera>({ x: 0, y: 0, zoom: 1 });
const viewportRef = useRef<Viewport>({ width: 1200, height: 800 });
// True when a gesture was sealed (committed) during the current touch
// sequence – suppresses the 2/3-finger tap undo/redo for that sequence.
const gestureSealedRef = useRef<boolean>(false);

// Store
const camera = useWhiteboard((s) => s.camera);
const tool = useWhiteboard((s) => s.tool);
const style = useWhiteboard((s) => s.style);
const selectedIds = useWhiteboard((s) => s.selectedIds);
// Membership test for the render loop. `selectedIds.includes` inside
// `elements.map` is O(elements x selection) on EVERY render, and renders
// happen on every pointer move while dragging a big selection.
const selectedSet = useMemo(() => new Set(selectedIds), [selectedIds]);
const editingId = useWhiteboard((s) => s.editingId);
const touchSettings = useWhiteboard((s) => s.touchSettings);
const snapToGridActive = useWhiteboard((s) => s.snapToGrid);

```

```

const toggleSnapToGrid = useWhiteboard((s) => s.toggleSnapToGrid);
const toggleMinimap = useWhiteboard((s) => s.toggleMinimap);
const panCamera = useWhiteboard((s) => s.panCamera);
const zoomTo = useWhiteboard((s) => s.zoomTo);
const zoomToFit = useWhiteboard((s) => s.zoomToFit);
const resetCamera = useWhiteboard((s) => s.resetCamera);
const setCamera = useWhiteboard((s) => s.setCamera);
const selectAll = useWhiteboard((s) => s.selectAll);
const selectFilterSaved = useWhiteboard((s) => s.touchSettings.selectFilter);
const selectFilter = useMemo(
  () => effectiveSelectFilter(selectFilterSaved),
  [selectFilterSaved]
);
const duplicateElements = useWhiteboard((s) => s.duplicateElements);
const toggleLock = useWhiteboard((s) => s.toggleLock);
const lockedMode = useWhiteboard((s) => s.lockedMode);
const setImageStudioId = useWhiteboard((s) => s.setImageStudioId);
const bringForward = useWhiteboard((s) => s.bringForward);
const sendBackward = useWhiteboard((s) => s.sendBackward);
const setTool = useWhiteboard((s) => s.setTool);
const lastStrokeTool = useWhiteboard((s) => s.lastStrokeTool);
const setSelected = useWhiteboard((s) => s.setSelected);
const addToSelection = useWhiteboard((s) => s.addToSelection);
const clearSelection = useWhiteboard((s) => s.clearSelection);
const setEditingId = useWhiteboard((s) => s.setEditingId);
const addElement = useWhiteboard((s) => s.addElement);
const addElements = useWhiteboard((s) => s.addElements);
const updateElement = useWhiteboard((s) => s.updateElement);
const deleteElements = useWhiteboard((s) => s.deleteElements);
const commitHistory = useWhiteboard((s) => s.commitHistory);
const createConnection = useWhiteboard((s) => s.createConnection);
const undo = useWhiteboard((s) => s.undo);
const redo = useWhiteboard((s) => s.redo);
const stepPage = useWhiteboard((s) => s.stepPage);
const wheelDeviceRef = useRef(createWheelDeviceState());
const activeBoardId = useWhiteboard((s) => s.activeBoardId);
// Screenshot mode (Select menu): the select tool's box or loop takes a
// picture of the area instead of selecting. Null whenever a press should do
// what it always does - the mode is off, or the tool is not Select/Lasso.
const screenshotMode = useCapture((s) => s.screenshotMode);
const captureArmed = screenshotShapeFor(screenshotMode, tool);

cameraRef.current = camera;
viewportRef.current = viewport;

const activeBoard = useActiveBoard();
const elements = getBoardElements(activeBoard);

// Notebook page geometry. The page lives in WORLD space from (0,0) to
// (w,h), so all the existing tools, hit-testing and camera maths keep
// working unchanged - a page is just a marked-out region of the canvas.
const isNb = isNotebook(activeBoard);
const activePage = getActivePage(activeBoard);
const pageDim = pageSizeOf(activeBoard);
const pageStride = pageDim.height + PAGE_GAP;
const allPages = getPages(activeBoard);
const activeIndex = getActivePageIndex(activeBoard);

// The pages to draw around the active one. The active page sits at world
// Y = 0 and its neighbours are offset by whole strides, so the notebook
// reads as one continuous scroll while the coordinate system stays anchored.
// Only what the viewport can reach is built - a 200-page notebook must not
// render 200 sheets of ink on every frame.
const neighbourPages = React.useMemo(() => {
  if (!isNb || allPages.length === 0) return [];
  const halfWorld = viewport.height / 2 / Math.max(camera.zoom, 0.01);
  const topY = camera.y - halfWorld;
  const bottomY = camera.y + halfWorld;
  const out: Array<{ id: string; index: number; offsetY: number }> = [];
  for (let i = 0; i < allPages.length; i++) {
    if (i === activeIndex) continue;
    const offsetY = (i - activeIndex) * pageStride;
    // Skip pages whose sheet is entirely outside the visible band.
    if (offsetY + pageDim.height < topY - pageStride) continue;
    if (offsetY > bottomY + pageStride) continue;
    out.push({ id: allPages[i].id, index: i, offsetY });
  }
  return out;
});

```

```

}, [isNb, allPages, activeIndex, pageStride, pageDim.height, camera.y, camera.zoom, viewport.height]);

// Board paper. An explicit colour wins; otherwise the theme's own background
// shows through (no rect painted), so the board still follows light/dark.
const { resolvedTheme } = useTheme();
// A notebook page carries its own paper, falling back to the notebook's -
// which is how one document can mix templates, the thing Notability is
// criticised for not allowing.
const paperSource = isNb
  ? backgroundForPage(activeBoard!, activePage) ?? activeBoard?.background
  : activeBoard?.background;
const paperPattern = paperSource?.pattern ?? "dots";
// Paper spacing. Snapping uses the SAME value, so the grid you snap to is
// always the grid you can see - snapping to an invisible 24px lattice while
// looking at 64px paper is disorienting.
const paperSize = Math.min(
  BOARD_PATTERN_MAX_SIZE,
  Math.max(BOARD_PATTERN_MIN_SIZE, paperSource?.size ?? GRID_SIZE)
);
const paperColor = paperColorFor(paperSource, resolvedTheme);

// Continuous scrolling. Once the viewport centre crosses the middle of the
// gap between two sheets, the next page takes over as the anchored page and
// the camera slides by exactly one stride to compensate - the two cancel, so
// scrolling feels like one long strip while the coordinate system, and every
// piece of code that depends on it, stays put.
const reanchorPage = useWhiteboard((s) => s.reanchorPage);
/**
 * Set when the PEN picked the live page (see the pointer-down handler).
 * While the camera has not moved since, the scroll rule below must not hand
 * the page back to whichever sheet is under the centre of the screen -
 * otherwise it switched back the moment you lifted the pen, clearing the
 * selection you had just made on the page you were working on.
 */
const anchorLockRef = useRef<{ camY: number } | null>(null);
useEffect(() => {
  if (!isNb || allPages.length <= 1) return;
  // Not mid-gesture: re-anchoring under an in-progress stroke would move the
  // world out from under the points already captured.
  if (interaction.kind !== "idle") return;
  if (anchorLockRef.current) {
    if (Math.abs(camera.y - anchorLockRef.current.camY) < 0.5) return;
    anchorLockRef.current = null; // you scrolled: centre-of-screen rules again
  }

  const forward = pageDim.height + PAGE_GAP / 2;
  const backward = -PAGE_GAP / 2;
  if (camera.y > forward && activeIndex < allPages.length - 1) {
    reanchorPage(1, pageStride);
  } else if (camera.y < backward && activeIndex > 0) {
    reanchorPage(-1, pageStride);
  }
}, [camera.y, isNb, allPages.length, activeIndex, pageDim.height, pageStride, interaction.kind, reanchorPage
]);

// -----
// The active sheet may never leave the screen
// -----
// A bound expressed as a fraction of PAGE height cannot do this job: what
// counts as "off screen" depends on the viewport and the zoom, not on the
// paper. Measured on a Galaxy Tab S9 the camera sat at world y = -1250 with
// the sheet's top edge at screen y 1420 in a 1126-tall viewport - the page
// was completely below the screen, on an empty grey desk, and because only
// the anchored page is drawn there was nothing else to see or aim at.
//
// So the rule is stated in the terms that actually matter: after the camera
// settles, at least `MIN_VISIBLE` of the active sheet must intersect the
// viewport. If it does not, the camera is pulled back to the nearest
// position where it does. This also repairs a camera restored from storage
// that was already out of range.
useEffect(() => {
  if (!isNb) return;
  if (interaction.kind !== "idle") return; // never mid-gesture
  if (liveCamRef.current) return; // nor mid-pan

  if (momentumRafRef.current !== null) return; // the fling stops itself

  // Zoom first: a camera restored from storage (or a window that just got

```

```

// bigger) can sit below the floor, and no amount of panning fixes a page
// that is simply too small for the screen it is on.
const floor = minNotebookZoom(viewport, pageDim);
if (floor > 0 && camera.zoom < floor - 1e-3) {
  zoomTo(floor, viewport.width / 2, viewport.height / 2, viewport);
  return; // the position is settled on the next pass, at the new zoom
}

// Keep the paper on screen, with only a small strip of desk around it.
const next = clampCameraToSheet(camera, viewport, pageDim, {
  pageCount: allPages.length,
  activeIndex,
});
// Half a world unit of drift is invisible; correcting it anyway would be a
// set -> re-render -> set loop if the store's own bound ever disagreed.
if (Math.abs(next.y - camera.y) > 0.5 || Math.abs(next.x - camera.x) > 0.5) {
  setCamera(next);
}
}, [
  isNb,
  interaction.kind,
  camera.x,
  camera.y,
  camera.zoom,
  viewport.width,
  viewport.height,
  pageDim.width,
  pageDim.height,
  allPages.length,
  activeIndex,
  setCamera,
  zoomTo,
]);

// -----
// "+" in the gap between two sheets (and above the first, below the last)
// -----
// Hover the pen over the blank strip between two pages - or tap it - and a
// "+" appears there; tapping it inserts a page in exactly that place, with
// nothing above it moving (see insertPageAt in the store). GoodNotes'
// equivalent lives in its thumbnail view; here it is where you are writing.
//
// Gap k sits just above page k in the anchored frame, so its centre is
// (k - activeIndex) * stride - GAP/2 for every k from 0 (above the first
// page) to allPages.length (below the last) - one formula for all three.
const insertPageAt = useWhiteboard((s) => s.insertPageAt);
const [gapPlus, setGapPlus] = useState<number | null>(null);
const gapPlusRef = useRef<number | null>(null);
const gapPlusTimerRef = useRef<ReturnType<typeof setTimeout> | null>(null);

const gapAt = useCallback(
  (wx: number, wy: number, extraPx = 0): number | null => {
    if (!isNb || allPages.length === 0) return null;
    const k = Math.round((wy + PAGE_GAP / 2) / pageStride) + activeIndex;
    if (k < 0 || k > allPages.length) return null;
    const cy = (k - activeIndex) * pageStride - PAGE_GAP / 2;
    // The gap ONLY - not the paper either side of it. A wider band showed
    // the "+" while the pen was over the edge of a page, and a press there
    // then added a page instead of writing. Measured at 44% zoom, the old
    // band reached 11 px into each sheet. `extraPx` widens it for a finger
    // TAP, which has no hover and is much less precise than a nib.
    // Half the seam plus a few SCREEN pixels. The seam is a hairline now, so
    // a band of `PAGE_GAP / 2` alone would be a couple of pixels and the "+"
    // would be unhittable; a band in world units would grow into the paper
    // as you zoom out, which is the bug the note above describes.
    const slack = PAGE_GAP / 2 + (6 + extraPx) / camera.zoom;
    if (Math.abs(wy - cy) > slack) return null;
    const side = 40 / camera.zoom;
    if (wx < -side || wx > pageDim.width + side) return null;
    return k;
  },
  [isNb, allPages.length, pageStride, activeIndex, camera.zoom, pageDim.width]
);

const isTouchPress = (e: { pointerType: string }) => e.pointerType === "touch";

/** Show (k) or hide (null) the gap button, touching state only on change. */
const showGapPlus = useCallback((k: number | null, holdMs = 0) => {

```

```

    if (gapPlusTimerRef.current) {
      clearTimeout(gapPlusTimerRef.current);
      gapPlusTimerRef.current = null;
    }
    if (k !== gapPlusRef.current) {
      gapPlusRef.current = k;
      setGapPlus(k);
    }
    // A finger has no hover, so a tap pins the button long enough to reach it.
    if (k !== null && holdMs > 0) {
      gapPlusTimerRef.current = setTimeout(() => {
        gapPlusTimerRef.current = null;
        gapPlusRef.current = null;
        setGapPlus(null);
      }, holdMs);
    }
  }, []);

useEffect(
  () => () => {
    if (gapPlusTimerRef.current) clearTimeout(gapPlusTimerRef.current);
  },
  []
);

// Track exact viewport size with ResizeObserver
useEffect(() => {
  if (!containerRef.current) return;
  /**
   * ONE place that records the canvas size, because there are two readers
   * and they must never disagree: this component (the camera bound) and the
   * store (the notebook zoom floor).
   *
   * The observer used to update only the local state, so the store kept
   * whatever size the canvas had when the app started. Hide the sidebar -
   * which the canvas resizes for, and the window does not - and the zoom
   * floor was still computed for the narrower canvas, letting a notebook
   * zoom out further than it should and leaving exactly the grey desk this
   * was meant to remove.
   */
  const apply = (width: number, height: number) => {
    if (!(width > 0) || !(height > 0)) return;
    setViewport({ width, height });
    useWhiteboard.getState().setCanvasViewport({ width, height });
  };
  const updateSize = () => {
    if (containerRef.current) {
      const rect = containerRef.current.getBoundingClientRect();
      apply(rect.width, rect.height);
    }
  };
  updateSize();

  const observer = new ResizeObserver((entries) => {
    for (const entry of entries) {
      const { width, height } = entry.contentRect;
      apply(width, height);
    }
  });

  observer.observe(containerRef.current);
  window.addEventListener("resize", updateSize);
  return () => {
    observer.disconnect();
    window.removeEventListener("resize", updateSize);
  };
}, []);

// Keyboard shortcuts
useEffect(() => {
  const onKeyDown = (e: KeyboardEvent) => {
    if (editingId) return;

    // Ignore shortcuts if focused on input-ish elements – including BUTTON:
    // Space must keep activating a focused toolbar button, not arm pan mode.
    const activeEl = e.target as HTMLElement | null;
    const interactive =
      !!activeEl &&

```

```

    (activeEl.tagName === "INPUT" ||
    activeEl.tagName === "TEXTAREA" ||
    activeEl.tagName === "BUTTON" ||
    activeEl.tagName === "SELECT" ||
    activeEl.isContentEditable ||
    activeEl.closest?(['role="button"', [role="slider"]']));
if (interactive) {
  // Only Space matters here (button activation is native); everything
  // else falls through so Ctrl+Z etc. still work while a button has focus.
  if (e.code === "Space") return;
}
if (activeEl && (activeEl.tagName === "INPUT" || activeEl.tagName === "TEXTAREA" || activeEl.isContentEd
  itable)) {
  return;
}

// Never run canvas shortcuts while any modal overlay is open – Radix
// dialogs focus a plain DIV, and GraphView/LinkManager/etc. are custom
// overlays tagged with [data-wb-modal].
if (
  typeof document !== "undefined" &&
  document.querySelector(['role="dialog"'][data-state="open"], [data-wb-modal]')
) {
  return;
}

if (e.code === "Space" && !spaceHeld) {
  e.preventDefault();
  setSpaceHeld(true);
}

if (e.key === "Delete" || e.key === "Backspace") {
  if (selectedIds.length > 0) {
    e.preventDefault();
    deleteElements(selectedIds, true);
  }
}

// Undo/redo must not fire mid-gesture: the running move/resize loop
// would re-apply itself over the undone snapshot and desync history.
//
// Match on e.code, not e.key: e.key carries the SHIFTED/caps character,
// so Ctrl+Shift+Z arrives as "Z" and never matched `e.key === "z"` –
// the documented redo combo was dead, and Ctrl+Z itself stopped undoing
// with Caps Lock on. e.code is the physical key and is layout-stable.
if (isUndoCombo(e)) {
  e.preventDefault();
  if (interaction.kind === "idle") undo();
  return;
}
if (isRedoCombo(e)) {
  e.preventDefault();
  if (interaction.kind === "idle") redo();
  return;
}

// Page turns in a notebook. PageUp/PageDown are what every reader and
// note app uses; the bracket pair gives a one-hand alternative on
// keyboards where the tablet's cover has no PageUp.
if (isNb && !e.ctrlKey && !e.metaKey && !e.altKey) {
  if (e.code === "PageUp" || (e.shiftKey && e.code === "BracketLeft")) {
    e.preventDefault();
    if (interaction.kind === "idle") stepPage(-1);
    return;
  }
  if (e.code === "PageDown" || (e.shiftKey && e.code === "BracketRight")) {
    e.preventDefault();
    if (interaction.kind === "idle") stepPage(1);
    return;
  }
}

// Ctrl/Cmd + A -> Select all
if ((e.ctrlKey || e.metaKey) && e.code === "KeyA") {
  e.preventDefault();
  selectAll();
  return;
}

```

```
// Ctrl/Cmd + D -> Duplicate selection (must preventDefault or the
// browser opens its bookmark dialog)
if ((e.ctrlKey || e.metaKey) && e.code === "KeyD" && !e.shiftKey) {
  e.preventDefault();
  if (selectedIds.length > 0) duplicateElements(selectedIds);
  return;
}

// Ctrl/Cmd + Shift + L -> Lock / unlock selection
if ((e.ctrlKey || e.metaKey) && e.shiftKey && e.code === "KeyL") {
  e.preventDefault();
  if (selectedIds.length > 0) toggleLock(selectedIds);
  return;
}

// Layering: Ctrl+] / Ctrl+[ (also Ctrl+Shift+] style variants)
if ((e.ctrlKey || e.metaKey) && (e.key === "]" || e.key === "[")) {
  e.preventDefault();
  if (selectedIds.length > 0) {
    if (e.key === "]") bringForward(selectedIds);
    else sendBackward(selectedIds);
  }
  return;
}

// Zoom: Ctrl+0 reset, Ctrl+= in, Ctrl+- out, Shift+1 fit
if ((e.ctrlKey || e.metaKey) && e.code === "Digit0") {
  e.preventDefault();
  resetCamera();
  return;
}
if ((e.ctrlKey || e.metaKey) && (e.code === "Equal" || e.key === "+")) {
  e.preventDefault();
  zoomTo(camera.zoom * 1.2, viewport.width / 2, viewport.height / 2, viewport);
  return;
}
if ((e.ctrlKey || e.metaKey) && e.code === "Minus") {
  e.preventDefault();
  zoomTo(camera.zoom / 1.2, viewport.width / 2, viewport.height / 2, viewport);
  return;
}
if (e.shiftKey && e.code === "Digit1") {
  e.preventDefault();
  zoomToFit(viewport);
  return;
}

// Ctrl/Cmd + Shift + G -> Snap to grid toggle
if ((e.ctrlKey || e.metaKey) && e.shiftKey && (e.code === "KeyG" || e.key.toLowerCase() === "g")) {
  e.preventDefault();
  toggleSnapToGrid();
}

// Ctrl/Cmd + E -> Export Board
if ((e.ctrlKey || e.metaKey) && e.code === "KeyE") {
  e.preventDefault();
  window.dispatchEvent(new CustomEvent("wb:open-export"));
}

// Ctrl/Cmd + / -> Keyboard Shortcuts
if ((e.ctrlKey || e.metaKey) && (e.code === "Slash" || e.key === "/")) {
  e.preventDefault();
  window.dispatchEvent(new CustomEvent("wb:open-shortcuts"));
}

// Ctrl/Cmd + Shift + D -> Theme Toggle
if ((e.ctrlKey || e.metaKey) && e.shiftKey && (e.code === "KeyD" || e.key.toLowerCase() === "d")) {
  e.preventDefault();
  window.dispatchEvent(new CustomEvent("wb:toggle-theme"));
}

if (!e.ctrlKey && !e.metaKey && !e.altKey) {
  if (e.key === "?") {
    e.preventDefault();
    window.dispatchEvent(new CustomEvent("wb:open-shortcuts"));
    return;
  }
}
```

```

switch (e.code) {
  case "KeyV": setTool("select"); break;
  case "KeyQ": setTool("lasso"); break;
  case "KeyP": setTool(lastStrokeTool ?? "gel"); break;
  case "KeyH": setTool("pan"); break;
  case "KeyK": setTool("laser"); break;
  case "KeyI": setTool("textselect"); break;
  case "KeyY": setTool("highlighter"); break;
  case "KeyE": setTool("eraser"); break;
  case "KeyR": setTool("rectangle"); break;
  case "KeyO": setTool("ellipse"); break;
  case "KeyD": setTool("diamond"); break;
  case "KeyL": setTool("line"); break;
  case "KeyA": setTool("arrow"); break;
  case "KeyT": setTool("text"); break;
  case "KeyN": setTool("sticky"); break;
  case "KeyC": setTool("connect"); break;
  case "KeyM":
    e.preventDefault();
    toggleMinimap();
    break;
  case "KeyS":
    e.preventDefault();
    toggleSnapToGrid();
    break;
  case "Escape":
    // Cancel any active interaction first (draft discarded, overlays
    // cleared), then deselect / exit editor / return to select tool.
    setInteraction({ kind: "idle" });
    setDraftElement(null);
    setRubberBand(null);
    clearSelection();
    setEditingId(null);
    setTool("select");
    break;
}
};

const onKeyUp = (e: KeyboardEvent) => {
  if (e.code === "Space") {
    setSpaceHeld(false);
  }
};

// If the window loses focus mid-shortcut (alt-tab while Space held),
// keyup never fires – clear the held state so the canvas isn't stuck panning.
const onBlur = () => setSpaceHeld(false);

window.addEventListener("keydown", onKeyDown);
window.addEventListener("keyup", onKeyUp);
window.addEventListener("blur", onBlur);
return () => {
  window.removeEventListener("keydown", onKeyDown);
  window.removeEventListener("keyup", onKeyUp);
  window.removeEventListener("blur", onBlur);
};
}, [
  editingId,
  spaceHeld,
  selectedIds,
  interaction,
  camera.zoom,
  viewport,
  deleteElements,
  undo,
  redo,
  stepPage,
  isNb,
  selectAll,
  duplicateElements,
  toggleLock,
  bringForward,
  sendBackward,
  zoomTo,
  zoomToFit,
  resetCamera,

```

```

    setTool,
    clearSelection,
    setEditingId,
    toggleSnapToGrid,
    toggleMinimap,
  ]);

// Precision Coordinate Calculation (Subtracts Container Left/Top)
const getContainerPoint = useCallback(
  (clientX: number, clientY: number) => {
    const rect = containerRef.current?.getBoundingClientRect();
    if (!rect) {
      return { x: clientX, y: clientY, width: viewport.width, height: viewport.height };
    }
    return {
      x: clientX - rect.left,
      y: clientY - rect.top,
      width: rect.width,
      height: rect.height,
    };
  },
  [viewport]
);

const getWorldPoint = useCallback(
  (clientX: number, clientY: number) => {
    const pt = getContainerPoint(clientX, clientY);
    return screenToWorld(pt.x, pt.y, camera, { width: pt.width, height: pt.height });
  },
  [getContainerPoint, camera]
);

// The Select-text tool: hit-tests the page's text from these handlers, so
// no layer ever sits over the page (see PdfTextSelect.tsx).
const textSelect = usePdfTextSelect({
  tool,
  board: activeBoard,
  isNb,
  activePage,
  activeIndex,
  pageDim,
  camera,
  viewport,
  getWorldPoint,
  addElements,
  setCamera,
});

// -----
// Ink stays on the paper (notebooks only)
// -----
// A whiteboard is deliberately unbounded. A notebook is a stack of sheets,
// and writing beside a sheet or in the dead gap between two of them produced
// ink that was saved but never printed, never exported and impossible to
// find again once you scrolled. So in a notebook every point that becomes
// CONTENT is clamped to the active sheet, while `getWorldPoint` stays honest
// for panning, hit-testing and the marquee – those must still work off-page.

/** A world point clamped onto the sheet, for anything that creates content. */
const getInkPoint = useCallback(
  (clientX: number, clientY: number) => {
    const wp = getWorldPoint(clientX, clientY);
    if (!isNb) return wp;
    return clampToSheet(wp.x, wp.y, pageDim.width, pageDim.height);
  },
  [getWorldPoint, isNb, pageDim.width, pageDim.height]
);

/**
 * May a content gesture START here?
 *
 * Beginning a stroke off-page and clamping it would draw a phantom line
 * pinned to the margin that the user never asked for, so an off-page press
 * pans instead – the same thing GoodNotes does. The margin is generous
 * because a press a few units outside the border is plainly aimed at the
 * page, and snapping that back is helpful rather than surprising.
 */
const canStartInkAt = useCallback(

```

```

(wx: number, wy: number) =>
  !isNb || isOnActiveSheet(wx, wy, pageDim.width, pageDim.height, SHEET_GRAB_MARGIN),
  [isNb, pageDim.width, pageDim.height]
);

// -----
// Inserting an image – ONE way in, for the toolbar button and paste alike
// -----
// The toolbar used to drop an image at camera.x/y, up to 500 units wide,
// with no idea a page existed, and left it UNselected with the pen still
// active: it landed "in a random place", and touching it drew on it instead
// of moving it. Now it goes in the middle of what you can see, sized to fit,
// fully on the paper (lib/whiteboard/placement.ts), comes in SELECTED with
// the Select tool active so it can be dragged, resized and rotated straight
// away, and its pixels go to IndexedDB rather than into the saved notes.
const insertImage = (
  dataUrl: string,
  at: { x: number; y: number } | null,
  opts: {
    /** Several pictures at once fan out instead of landing on each other. */
    stagger?: number;
    /** A picture already in the store: reuse it rather than store it twice. */
    keepSrc?: string;
    /** Open the image editor straight away (a fresh camera photo). */
    edit?: boolean;
    /**
     * The size it should have on the page, in world units - a capture is
     * rendered at 2-3x its size, and placing it by its pixels would paste
     * it back two or three times larger than what was captured.
     */
    size?: { width: number; height: number };
  } = {}
) => {
  // A paste aimed at the sheet above or below the live one goes onto that
  // sheet, not squeezed onto the edge of the live page.
  if (at && isNb && allPages.length > 1) {
    const target = sheetIndexAt(at.x, at.y, activeIndex, allPages.length, pageDim.width, pageDim.height);
    if (target !== null && target !== activeIndex) {
      const delta = target - activeIndex;
      anchorLockRef.current = { camY: cameraRef.current.y - delta * pageStride };
      reanchorPage(delta, pageStride);
      at = { x: at.x, y: at.y - delta * pageStride };
    }
  }
  const img = new Image();
  img.onload = () => {
    const rect = placeImage({
      naturalWidth: opts.size?.width || img.naturalWidth || 400,
      naturalHeight: opts.size?.height || img.naturalHeight || 300,
      camera: cameraRef.current,
      viewport: viewportRef.current,
      page: isNb ? pageDim : null,
      at,
    });
    const imageId = nanoid(8);
    const fan = (opts.stagger ?? 0) * (28 / (cameraRef.current.zoom || 1));
    const newImage: ImageElement = {
      id: imageId,
      type: "image",
      x: rect.x + fan,
      y: rect.y + fan,
      width: rect.width,
      height: rect.height,
      naturalWidth: img.naturalWidth || rect.width,
      naturalHeight: img.naturalHeight || rect.height,
      rotation: 0,
      opacity: 1,
      // Replaced by an asset: reference once the pixels are in IndexedDB.
      src: opts.keepSrc ?? dataUrl,
      createdAt: Date.now(),
      updatedAt: Date.now(),
    };
    addElement(newImage, true);
    setTool("select");
    setSelected([imageId]);
    if (opts.edit) setImageStudioId(imageId);
    if (opts.keepSrc) return;
    storeImage(dataUrl).then((src) => {

```

```

        if (src !== dataUrl) updateElement(imageId, { src } as never, false);
    });
};
img.src = dataUrl;
};
const insertImageRef = useRef(insertImage);
insertImageRef.current = insertImage;

// The toolbar's "add image" button hands the picked file over here, because
// only the canvas knows the viewport and the page it has to fit.
useEffect(() => {
    const onInsert = (e: Event) => {
        const d = (
            e as CustomEvent<{
                dataUrl?: string;
                index?: number;
                edit?: boolean;
                size?: { width: number; height: number };
            }>
        ).detail;
        const dataUrl = d?.dataUrl;
        if (!dataUrl) return;
        const opts = { stagger: d.index ?? 0, edit: !!d.edit, size: d.size };
        if (isAssetRef(dataUrl)) {
            // "Use again" from the image sheet: measure the stored pixels, keep the reference.
            loadImage(assetIdOf(dataUrl)).then((pixels) => {
                if (pixels) insertImageRef.current(pixels, null, { ...opts, keepSrc: dataUrl });
            });
            return;
        }
        insertImageRef.current(dataUrl, null, opts);
    };
    window.addEventListener("pronotes:insertimage", onInsert);
    return () => window.removeEventListener("pronotes:insertimage", onInsert);
}, []);

// Clipboard Image Paste Handler
useEffect(() => {
    const onPaste = (e: ClipboardEvent) => {
        const activeEl = document.activeElement;
        const isInput =
            activeEl?.tagName === "INPUT" ||
            activeEl?.tagName === "TEXTAREA" ||
            activeEl?.getAttribute("contenteditable") === "true";

        if (isInput) return;

        const items = e.clipboardData?.items;
        const hasImage = !!items && Array.from(items).some((it) => it.type.startsWith("image/"));
        // No picture on the system clipboard, but a capture copied here that the
        // system refused (the Android WebView does): paste that. Ours is only
        // used when the system never got it, so it cannot shadow a newer copy.
        if (!hasImage) {
            const clip = appClipForPaste();
            if (clip) {
                e.preventDefault();
                insertImageRef.current(clip.dataUrl, null, { size: clip.worldSize });
            }
            return;
        }
        if (!items) return;

        for (let i = 0; i < items.length; i++) {
            const item = items[i];
            if (item.type.startsWith("image/")) {
                e.preventDefault();
                const file = item.getAsFile();
                if (!file) continue;

                const reader = new FileReader();
                reader.onload = (event) => {
                    const dataUrl = event.target?.result as string;
                    if (!dataUrl) return;

                    // Paste goes where the pointer is, if it is over the canvas.
                    let at: { x: number; y: number } | null = null;
                    if (lastPointerPosRef.current && containerRef.current) {
                        const rect = containerRef.current.getBoundingClientRect();

```

```

        const { clientX, clientY } = lastPointerPosRef.current;
        if (
            clientX >= rect.left &&
            clientX <= rect.right &&
            clientY >= rect.top &&
            clientY <= rect.bottom
        ) {
            at = getWorldPoint(clientX, clientY);
        }
        insertImageRef.current(dataUrl, at);
    };
    reader.readAsDataURL(file);
    break;
}
}
};

window.addEventListener("paste", onPaste);
return () => {
    window.removeEventListener("paste", onPaste);
};
}, [camera, getWorldPoint, addElement, setSelected]);

// Wheel zoom and pan.
//
// Attached as a NON-PASSIVE native listener rather than React's onWheel:
// React registers wheel handlers passively, so preventDefault() inside
// onWheel is silently ignored (the console fills with "Unable to
// preventDefault inside passive event listener invocation"). Without it, a
// trackpad pinch or Ctrl+wheel zooms the whole BROWSER PAGE on top of the
// canvas.
useEffect(() => {
    const node = containerRef.current;
    if (!node) return;

    const onWheel = (e: WheelEvent) => {
        e.preventDefault();
        // Decide the DEVICE across a run of events and hold that decision, so a
        // single ambiguous event mid-gesture cannot flip scrolling into zooming.
        wheelDeviceRef.current = observeWheel(wheelDeviceRef.current, {
            deltaX: e.deltaX,
            deltaY: e.deltaY,
            deltaMode: e.deltaMode,
            time: e.timeStamp || Date.now(),
        });
        const source = wheelSourceOf(wheelDeviceRef.current, e);
        const action = resolveWheelAction(e, touchSettings.wheelMode ?? "auto", source);
        if (action.kind === "zoom") {
            const pt = getContainerPoint(e.clientX, e.clientY);
            zoomTo(cameraRef.current.zoom * action.factor, pt.x, pt.y, viewportRef.current);
        } else {
            // Scroll and drag deltas have opposite signs – see cameraPanForScroll.
            // The touch and space-drag callers below pass their deltas unchanged
            // because those really are drag deltas.
            const pan = cameraPanForScroll(action);
            panCamera(pan.dx, pan.dy);
        }
    };

    node.addEventListener("wheel", onWheel, { passive: false });
    return () => node.removeEventListener("wheel", onWheel);
    // Camera and viewport are read through refs so this listener is attached
    // once instead of being torn down and rebuilt on every zoom step.
}, [touchSettings.wheelMode, getContainerPoint, zoomTo, panCamera]);

// Find topmost element under world coordinate
const findElementAt = useCallback(
    (wx: number, wy: number, isTouch = false, customTol?: number): WhiteboardElement | null => {
        const tol = customTol ?? (tool === "eraser" ? Math.max((style.eraserSize || 24) / 2, HIT_THRESHOLD) : (isTouch ? 12 : HIT_THRESHOLD));
        // Locked-items mode: only locked things answer a touch (hitTestElement
        // skips locked elements, so each is tested as if unlocked).
        const lockedPick = lockedMode && (tool === "select" || tool === "lasso");
        for (let i = elements.length - 1; i >= 0; i--) {
            const el = elements[i];
            if (lockedPick) {
                if (el.locked && el.type !== "connection" && hitTestElement({ ...el, locked: false }, wx, wy, tol, e

```

```

        lements)) {
            return el;
        }
        continue;
    }
    if (hitTestElement(el, wx, wy, tol, elements)) {
        return el;
    }
}
return null;
},
[elements, tool, style.eraserSize, lockedMode]
);

/**
 * Erase at a world point. In "pixel" mode a stroke is split around the eraser
 * and its surviving pieces replace it; everything else (and "object" mode) is
 * deleted whole, because a shape or a text box has no partial form.
 *
 * `commit` is true only for the FIRST erase of a gesture: the drag then folds
 * into that single history entry, so one swipe is one undo rather than dozens.
 */
const eraseAt = useCallback(
    (wx: number, wy: number, isTouch: boolean, commit: boolean) => {
        // Where the eraser was at the previous event. Erasing only at the current
        // position leaves gaps whenever the drag outruns the event rate, so the
        // swept segment from there to here is what actually erases.
        const prev = lastErasePosRef.current ?? { x: wx, y: wy };
        lastErasePosRef.current = { x: wx, y: wy };
        const radius = Math.max((style.eraserSize || 24) / 2, 1);

        if (style.eraserMode === "object") {
            const hit = findElementAt(wx, wy, isTouch);
            if (hit) deleteElements([hit.id], commit);
            return;
        }

        // Pixel mode: cut through every stroke under the eraser, not just the
        // topmost one – the eraser is a physical brush, not a picker.
        // The LIVE elements, not this render's: an S Pen reports several moves
        // between renders, and each cut must start from the previous one's
        // result - from a stale copy, a later cut rebuilt the stroke from its
        // uncut original and brought erased ink back (doubled).
        const live = getBoardElements(
            useWhiteboard.getState().boards.find((b) => b.id === activeBoardId) ?? null
        );
        const strokesHit = live.filter((el): el is StrokeElement => {
            if (el.type !== "stroke" || el.locked) return false;
            // A turned stroke's box says little about where its ink is; let the
            // precise test below decide.
            if (el.rotation) return true;
            const b = getElementBounds(el);
            // Cheap reject against the swept segment's bounding box, expanded by
            // the radius – must match what eraseFromStroke actually tests.
            const minX = Math.min(prev.x, wx) - radius;
            const maxX = Math.max(prev.x, wx) + radius;
            const minY = Math.min(prev.y, wy) - radius;
            const maxY = Math.max(prev.y, wy) + radius;
            return !(maxX < b.x || minX > b.x + b.width || maxY < b.y || minY > b.y + b.height);
        });

        let didCommit = false;
        for (const stroke of strokesHit) {
            // Cut the line you SEE: the smoothed points, and for a turned stroke
            // its points turned into place. Cutting the raw points and letting
            // each piece be re-smoothed (and re-turned about its own centre)
            // made the writing beside every cut bend and shift.
            let pts = settledStrokePoints(stroke);
            if (stroke.rotation) {
                const c = getElementCenter(stroke);
                pts = pts.map(([px, py, pr]) => {
                    const r = rotatePoint(stroke.x + px, stroke.y + py, c.x, c.y, stroke.rotation);
                    return [r.x - stroke.x, r.y - stroke.y, pr] as [number, number, number];
                });
            }
            const runs = eraseFromStroke(
                pts,

```

```

    prev.x - stroke.x,
    prev.y - stroke.y,
    wx - stroke.x,
    wy - stroke.y,
    radius
  );
  if (runs === null) continue; // untouched

  if (!didCommit && commit) {
    commitHistory();
    didCommit = true;
  }

  if (runs.length === 0) {
    deleteElements([stroke.id], false);
    continue;
  }

  // Re-origin each piece so its points stay relative to its own element.
  // An end the eraser made is blunt; the stroke's own ends keep theirs.
  const first = pts[0];
  const last = pts[pts.length - 1];
  const same = (a: number[], b: number[]) => Math.abs(a[0] - b[0]) < 1e-6 && Math.abs(a[1] - b[1]) < 1e-6;
  const pieces: StrokeElement[] = runs.map((run, i) => {
    const ox = run[0][0];
    const oy = run[0][1];
    return {
      ...stroke,
      id: i === 0 ? stroke.id : nanoid(8),
      x: stroke.x + ox,
      y: stroke.y + oy,
      rotation: 0,
      points: run.map(([px, py, pr]) => [px - ox, py - oy, pr] as [number, number, number]),
      settled: true,
      cutStart: same(run[0], first) ? stroke.cutStart : true,
      cutEnd: same(run[run.length - 1], last) ? stroke.cutEnd : true,
      path: undefined,
      updatedAt: Date.now(),
    };
  });

  updateElement(pieces[0].id, pieces[0], false);
  if (pieces.length > 1) addElements(pieces.slice(1), false);
}

// Drawn shapes and lines erase whole (they have no partial form). Text
// boxes, notes and pictures are not ink: rubbing out writing near them
// must never delete them - "Whole" mode is there for that.
const other = findElementAt(wx, wy, isTouch);
if (other && other.type !== "stroke" && other.type !== "text" && other.type !== "sticky" && other.type !== "image") {
  deleteElements([other.id], commit && !didCommit);
}
},
[
  activeBoardId,
  style.eraserSize,
  style.eraserMode,
  findElementAt,
  deleteElements,
  updateElement,
  addElements,
  commitHistory,
]
);

// Capture on the stable SVG root – capturing on e.target (a child node that can
// unmount mid-gesture when overlays re-render) silently drops the capture and
// leaves the interaction stuck.
// -----
// Panning: one camera update per FRAME
// -----
// Scrolling a notebook by finger was juddery because every single
// pointermove did TWO React renders – `panCamera` (a store write) and
// `setInteraction` (component state) – and a tablet digitiser reports
// 120-240 moves a second. That is up to ~480 renders per second of a tree
// holding every element, every neighbouring page and any PDF layer, so the

```

```

// camera fell behind the finger and the page appeared to drag.
//
// Now a move only writes to refs. A single RAF flush applies the summed
// delta once per displayed frame, which is the most the screen can show
// anyway, and the gesture's last position lives in a ref so it never
// re-renders at all. The total travel is unchanged: the deltas are summed,
// not sampled, so the page still tracks the finger exactly 1:1.
//
// The coast after release lives in startMomentum below. An earlier version
// overshot (1242px for an 800px drag on a Galaxy Tab S9) because it averaged
// velocity over the whole drag; the current one uses Android's 100 ms / 40 ms
// rules, so a drag that has stopped does not throw.

/** Where the pan gesture last was, in container coords. Ref, not state. */
const panLastRef = useRef<{ x: number; y: number } | null>(null);
/** Movement seen since the last frame flush. */
const panAccumRef = useRef<{ dx: number; dy: number }>({ dx: 0, dy: 0 });
const panRafRef = useRef<number | null>(null);

// --- The camera leaves React while a finger is down -----
// Coalescing to one store write per frame was not enough. Each write still
// re-rendered a 2,500-line component holding every element, every
// neighbouring sheet and the PDF layer, and `dumpsys gfxinfo` showed the
// symptom precisely: frames were being produced on time, but "Number High
// input latency: 230" — the main thread was busy, so touches were handled
// late. That is what "slow" felt like.
//
// During a pan the camera therefore moves by writing the transform straight
// onto the SVG nodes: zero React renders, two attribute writes a frame. The
// store is told once, when the finger lifts, so undo, autosave, the minimap
// and everything else see exactly one camera change per gesture.
const worldRef = useRef<SVGGElement | null>(null);
const boardPaperRef = useRef<SVGPatternElement | null>(null);
/** The camera as the DOM currently shows it, while a pan is in flight. */
const liveCamRef = useRef<{ x: number; y: number } | null>(null);
/** Whole-gesture screen delta, committed to the store on release. */
const panTotalRef = useRef<{ dx: number; dy: number }>({ dx: 0, dy: 0 });

/** Writes the live camera onto the DOM. Must mirror what the JSX renders. */
const applyLiveTransform = useCallback(() => {
  const cam = liveCamRef.current;
  const g = worldRef.current;
  if (!cam || !g) return;
  const z = cameraRef.current.zoom;
  g.setAttribute(
    "transform",
    `translate(${viewportRef.current.width / 2}, ${viewportRef.current.height / 2}) scale(${z}) translate(${
      -cam.x}, ${-cam.y})`
  );
  // The whiteboard's paper pattern is painted in SCREEN space, so it has to
  // be slid by hand too or the dots sit still while the content moves.
  const pat = boardPaperRef.current;
  if (pat) {
    pat.setAttribute("x", String(viewportRef.current.width / 2 - cam.x * z));
    pat.setAttribute("y", String(viewportRef.current.height / 2 - cam.y * z));
  }
}, []);

/**
 * Renders the live in-flight stroke directly onto the SVG <path> DOM node.
 * Incorporates transient hardware predictions (R1-DEF-04) and bypasses React reconciliation.
 */
const applyLiveDraftStroke = useCallback(() => {
  const live = liveDraftStrokeRef.current;
  const pathEl = liveDraftPathRef.current;
  const groupEl = liveDraftGroupRef.current;
  if (!live || !pathEl || !groupEl) return;

  const pred = predictedPointsRef.current;
  const renderPoints = pred.length > 0 ? [...live.points, ...pred] : live.points;

  const tempStroke: StrokeElement = {
    id: live.id,
    type: "stroke",
    x: live.originX,
    y: live.originY,
    rotation: 0,
    opacity: 1,
  }

```